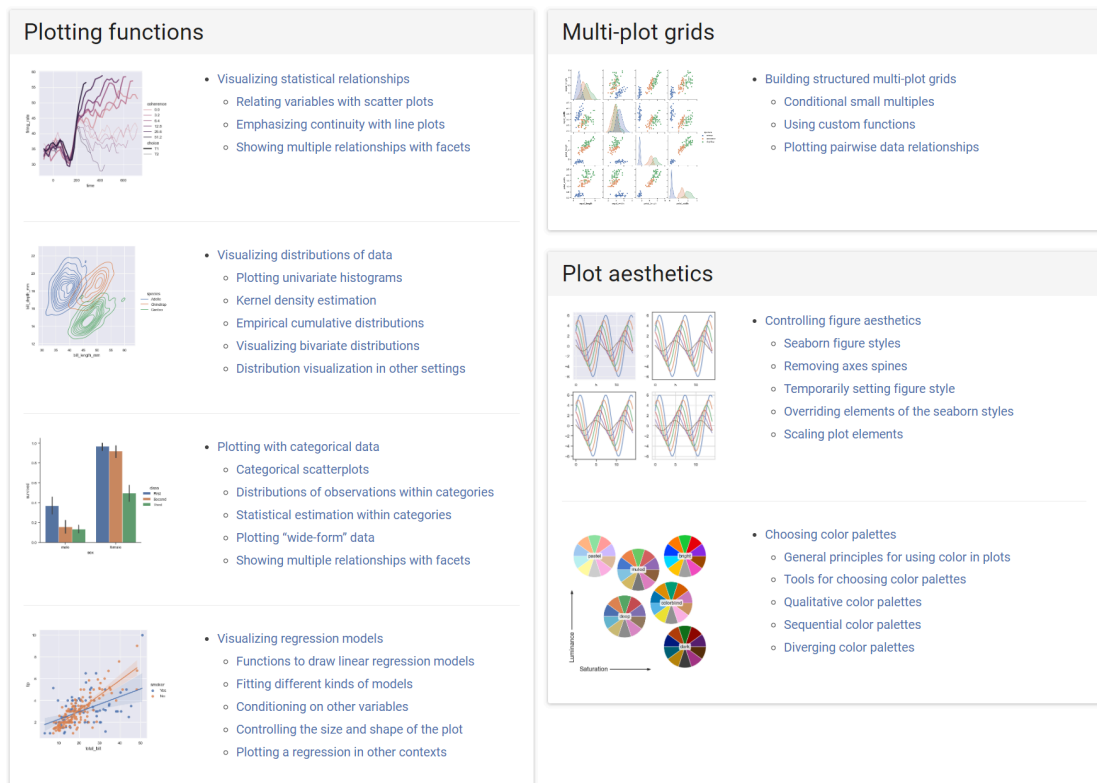


Seaborn

- <https://seaborn.pydata.org/>
- matplotlib의 기능에 스타일을 확장한 파이썬 시각화 도구
- high-level interface
- 패키지는 설치해야 함(pip install seaborn)
 - 아나콘다에 seaborn 패키지 포함되어 있으므로 이미 설치되어 있음
- seaborn을 사용하기 위해서는 matplotlib가 import 되어 있어야 함



- <https://seaborn.pydata.org/examples/index.html>
- <https://seaborn.pydata.org/tutorial.html>

패키지 импорт

```
In [1]: import numpy as np
import pandas as pd
import matplotlib as mp
import matplotlib.pyplot as plt
import seaborn as sns

print(f'Numpy:{np.__version__}, Pandas:{pd.__version__}, Matplotlib:{mp.__version__}, Seaborn:{sns.__version__}')

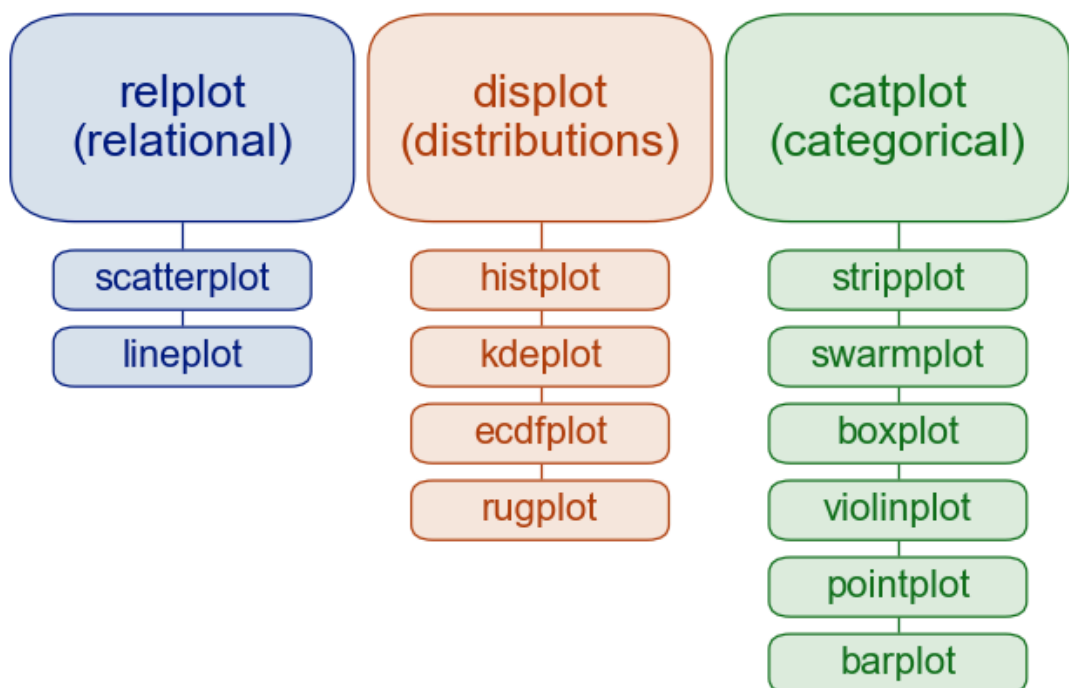
Numpy:2.5.2, Pandas:3.0.5, Matplotlib:3.11.1, Seaborn:0.13.2
```

```
In [2]: # 한글문자 출력을 위한 설정
plt.rcParams['axes.unicode_minus'] = False
```

```
plt.rcParams['font.family']='Malgun Gothic' # 'AppleGothic', 'NanumGothic'
```

지원되는 플롯 유형

- Relational plots
- Distribution plots
- Categorical plots
- Regression plots
- Multi-plot grids
- Figure theming
- Color palettes



시각화의 목적에 따른 플롯

- 데이터 분포 파악
- 데이터 관계 파악
- 집단간 비교
- 시간에 따른 변화

1. 데이터 분포(distribution) 파악

- 데이터의 중심(대표), 산포(퍼짐), 왜도, 첨도 등의 분포 특징을 파악하기 위해 사용
- **히스토그램(histogram)**
 - matplotlib.pyplot.hist()
 - seaborn.displot(kind='hist')
 - seaborn.histplot()
 - DataFrame.plot(kind='hist')
 - DataFrame.plot.hist()
- **박스플롯(boxplot)**
 - matplotlib.pyplot.boxplot()
 - seaborn.catplot(kind='box')
 - seaborn.boxplot()
 - DataFrame.plot(kind='box')
 - DataFrame.plot.box()
 - DataFrame.boxplot()
- **커널 밀도 곡선(Kernel Density Estimation plot)**
 - seaborn.displot(kind='kde')
 - seaborn.kdeplot()
 - DataFrame.plot(kind='kde')
 - DataFrame.plot.kde()

예제 데이터. iris dataset

```
In [3]: iris = sns.load_dataset('iris')
iris.head()
```

```
Out[3]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
In [4]: iris.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal_length    150 non-null   float64
1   sepal_width     150 non-null   float64
2   petal_length    150 non-null   float64
3   petal_width     150 non-null   float64
4   species         150 non-null   str
dtypes: float64(4), str(1)
memory usage: 6.0 KB
```

```
In [5]: iris.describe()
```

```
Out[5]:
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

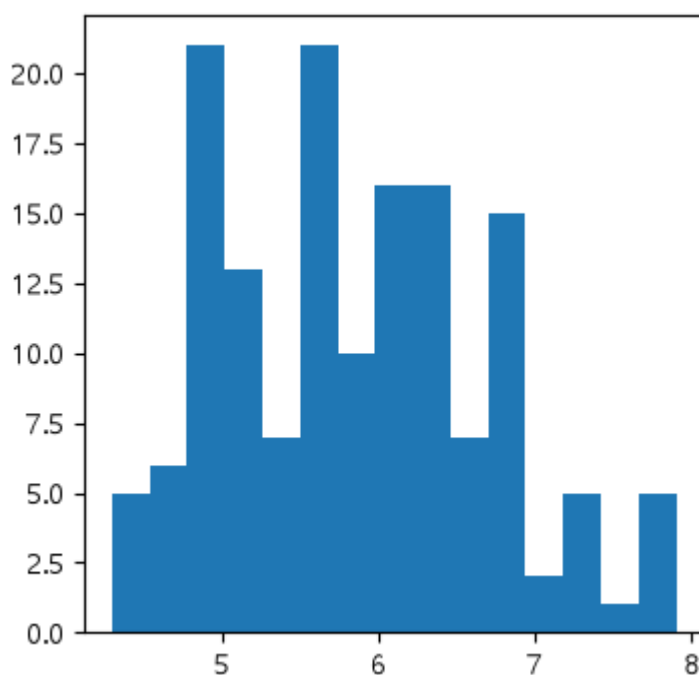
1) 히스토그램(histogram)

- 표로 되어 있는 도수 분포를 그림으로 나타낸 것
- 계급구간, 계급값, 빈도, 상대빈도

① matplotlib.pyplot.hist()

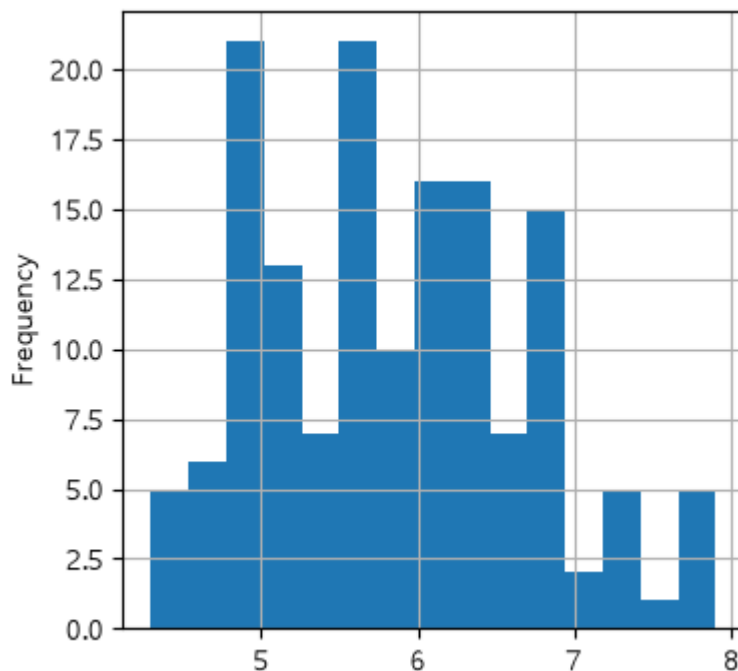
- 형식
`hist(x, bins=None, range=None, density=False, cumulative=False, ...)`

```
In [6]: plt.figure(figsize=(4,4))  
plt.hist(iris.sepal_length, bins=15)  
plt.show()
```



② pandas.DataFrame.hist()

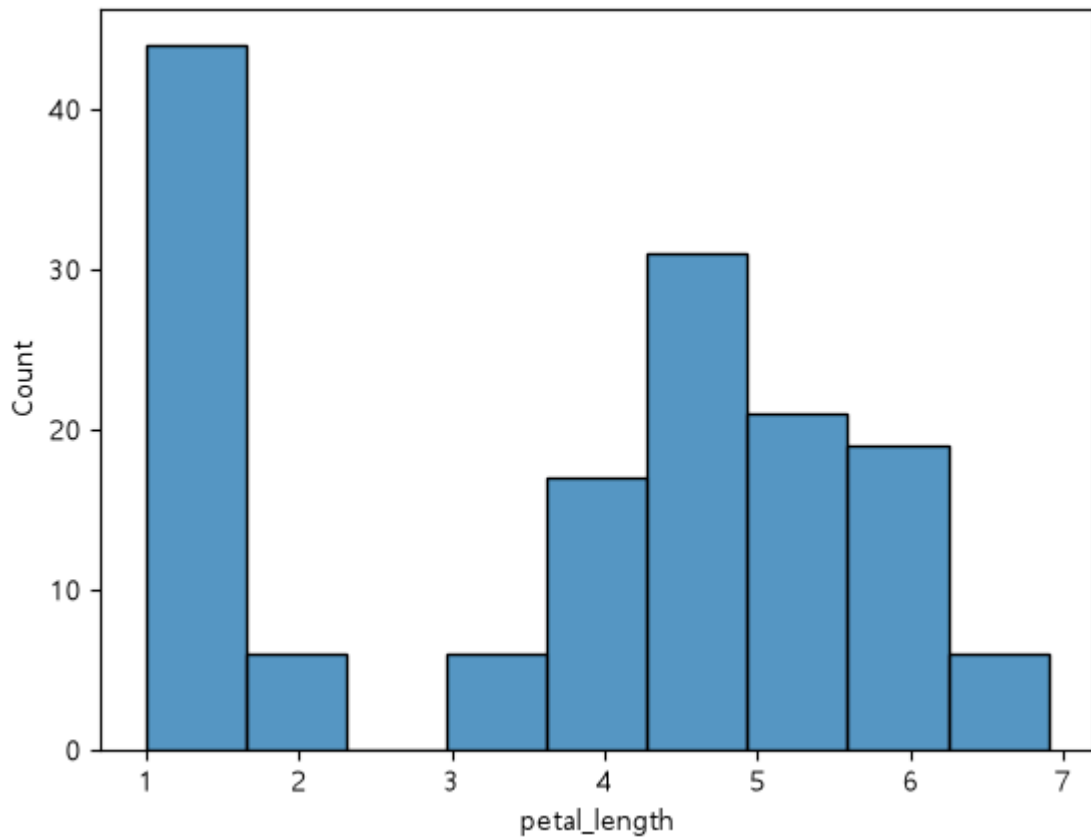
```
In [7]: # iris.sepal_length.hist(bins=15, grid=True, figsize=(4,4))
iris.sepal_length.plot(kind='hist', bins=15, grid=True, figsize=(4,4))
# kind=
# - 'line' : line plot (default)
# - 'bar' : vertical bar plot
# - 'barh' : horizontal bar plot
# - 'hist' : histogram
# - 'box' : boxplot
# - 'kde' : Kernel Density Estimation plot
# - 'density' : same as 'kde'
# - 'area' : area plot
# - 'pie' : pie plot
# - 'scatter' : scatter plot (DataFrame only)
# - 'hexbin' : hexbin plot (DataFrame only)
plt.show()
```



③ seaborn의 히스토그램 함수

- 형식1.
`seaborn.histplot(data, x, y, hue, weights, stat, bins, binwidth, kde, ...)`
 - 반환 : ax
 - <https://seaborn.pydata.org/generated/seaborn.histplot.html#seaborn.histplot>
- 형식2.
`displot(kind='hist', x, y, data, hue, ...)`
 - <https://seaborn.pydata.org/generated/seaborn.displot.html#seaborn.displot>

```
In [8]: sns.histplot(data=iris, x='petal_length')
# sns.histplot(data=iris, y='petal_length')
plt.show()
```

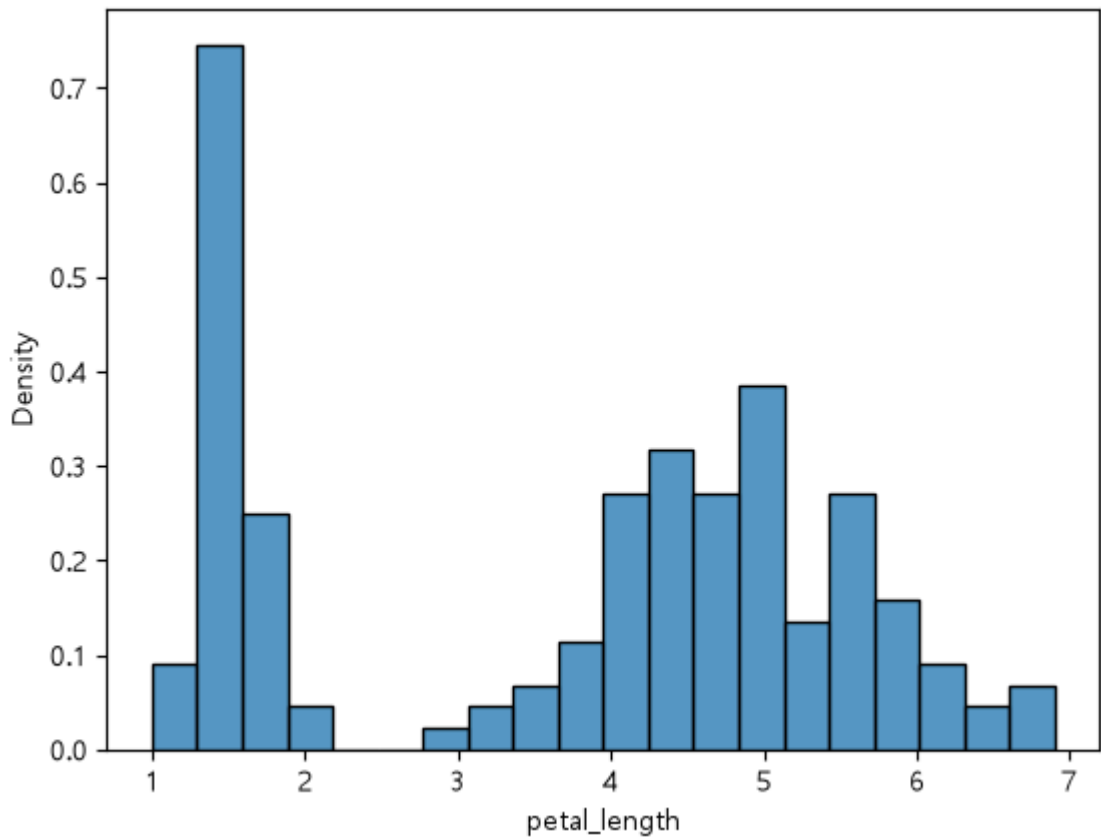


`sns.histplot(stat=)`

- `stat='count'` : 각 bin의 관측값수, 기본값
- `stat='frequency'` : 각 bin의 너비로 나눈 관측값수
- `stat='probability'` or `'proportion'` : 막대 높이의 합이 1이 되도록 정규화
- `stat='percent'` : 막대 높이 합이 100이 되도록 정규화
- `stat='density'` : 히스토그램의 전체 면적이 1이 되도록 정규화

```
In [9]: # sns.histplot(data=iris, x='petal_length', stat='frequency')
# sns.histplot(data=iris, x='petal_length', stat='probability')
# sns.histplot(data=iris, x='petal_length', stat='percent')
sns.histplot(data=iris, x='petal_length', stat='density', bins=20)

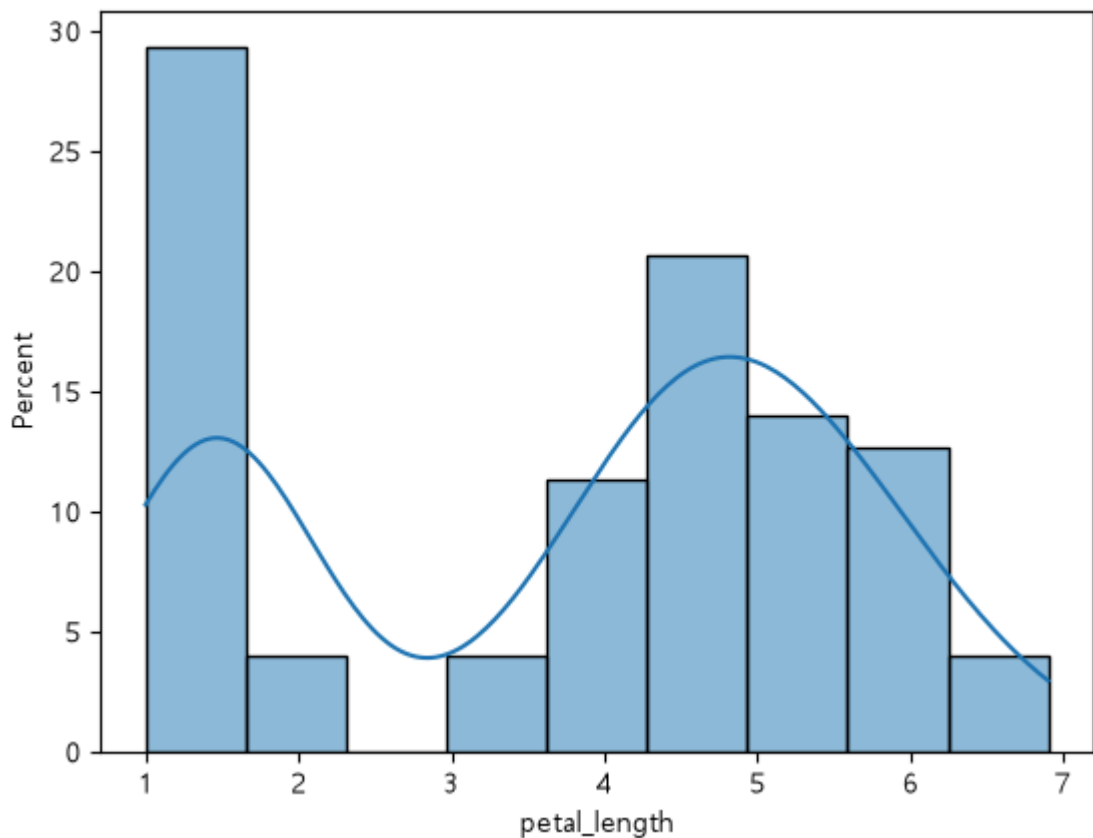
plt.show()
```



`sns.histplot(kde=True)`

- 커널밀도곡선(Kernel Density Estimation curve)을 히스토그램 위에 그림

```
In [10]: sns.histplot(data=iris, x='petal_length', stat='percent', kde=True)
plt.show()
```

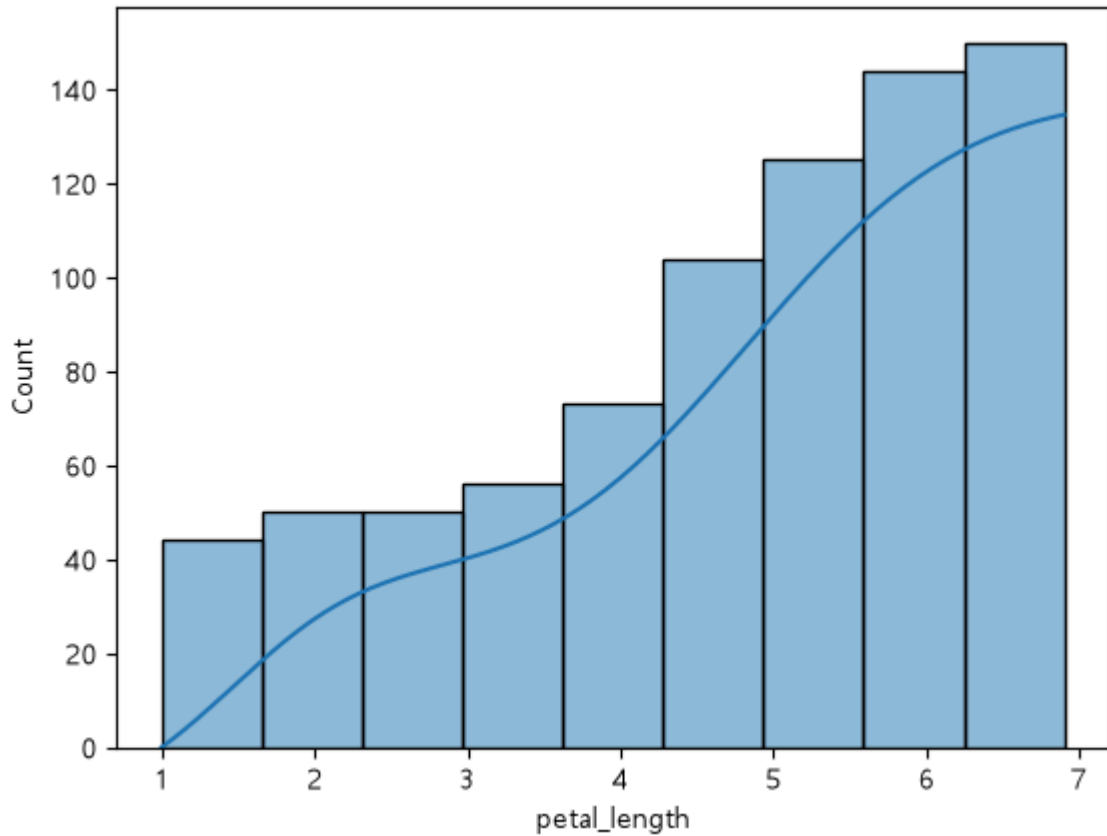


```
sns.histplot(cumulative=)
```

```
In [11]: import scipy  
         scipy.__version__
```

```
Out[11]: '1.18.1'
```

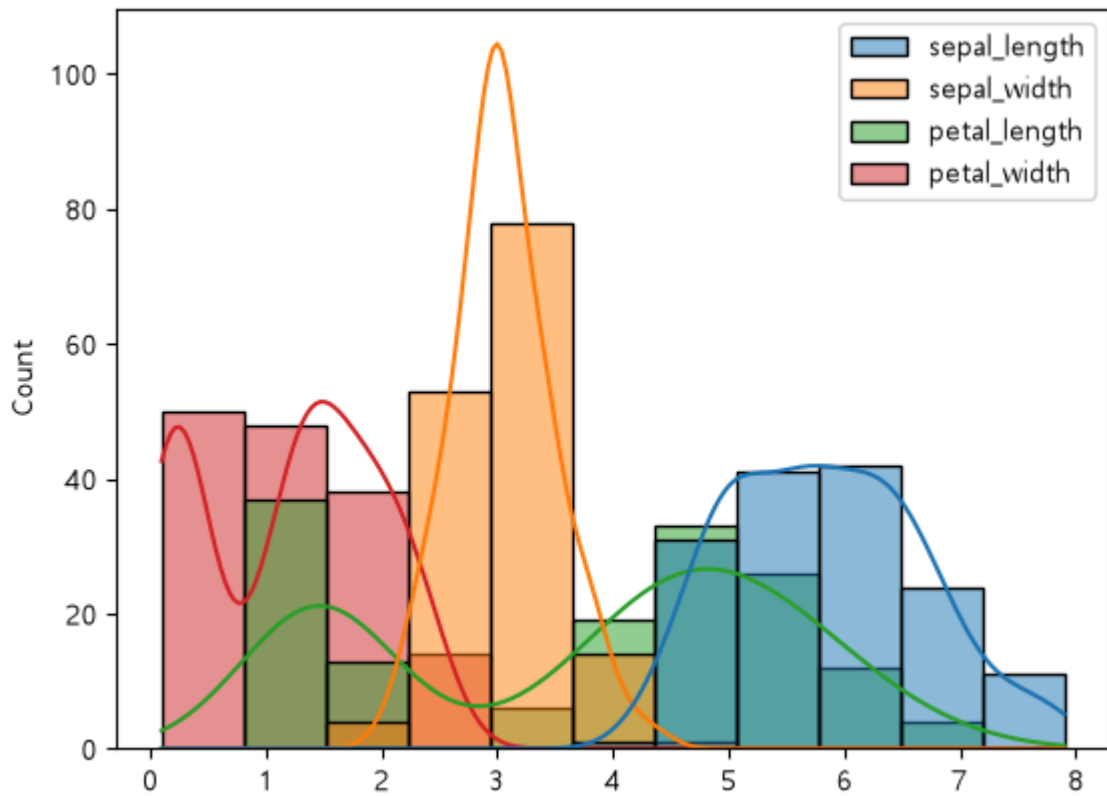
```
In [13]: sns.histplot(data=iris, x='petal_length', cumulative=True, kde=True)  
         plt.show()
```



sns.histplot() **data** 인수만 지정할 경우

- 수치형 데이터들에 대한 히스토그램을 모두 그림

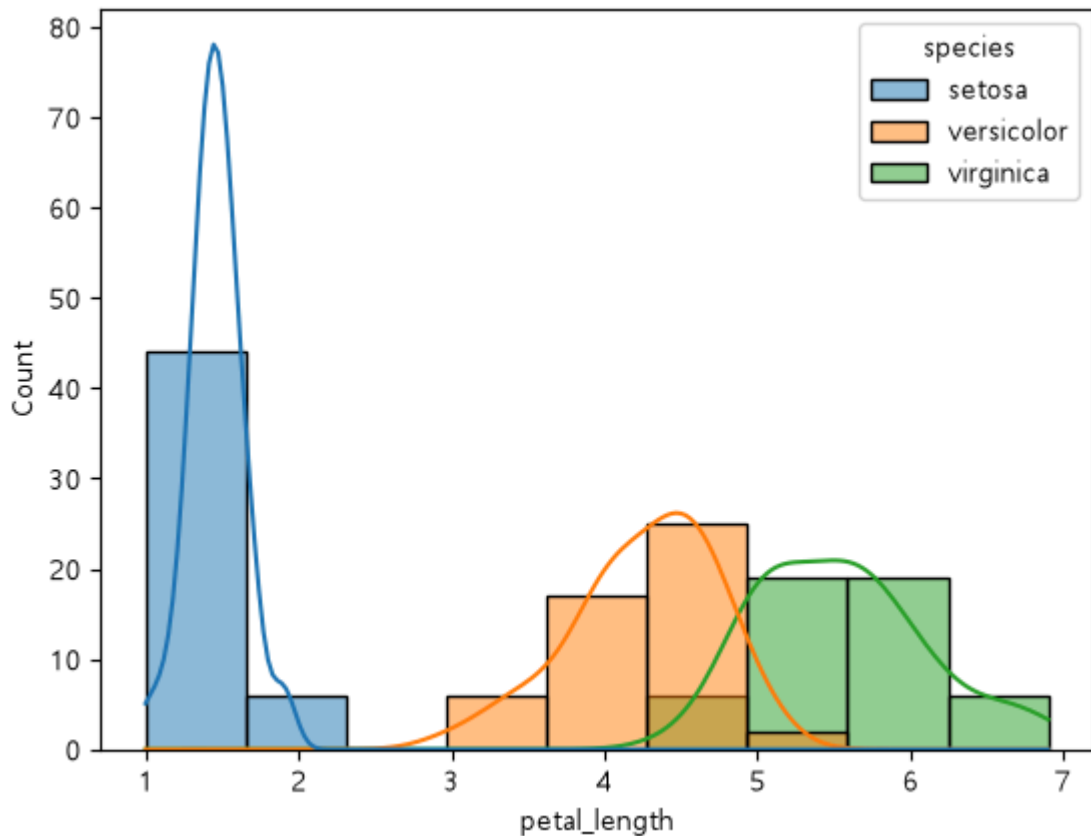
```
In [15]: sns.histplot(data=iris, kde=True)  
         plt.show()
```

`sns.histplot()`의 **hue** 인수에 범주형 데이터 지정할 경우

- 범주형 변수의 그룹별로 히스토그램을 그려 분포 비교 가능

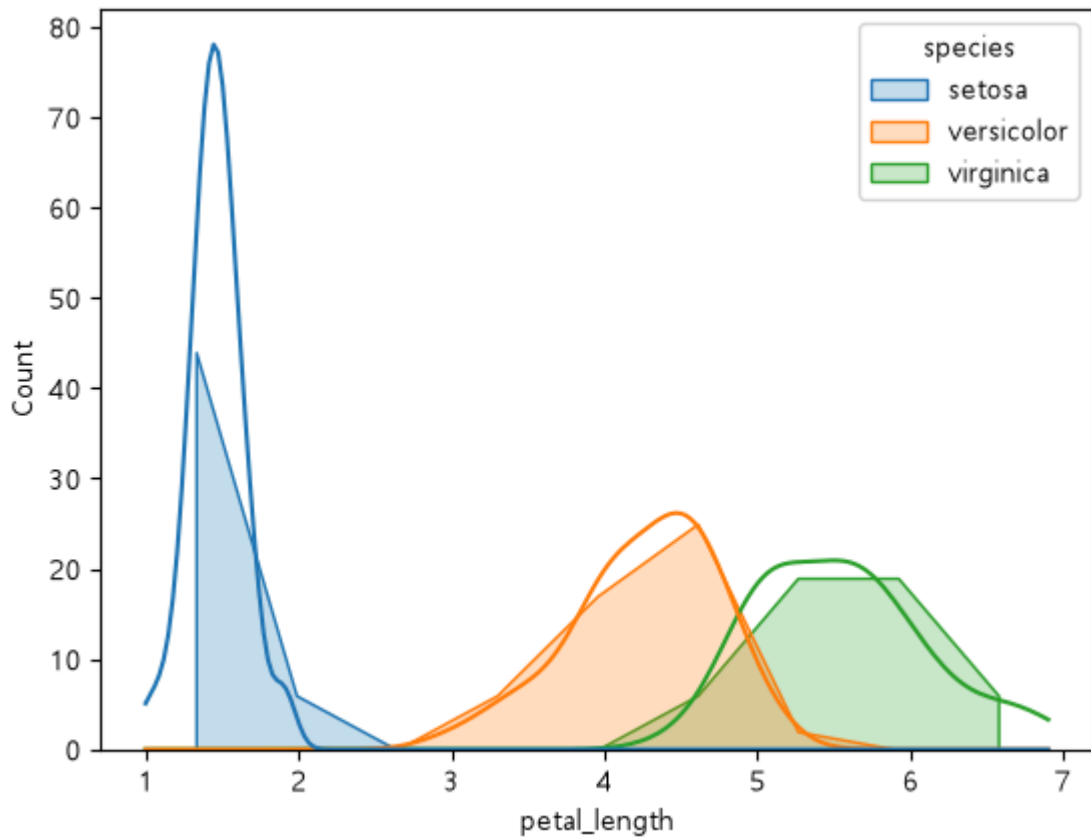
```
In [16]: # hue : 범주형 데이터를 지정
sns.histplot(data=iris, x='petal_length', kde=True, hue='species')
plt.show()
```



sns.histplot(**element**=)

- element = 'step' : 계단모양으로 그림
- element = 'poly' : 다각형으로 그림

```
In [18]: # sns.histplot(data=iris, x='petal_length', kde=True, hue='species', element='step')
sns.histplot(data=iris, x='petal_length', kde=True, hue='species', element='poly')
plt.show()
```



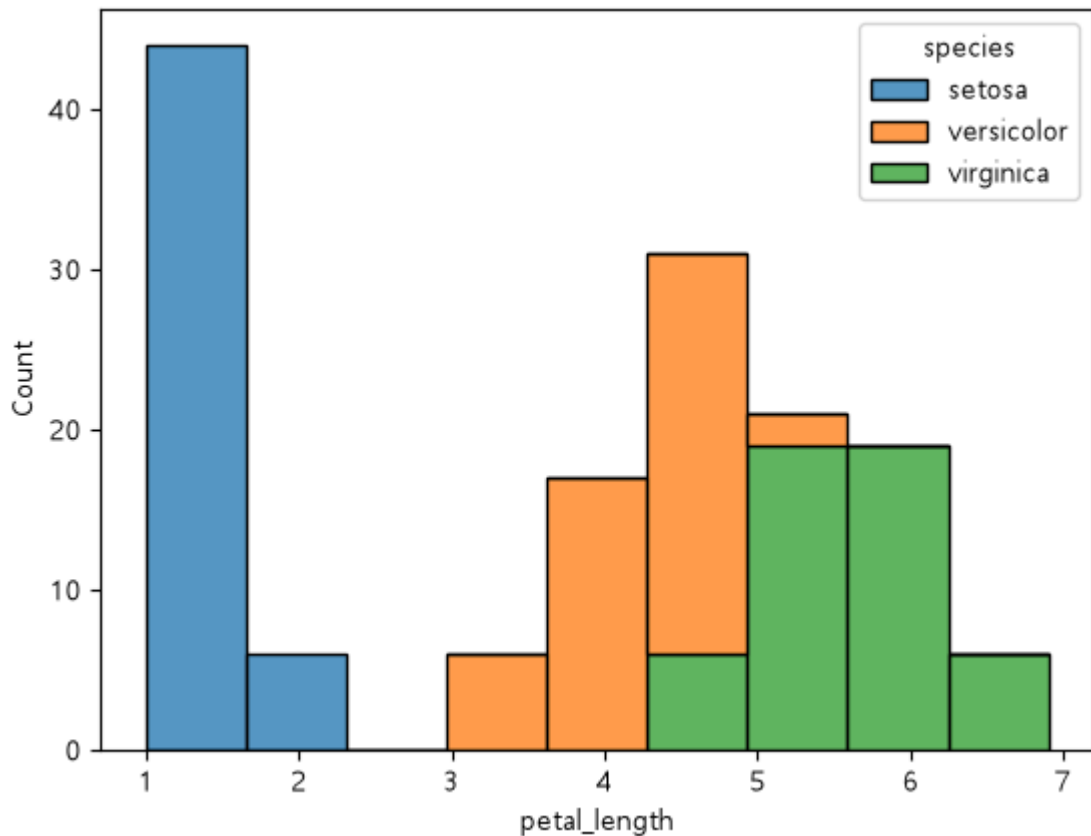
sns.histplot(**multiple**=)

두 변수가 지정된 경우만 사용

- multiple='layer', 기본값
- multiple='dodge'
- multiple='stack'
- multiple='fill'

```
In [26]: # sns.histplot(data=iris, x='petal_length', hue='species', multiple='layer')
# sns.histplot(data=iris, x='petal_length', hue='species', multiple='dodge')
sns.histplot(data=iris, x='petal_length', hue='species', multiple='stack')
# sns.histplot(data=iris, x='petal_length', hue='species', multiple='fill')

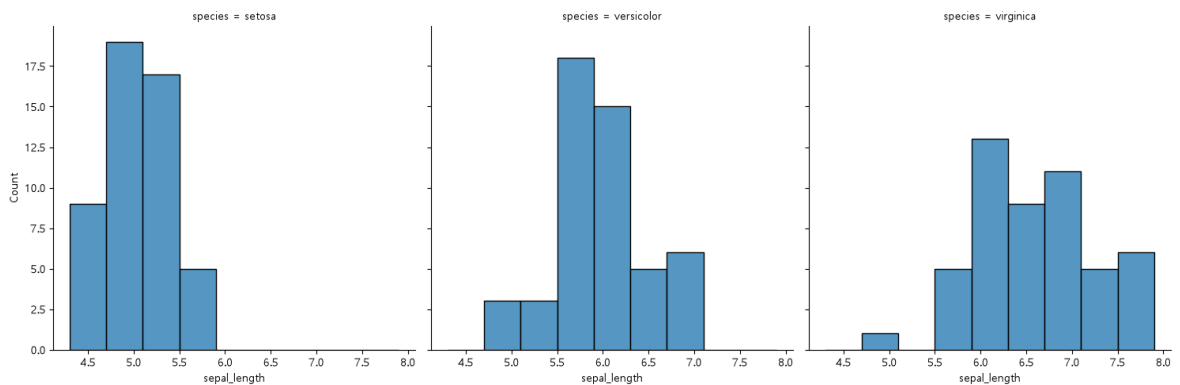
plt.show()
```



`sns.displot(kind='hist',)`

```
In [32]: # sns.displot(kind='hist', data=iris, x='sepal_length', row='species')
sns.displot(kind='hist', data=iris, x='sepal_length', col='species')

plt.show()
```



2) 박스플롯(boxplot)

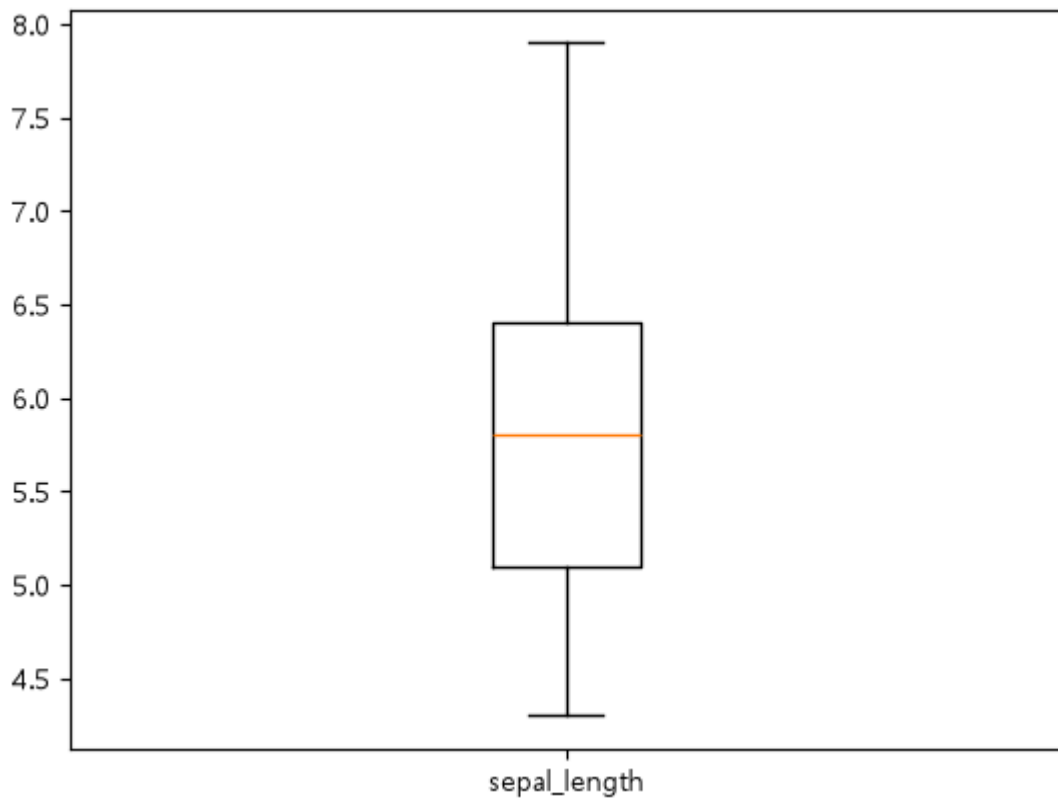
- '상자 수염 그림'(box-and-whisker plot) 또는 '상자 그림'(box plot, boxplot)이라 부름
- 수치적 자료를 표현하는 그래프
- 자료로부터 얻어낸 통계량인 5가지 요약 수치(five-number summary)를 가지고 그린다
 - 최소값
 - 제1사분위(Q1, lower quartile)
 - 제2사분위(Q2, median)

- 제3사분위(Q3, upper quartile)
- 최대값
- 이상치 확인
- 분포의 비대칭 확인

① matplotlib.pyplot의 박스플롯 함수

- 형식
`boxplot(x, notch, sym, vert, whis, widths, ...)`

```
In [34]: plt.boxplot(iris.sepal_length, tick_labels=['sepal_length'])
plt.show()
```

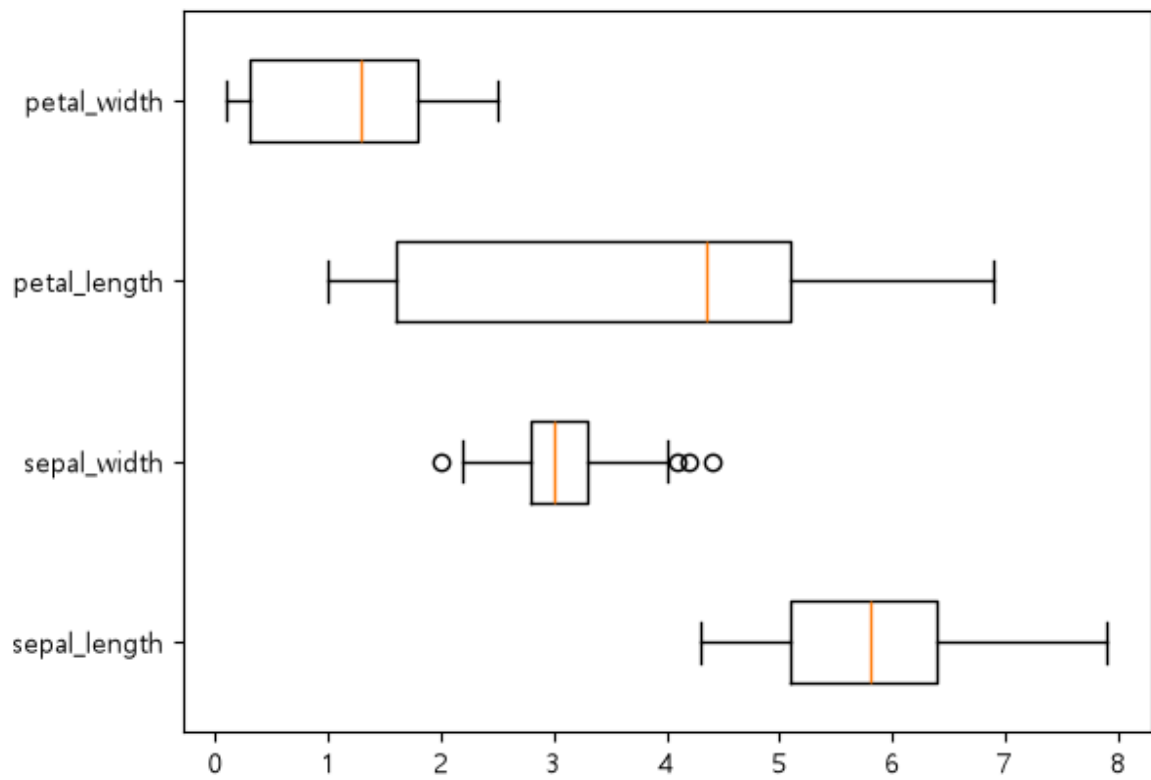


```
In [35]: iris.iloc[:, :-1].head()
```

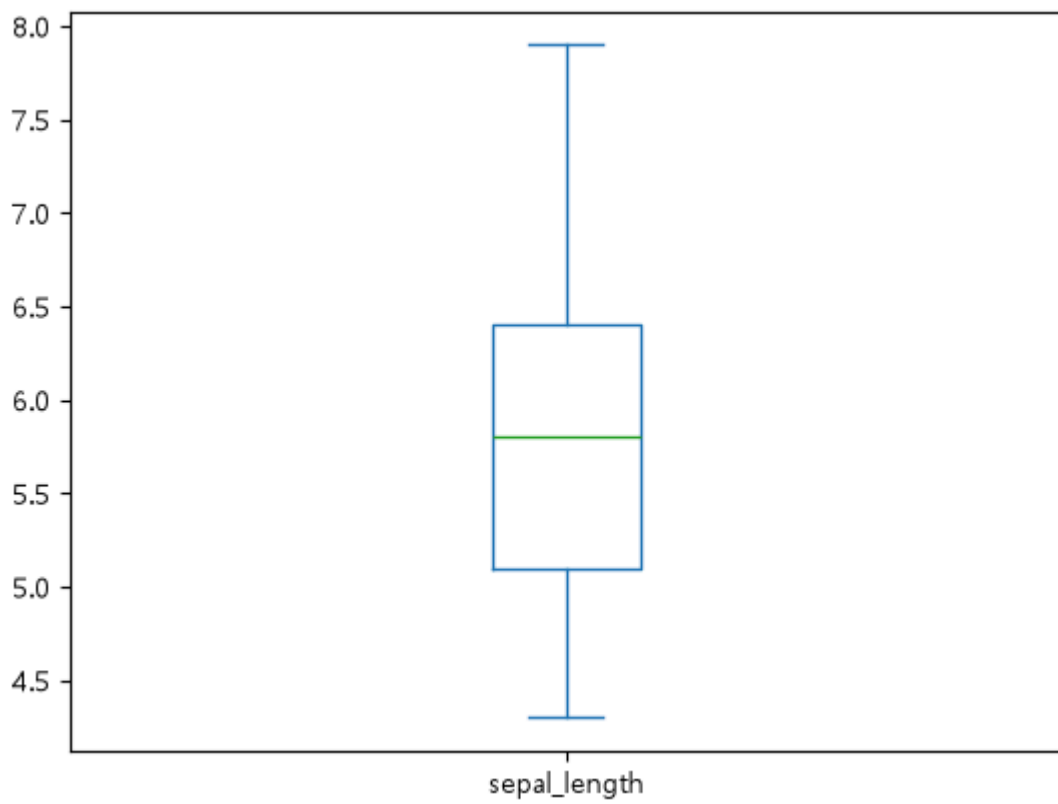
```
Out[35]:
```

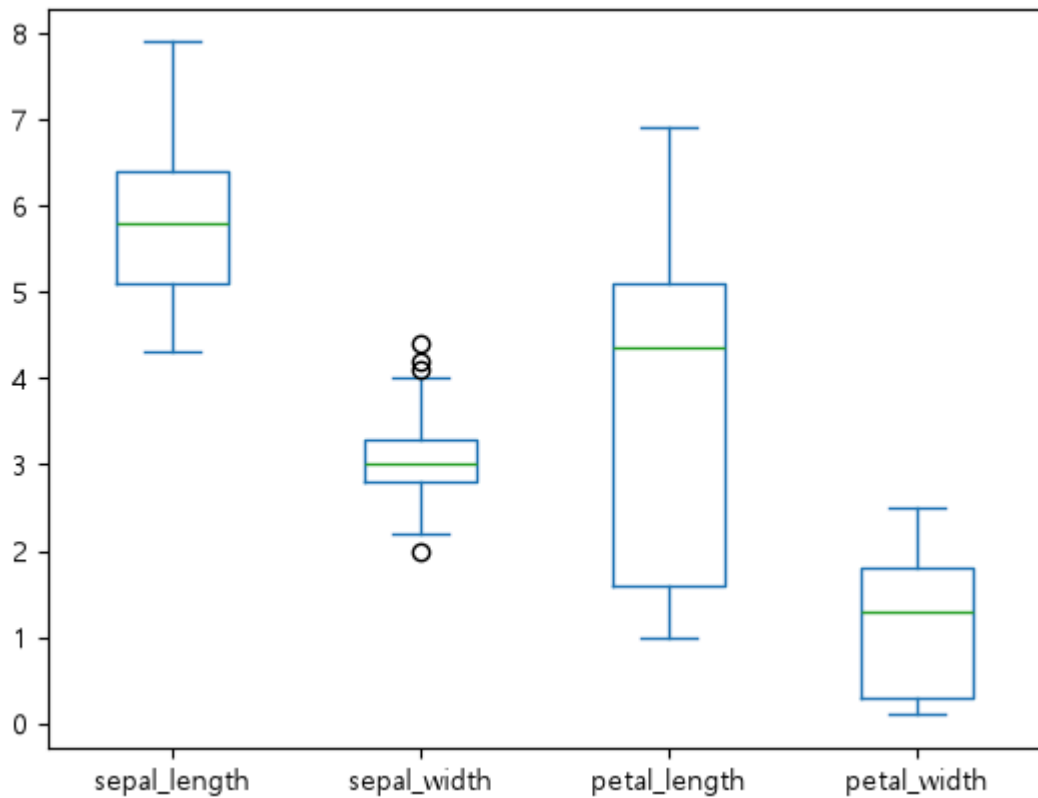
	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
In [41]: plt.boxplot(iris.iloc[:, :-1], tick_labels=iris.columns[:-1], orientation='horiz')
plt.show()
```



```
In [38]: # DataFrame.plot(kind='box')
iris['sepal_length'].plot(kind='box')
iris.iloc[:, :-1].plot(kind='box')
plt.show()
```

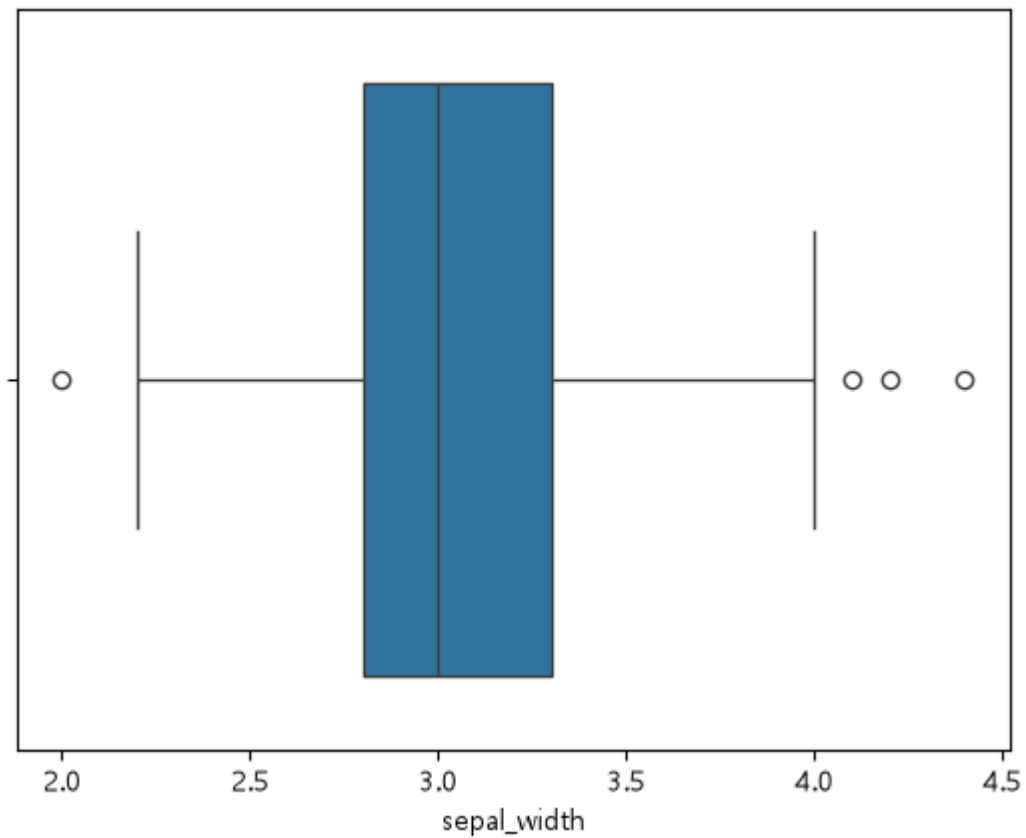




② seaborn의 박스플롯 함수

- 형식1.
`boxplot(x=None, y=None, hue=None, data=None, order=None, hue_order=None, orient=None, color=None, palette=None, saturation=0.75, width=0.8, dodge=True, fliersize=5, linewidth=None, whis=1.5, ... )`
 - 반환값 : ax (matplotlib Axes)
- 형식2.
`catplot(kind='box', x=None, y=None, hue=None, data=None, ... )`

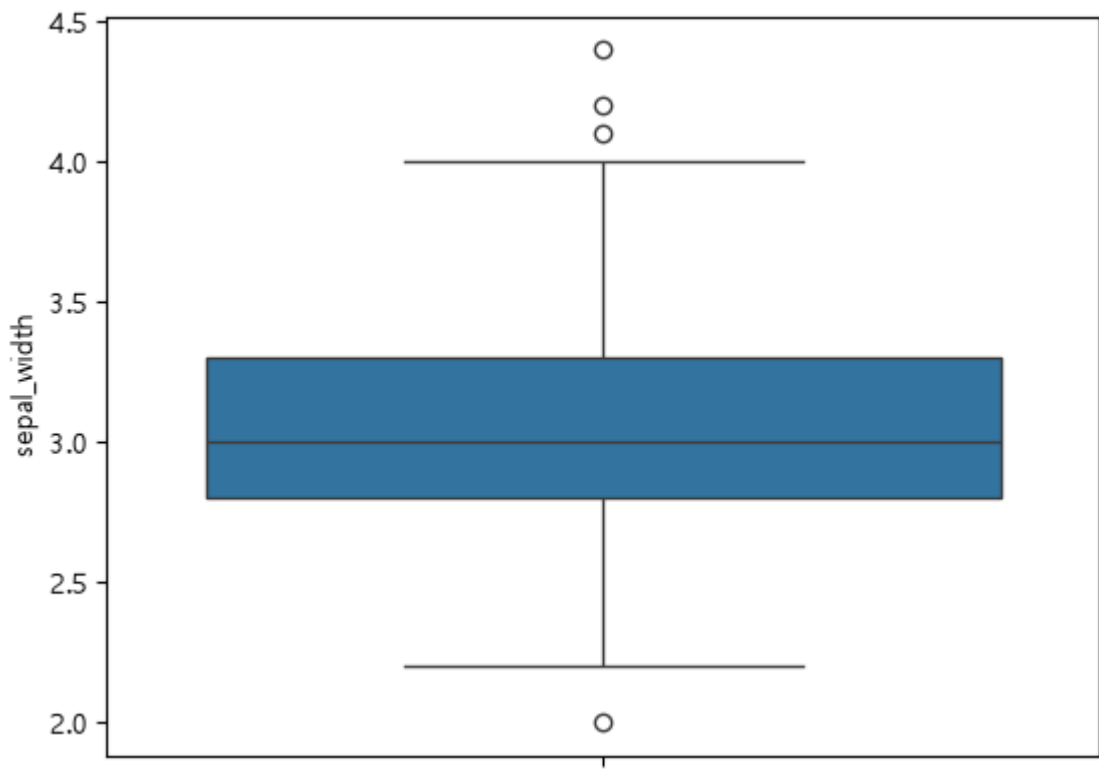
```
In [39]: sns.boxplot(data=iris, x='sepal_width')
plt.show()
```



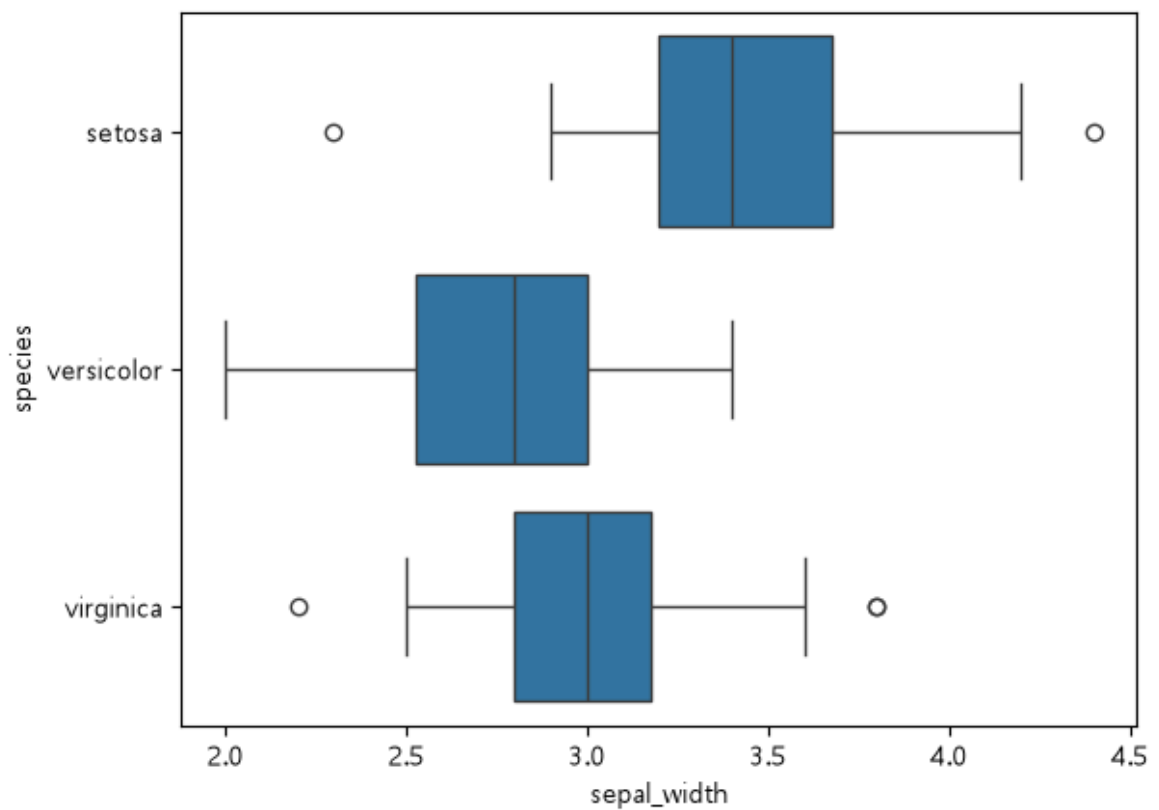
`sns.boxplot(data=, x=, y=)`

- x : 범주(수치)형 변수
- y : 수치(범주)형 변수
- data : x,y에 지정한 변수를 가진 데이터프레임

```
In [42]: sns.boxplot(data=iris, y='sepal_width')  
plt.show()
```

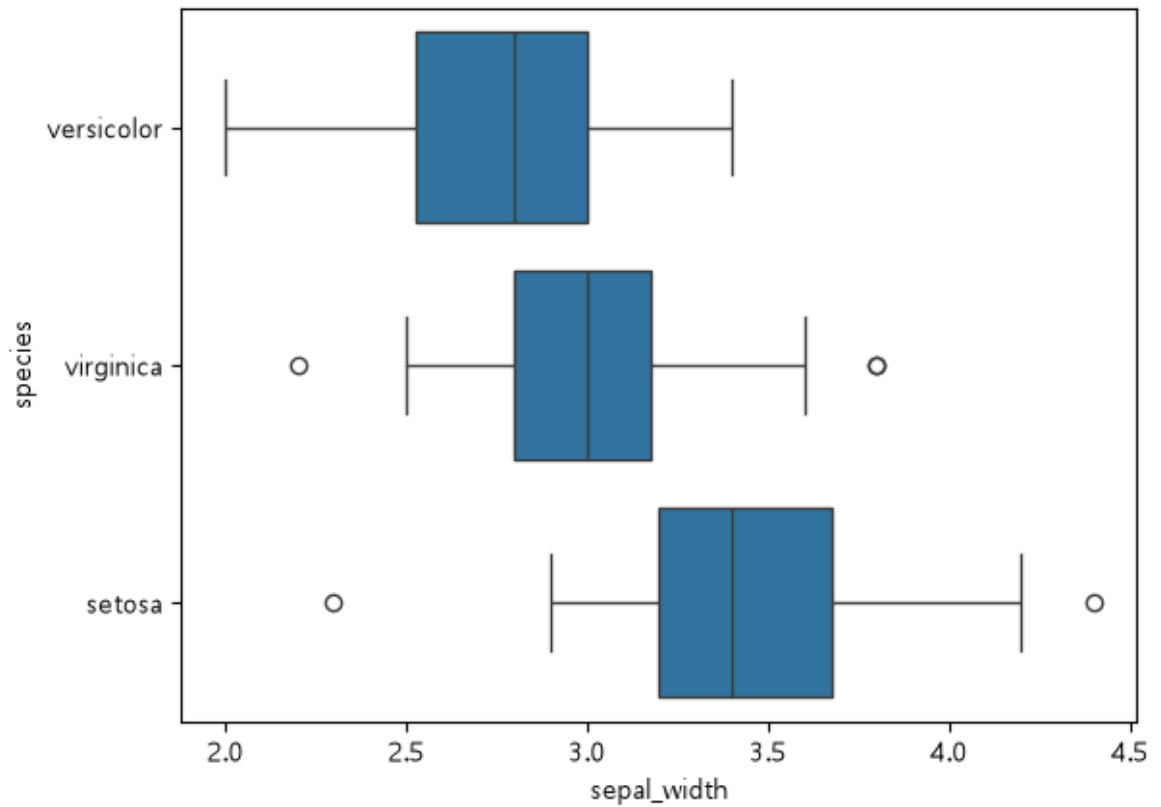


```
In [43]: sns.boxplot(data=iris, x='sepal_width', y='species')
plt.show()
```



`sns.boxplot(order=)` : 박스플롯 순서 변경

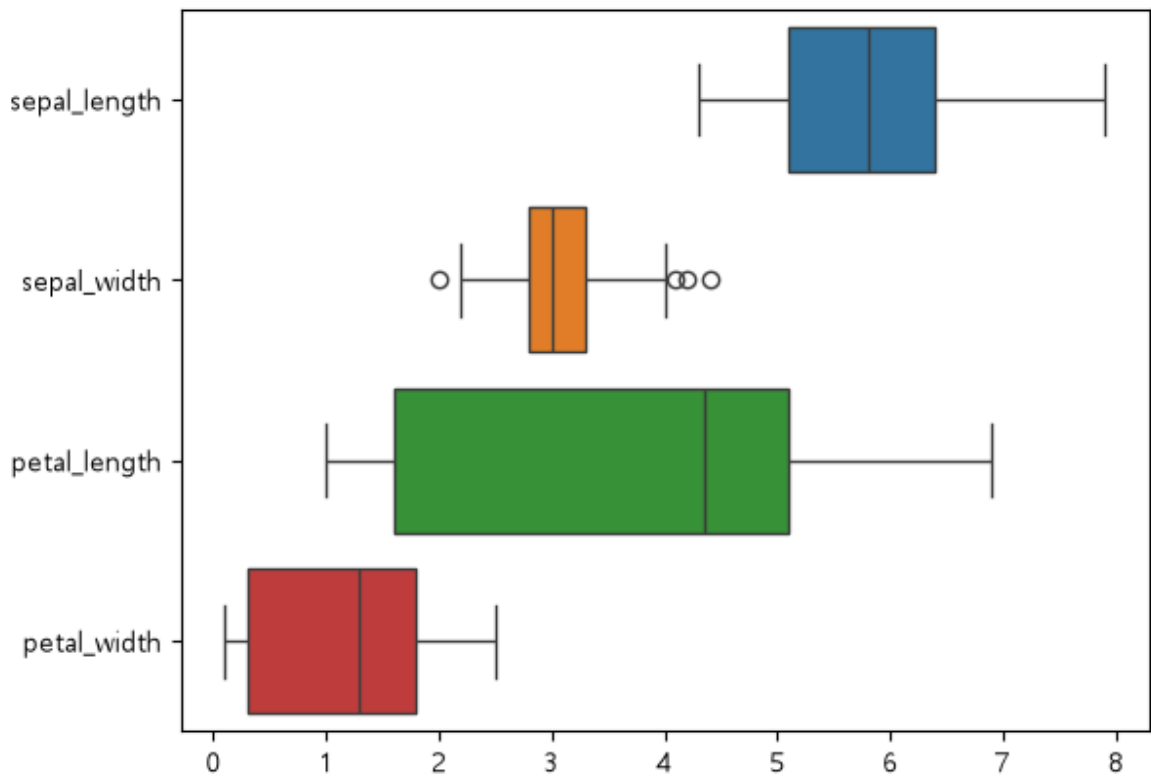
```
In [44]: sns.boxplot(data=iris, x='sepal_width', y='species', order=['versicolor', 'virginica'])
plt.show()
```

`sns.boxplot(orient=)` : 박스플롯 방향 변경

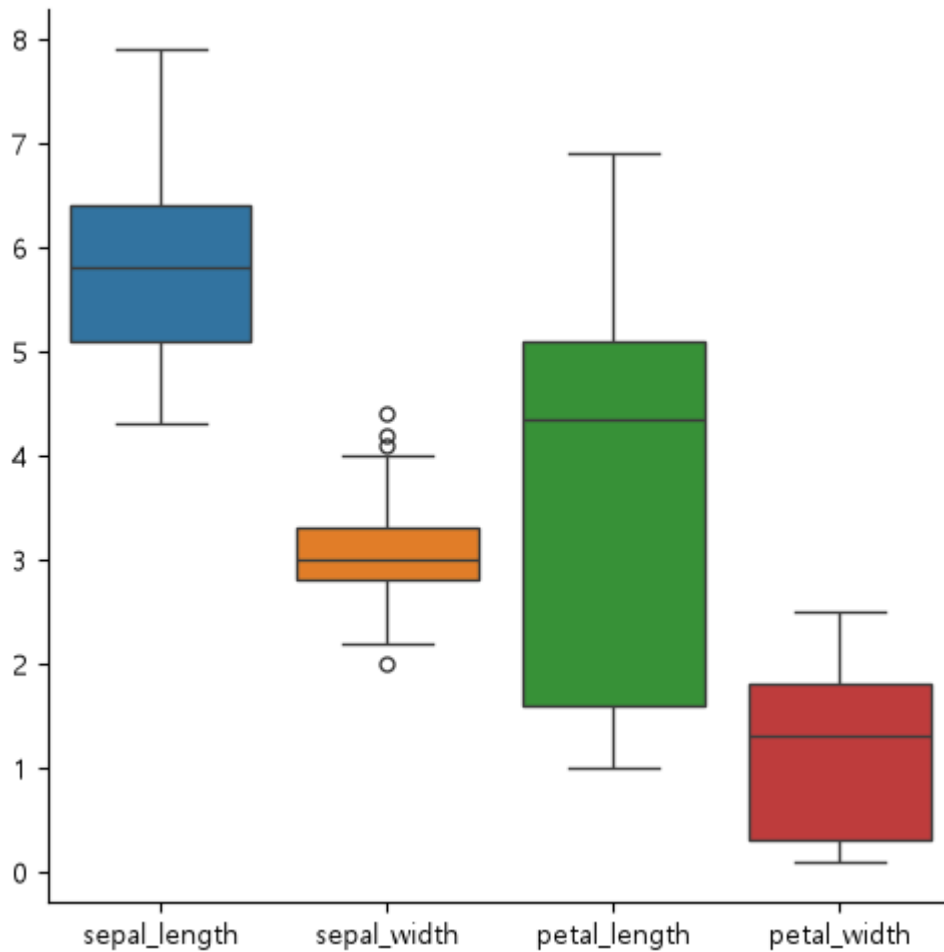
- orient = 'h' | 'v'

In [46]: `sns.boxplot(data=iris, orient='h')`
`plt.show()`



`sns.catplot(kind='box')`

```
In [48]: sns.catplot(kind='box', data=iris)
plt.show()
```

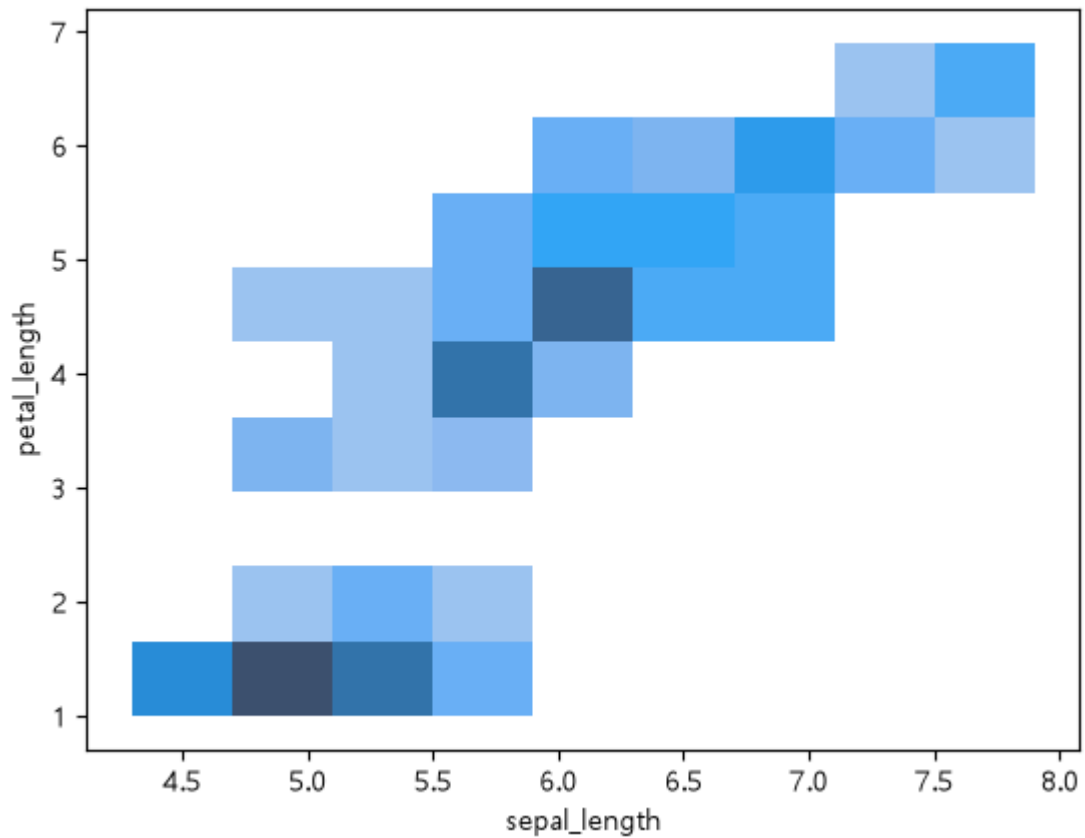


3) 이변량 분포(bivariate distribution)

- 이변량 자료 : 두 변수로 이루어진 분포

seaborn의 histplot()

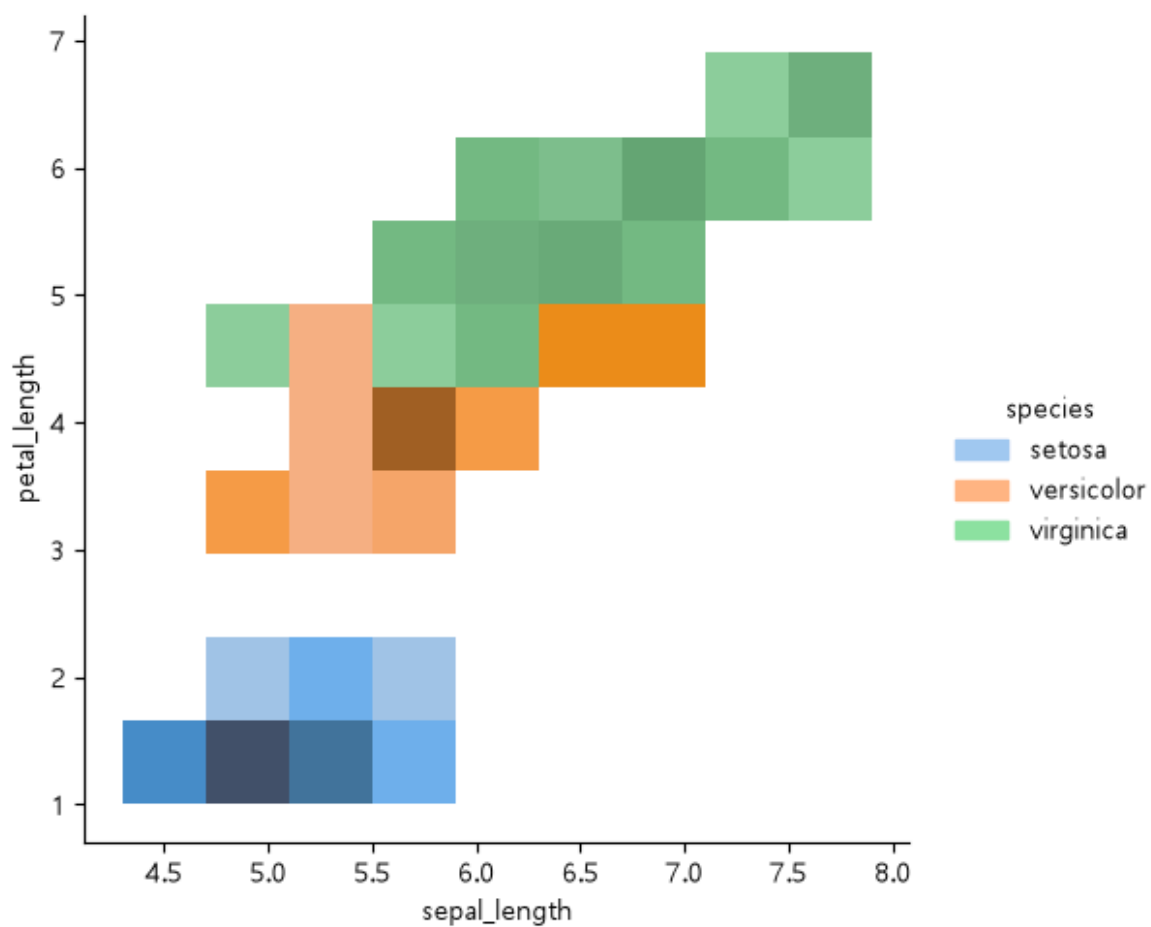
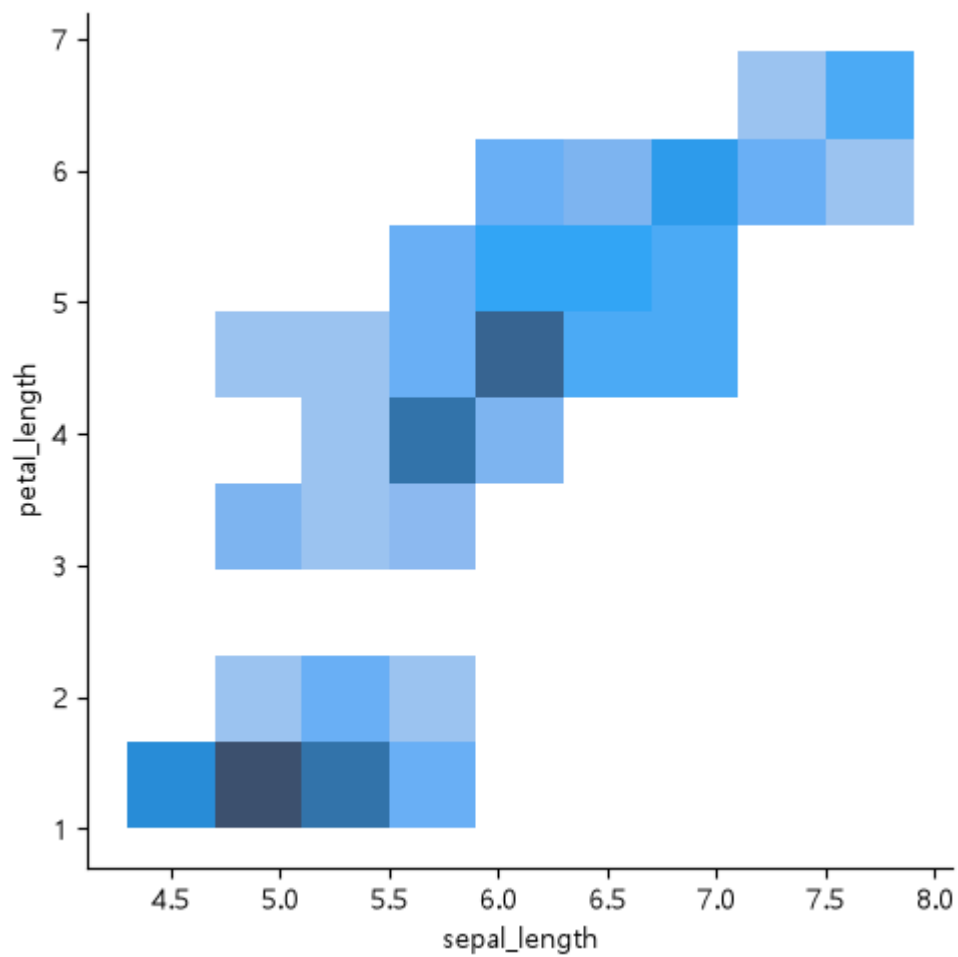
```
In [49]: sns.histplot(data=iris, x='sepal_length', y='petal_length')
plt.show()
```

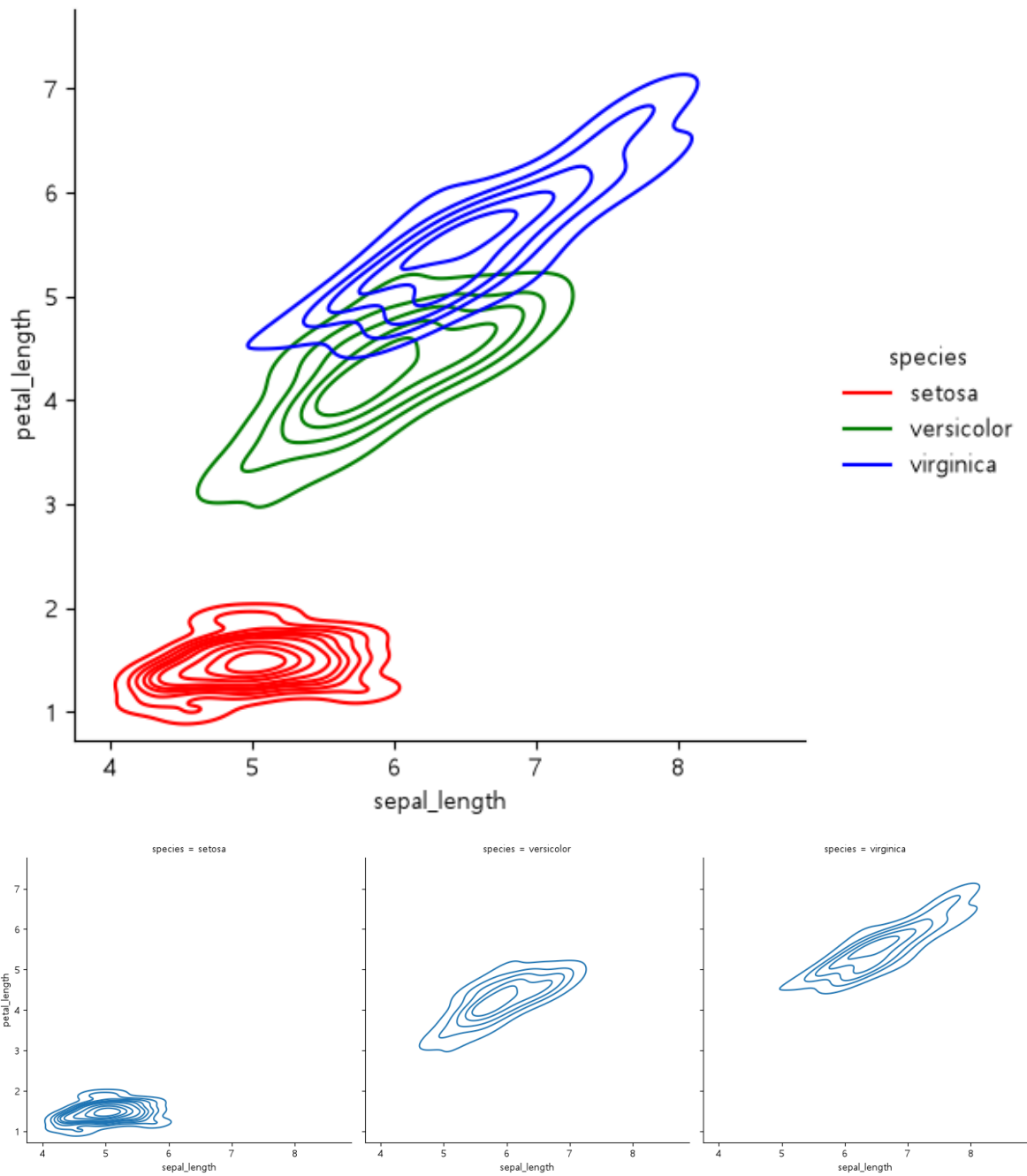


seaborn의 displot()

- 형식 : `displot(x, y, data, ...)`
- <https://seaborn.pydata.org/generated/seaborn.displot.html#seaborn.displot>

```
In [54]: sns.displot(data=iris, x='sepal_length', y='petal_length')
sns.displot(data=iris, x='sepal_length', y='petal_length', hue='species', palette='magma')
sns.displot(kind='kde', data=iris, x='sepal_length', y='petal_length', hue='species',
             palette=['red', 'green', 'blue'])
sns.displot(kind='kde', data=iris, x='sepal_length', y='petal_length', col='species')
plt.show()
```

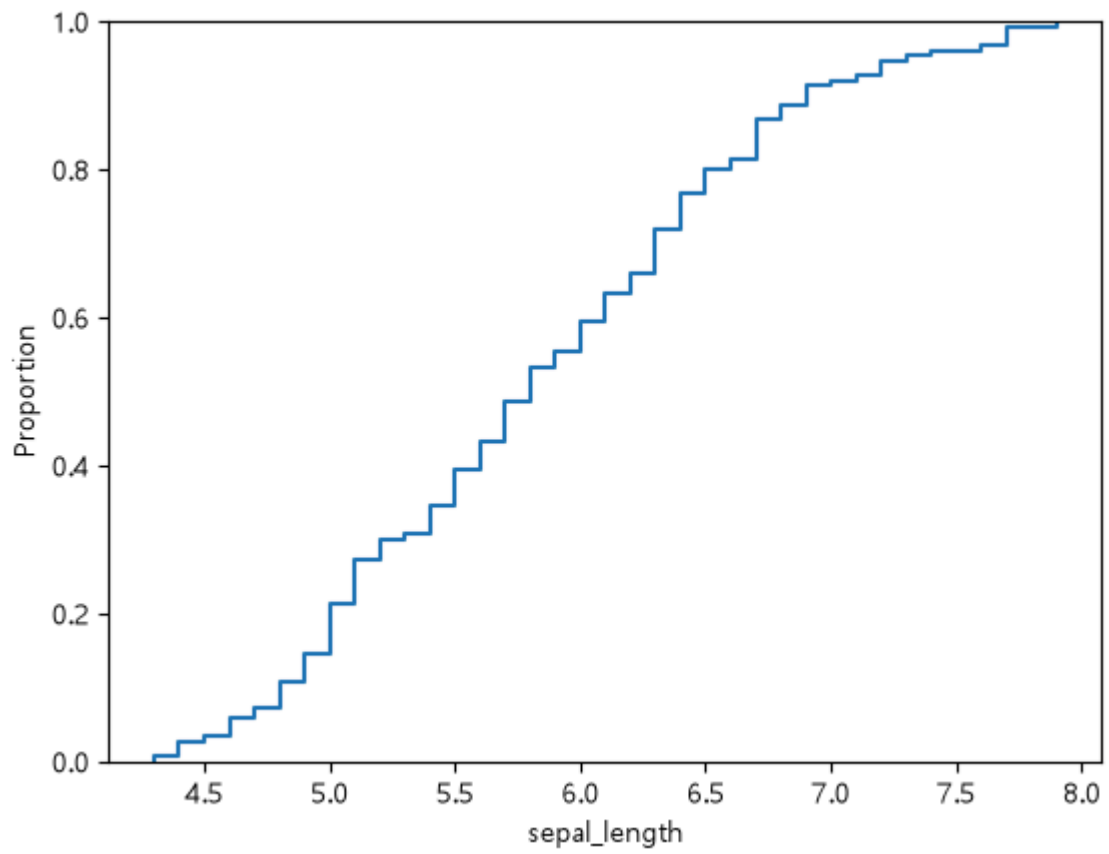




참고. `seaborn.ecdfplot()`

```
In [57]: # sns.displot(kind='ecdf', data=iris, x='sepal_length')
sns.ecdfplot(data=iris, x='sepal_length')

plt.show()
```



2. 데이터의 관계 파악

- 변수간 상관성(선형성) 파악
- 변수들간의 관계성 파악
- 산점도(scatter plot)
 - matplotlib.pyplot.scatter()
 - matplotlib.pyplot.plot()
 - seaborn.relplot(kind='scatter')
 - seaborn.scatterplot()
 - seaborn.regplot()
 - DataFrame.plot(kind='scatter')
 - DataFrame.plot.scatter()
- 버블플롯(bubble plot)
 - matplotlib.pyplot.scatter(, s= , c=)
 - seaborn.relplot(kind='scatter', size= ,)
 - seaborn.scatterplot(, size= ,)
- 산점도행렬(scatterplot matrix)
 - seaborn.pairplot()
 - pandas.plotting.scatter_matrix()
- 참고. 범주형변수들에 대한 교차표(분할표) 시각화 -> 모자이크 플롯(Mosaic plot)
 - <https://ko.wikipedia.org/wiki/%EB%AA%A8%EC%9E%90%EC%9D%B4%ED%81%A>

1) 산점도(scatter plot)

- 두 변수간의 선형성 파악
- 두 변수는 수치형 데이터

In [4]: `iris.head()`

Out[4]:

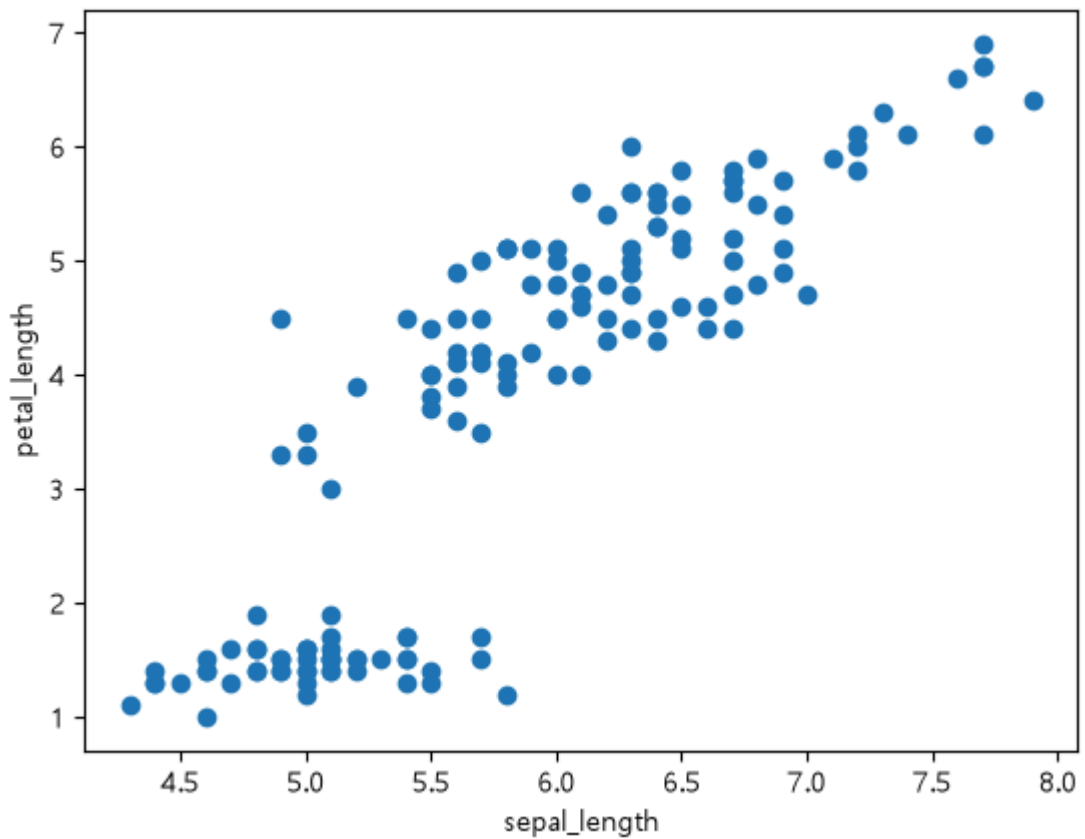
	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

① matplotlib.pyplot의 산점도 함수

- 형식.
`scatter(x, y, s=None, c=None, marker=None, cmap=None, norm=None, vmin=None, vmax=None, alpha=None, linewidths=None, *, edgecolors=None, plotnonfinite=False, data=None,...)`

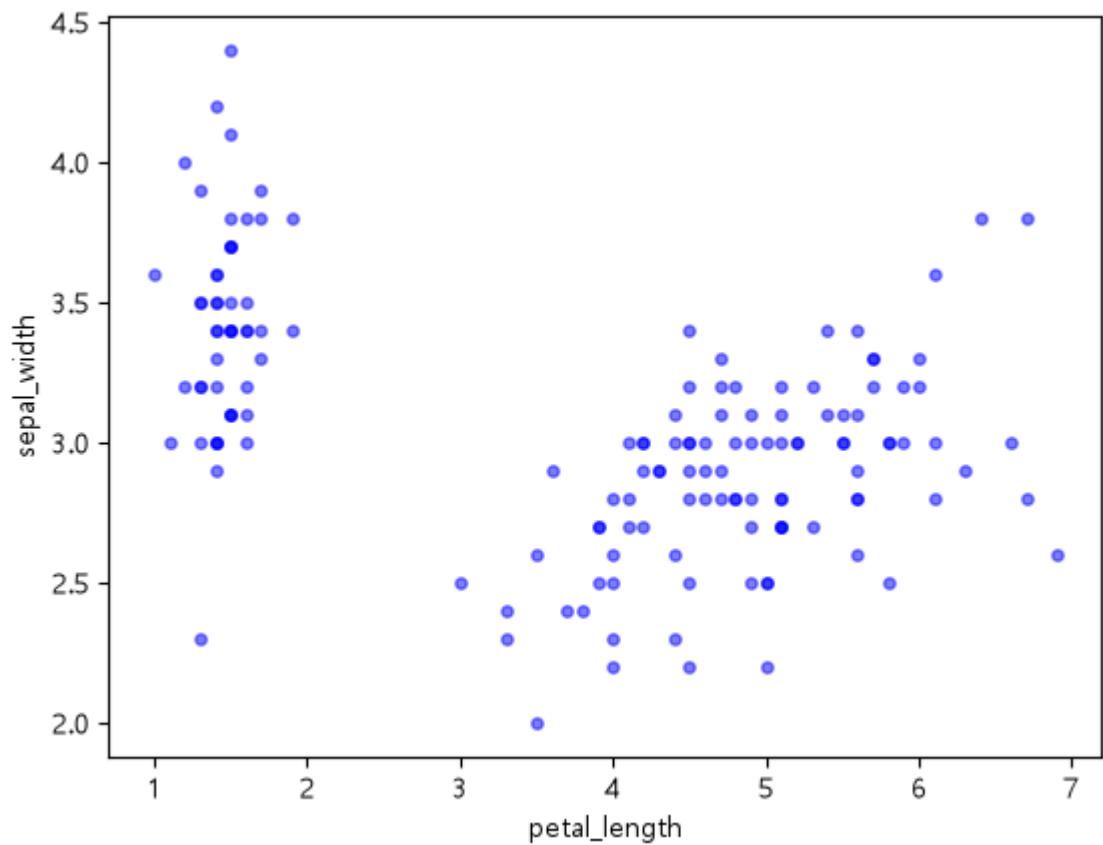
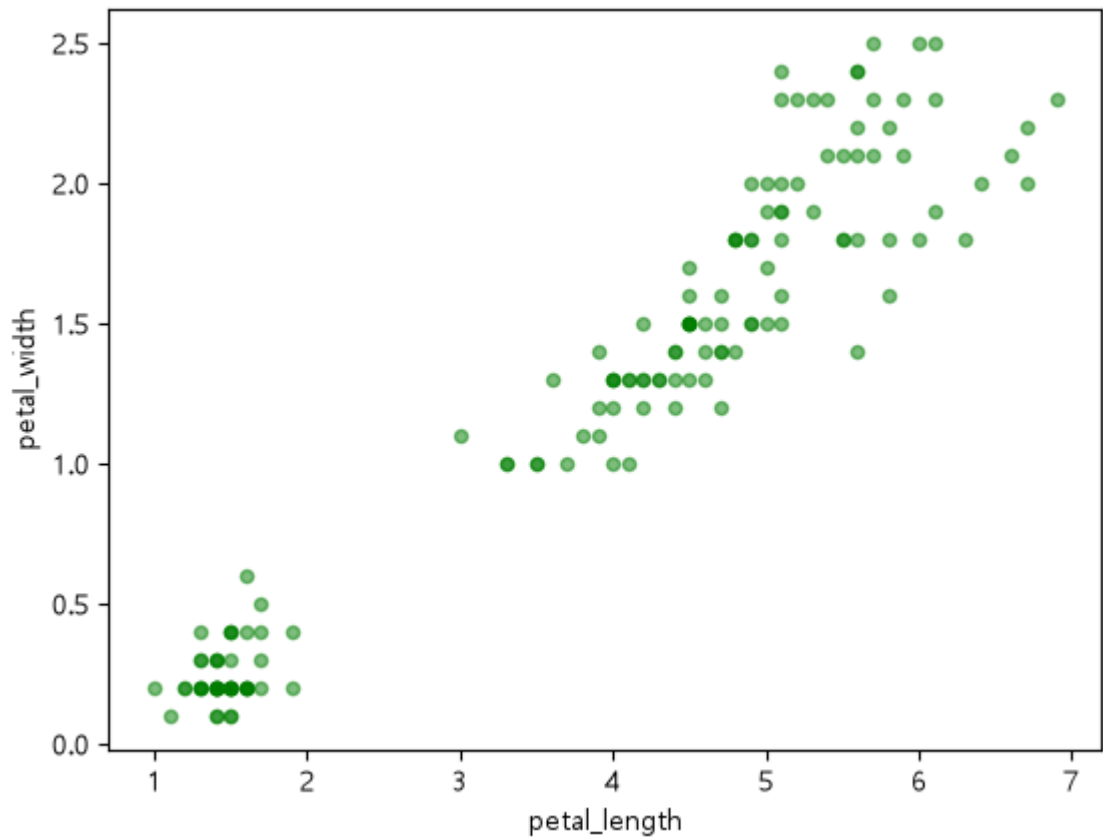
In [5]: `plt.scatter(data=iris, x='sepal_length', y='petal_length')`
`plt.xlabel('sepal_length')`

```
plt.ylabel('petal_length')
plt.show()
```



② pandas의 plot.scatter() 이용한 산점도 그리기

```
In [8]: iris.plot(kind='scatter', x='petal_length', y='petal_width',
                s=20, c='green', alpha=0.5)
iris.plot.scatter(x='petal_length', y='sepal_width',
                s=15, c='blue', alpha=0.5)
plt.show()
```

③ seaborn의 산점도 함수

- 형식1.
`scatterplot(x=None, y=None, hue=None, style=None, size=None,
data=None, palette=None,
hue_order=None, hue_norm=None, sizes=None,`

```
size_order=None,
        size_norm=None, markers=True, ... )
```

- <https://seaborn.pydata.org/generated/seaborn.scatterplot.html#seaborn.scatterplot>

- 형식2.

```
relplot(kind='scatter', x=None, y=None, hue=None, size=None,
        data=None, ... )
```

- <https://seaborn.pydata.org/generated/seaborn.relplot.html#seaborn.relplot>

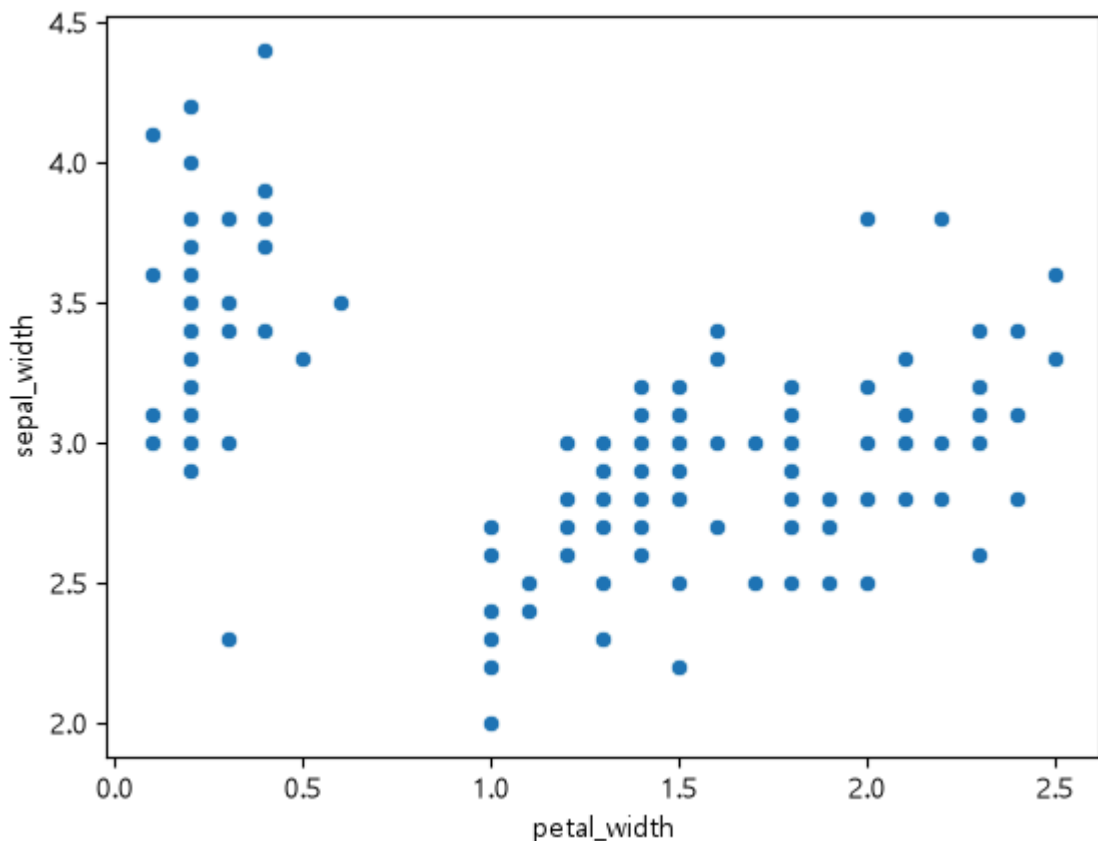
- 형식3.

```
regplot(x=None, y=None, data=None, x_estimator=None, x_bins=None,
        x_ci='ci', scatter=True, fit_reg=True, ci=95,...)
```

- <https://seaborn.pydata.org/generated/seaborn.regplot.html#seaborn.regplot>

sns.scatterplot()

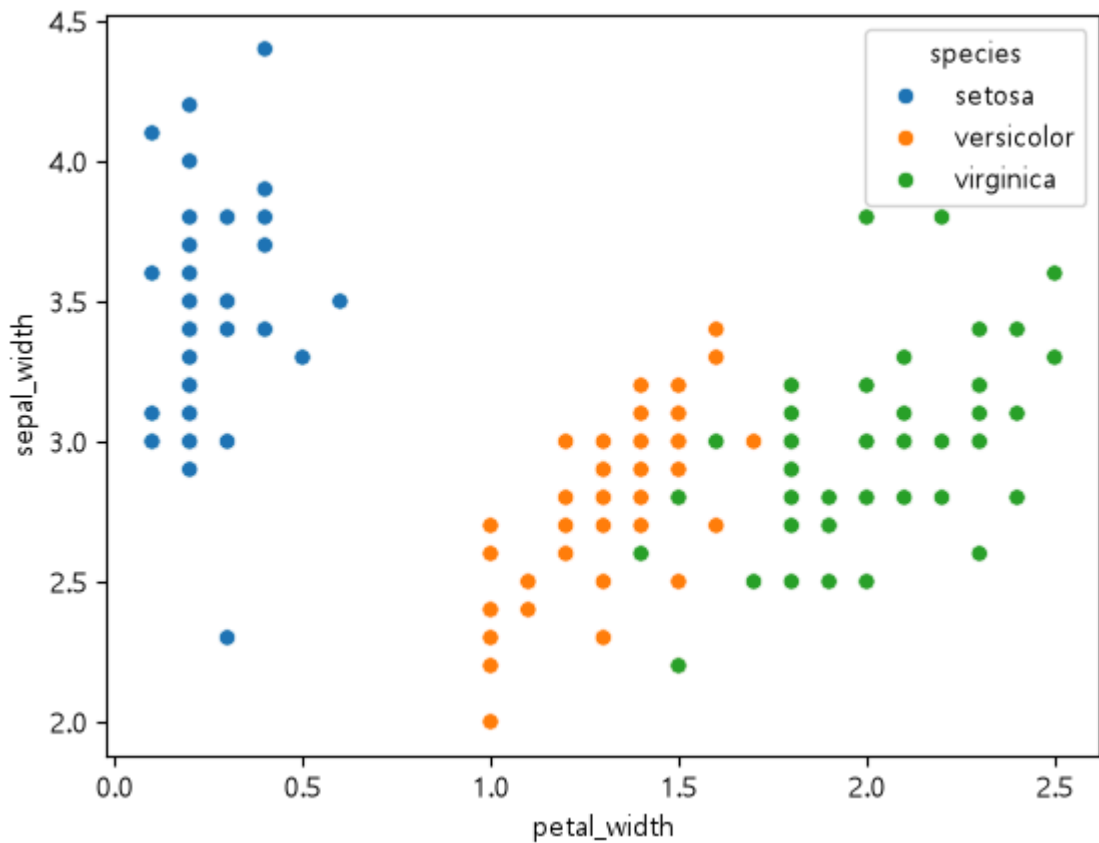
```
In [10]: sns.scatterplot(data=iris, x='petal_width', y='sepal_width')
plt.show()
```



sns.scatterplot(hue=)

- 범주형 변수의 범주에 따라 다른 색상으로 표시

```
In [11]: sns.scatterplot(data=iris, x='petal_width', y='sepal_width', hue='species')
plt.show()
```



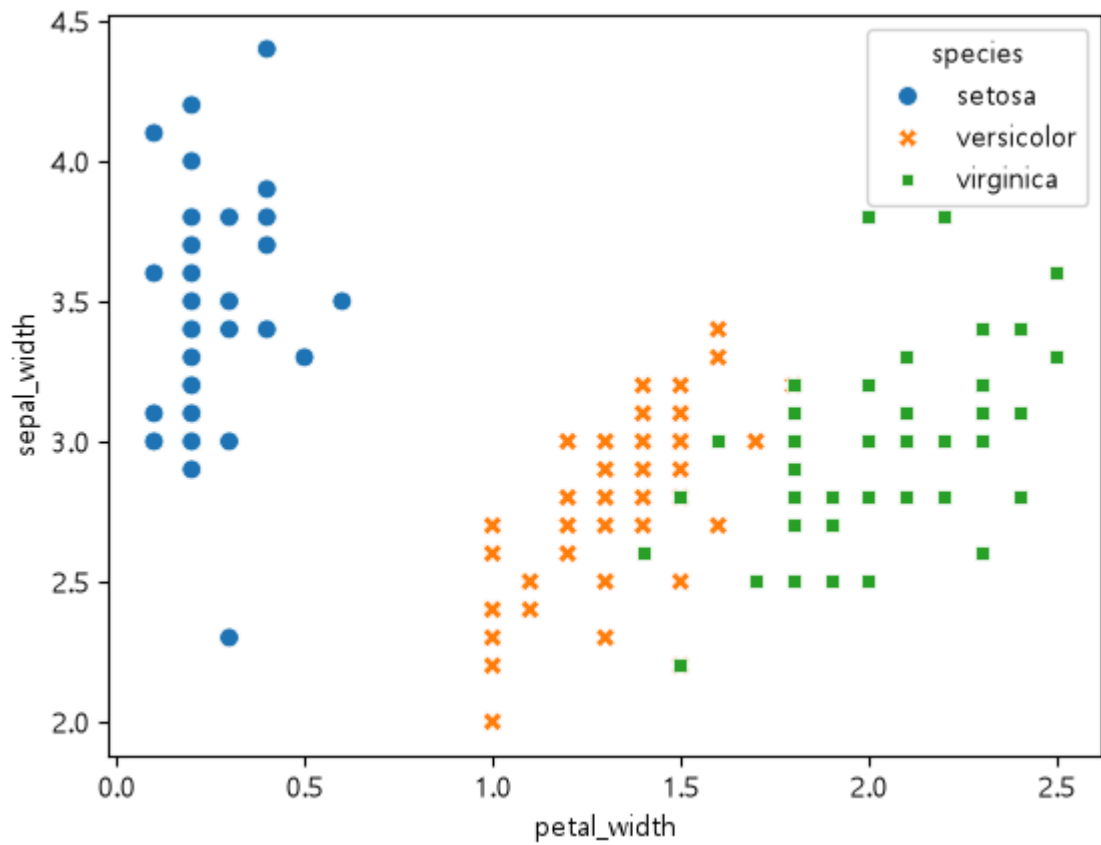
`sns.scatterplot(style=)`

- 범주형 변수의 범주에 따라 마커 모양을 다르게 표현

```
In [17]: sns.scatterplot(data=iris, x='petal_width', y='sepal_width',
                        style='species')
plt.show()
sns.scatterplot(data=iris, x='petal_width', y='sepal_width',
                hue='species', style='species')
plt.show()
```



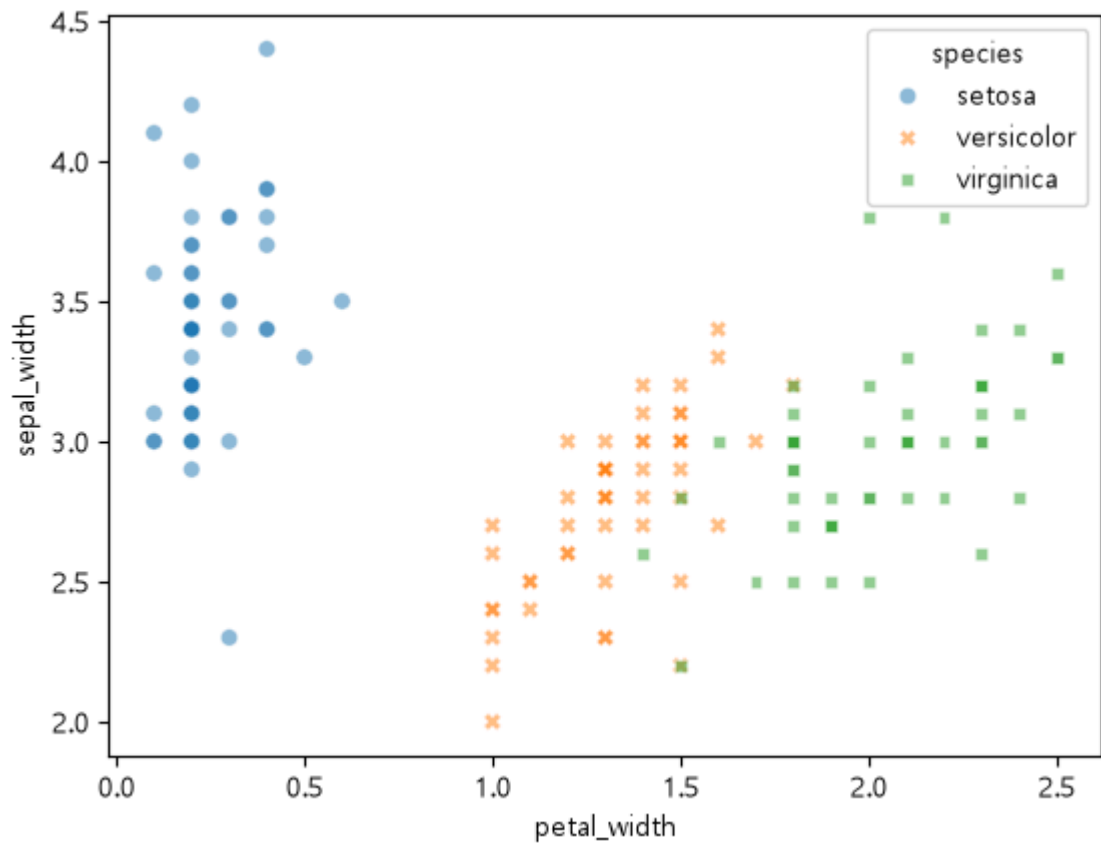
```
plt.show()
```



scatterplot(alpha=)

- 스타일 변경

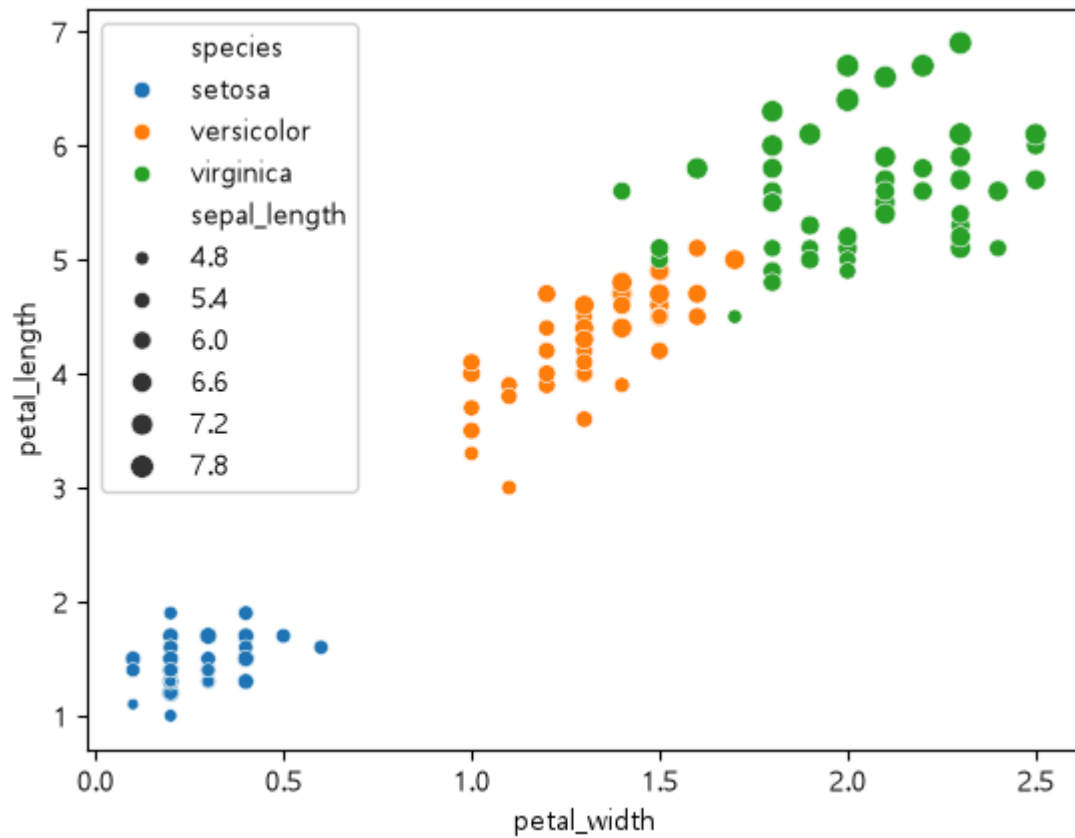
```
In [19]: sns.scatterplot(data=iris, x='petal_width', y='sepal_width',  
                        hue='species', style='species', alpha=0.5)  
plt.show()
```



scatterplot(**size=**)

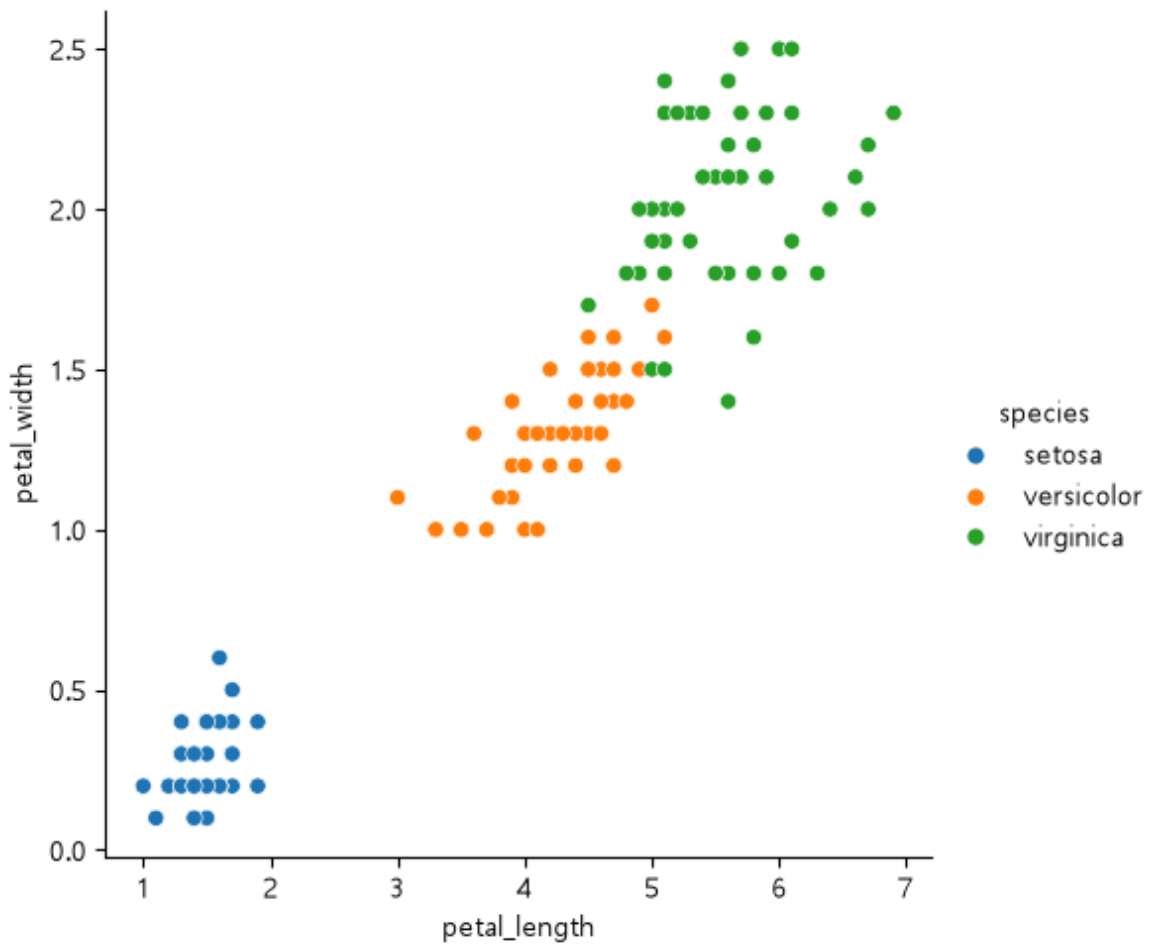
: bubble chart

```
In [20]: sns.scatterplot(data=iris, x='petal_width', y='petal_length',  
                        hue='species', size='sepal_length')  
plt.show()
```



sns.relplot(kind='scatter')

```
In [22]: sns.relplot(kind='scatter', data=iris,  
                    x='petal_length', y='petal_width', hue='species')  
plt.show()
```



`sns.relplot(palette=)`

- seaborn에서 제공되는 팔레트
- https://seaborn.pydata.org/tutorial/color_palettes.html

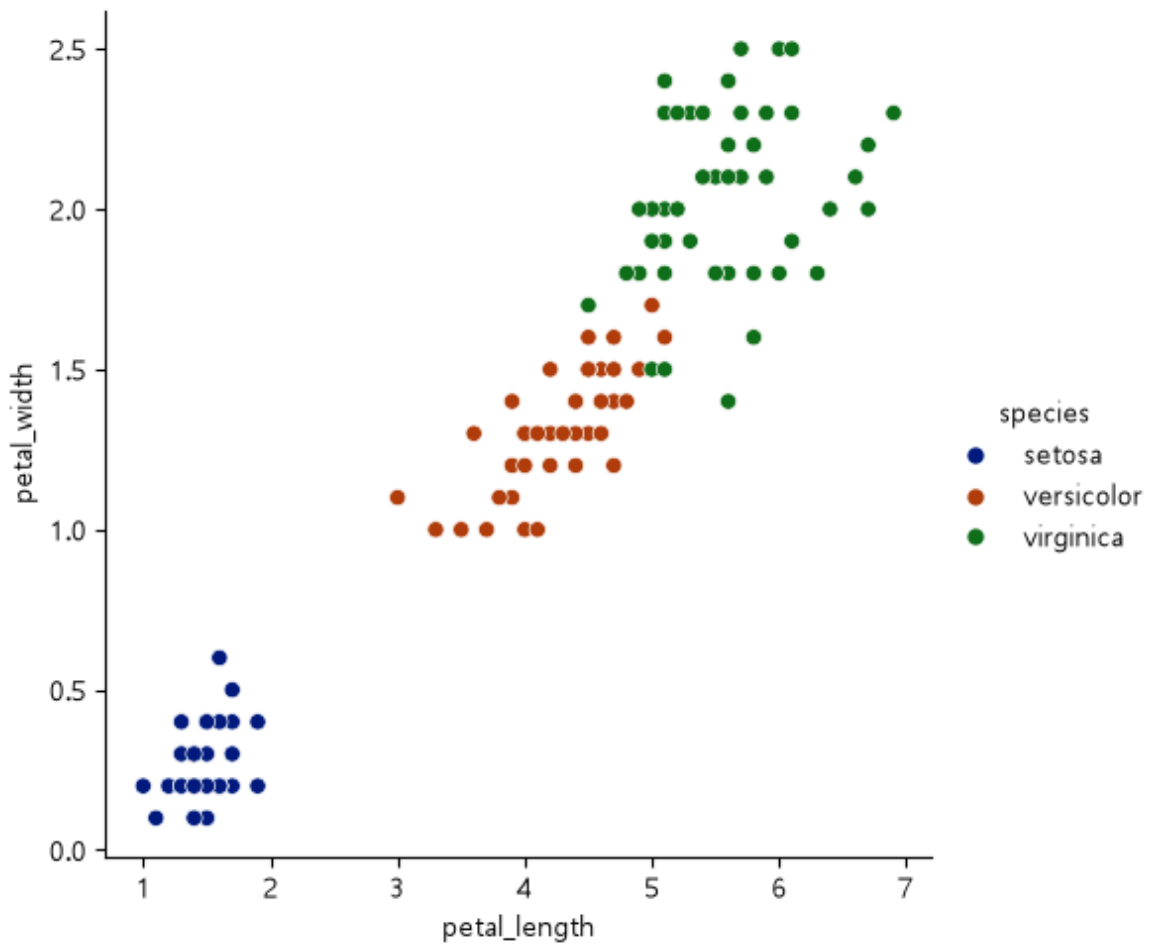
In [23]: `# 현재 적용된 컬러맵 반환`
`sns.color_palette()`



In [24]: `# 컬러 맵 설정`
`sns.color_palette('dark')`

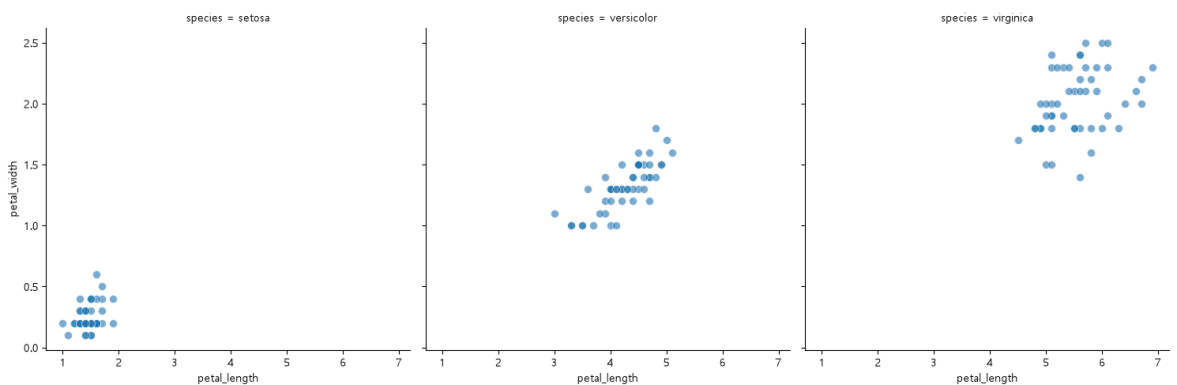


In [26]: `sns.relplot(kind='scatter', data=iris,
 x='petal_length', y='petal_width', hue='species',
 palette='dark')
plt.show()`



`sns.relplot(col=)`

```
In [29]: sns.relplot(kind='scatter', data=iris,
                    x='petal_length', y='petal_width', col='species',
                    alpha=0.6, s=50)
plt.show()
```

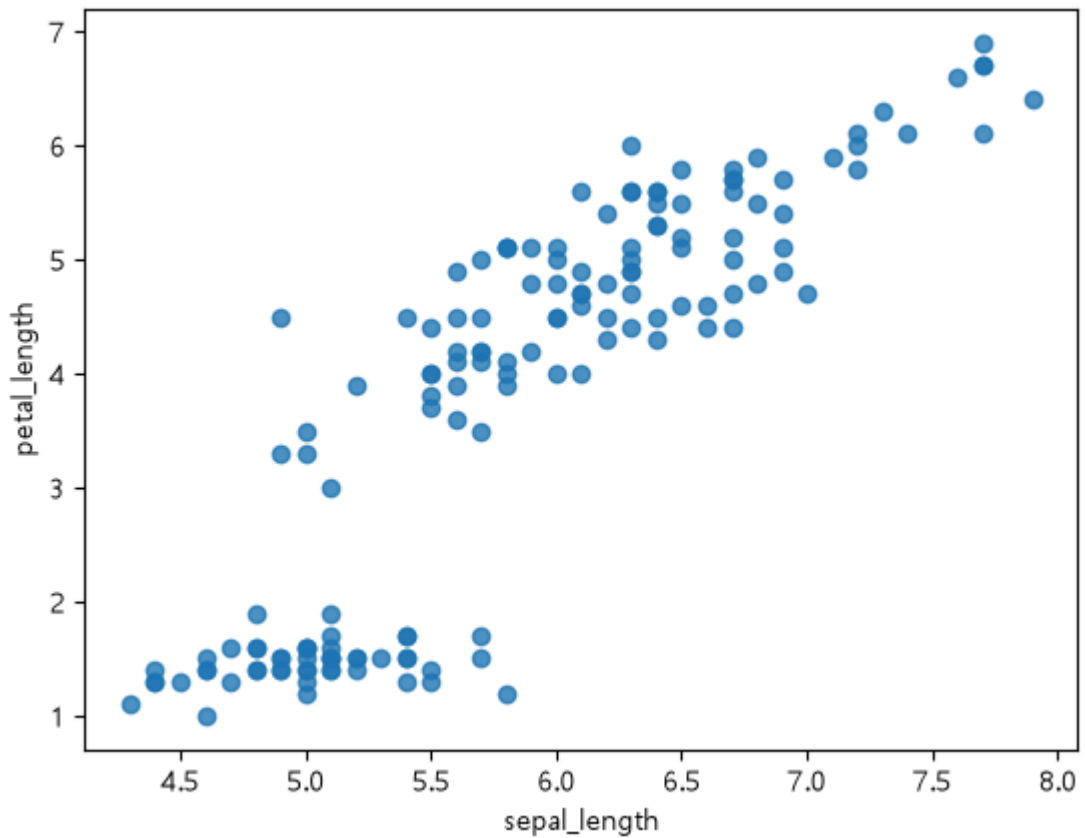


`sns.regplot()` 을 이용한 산점도

`sns.regplot(fit_reg=False)`

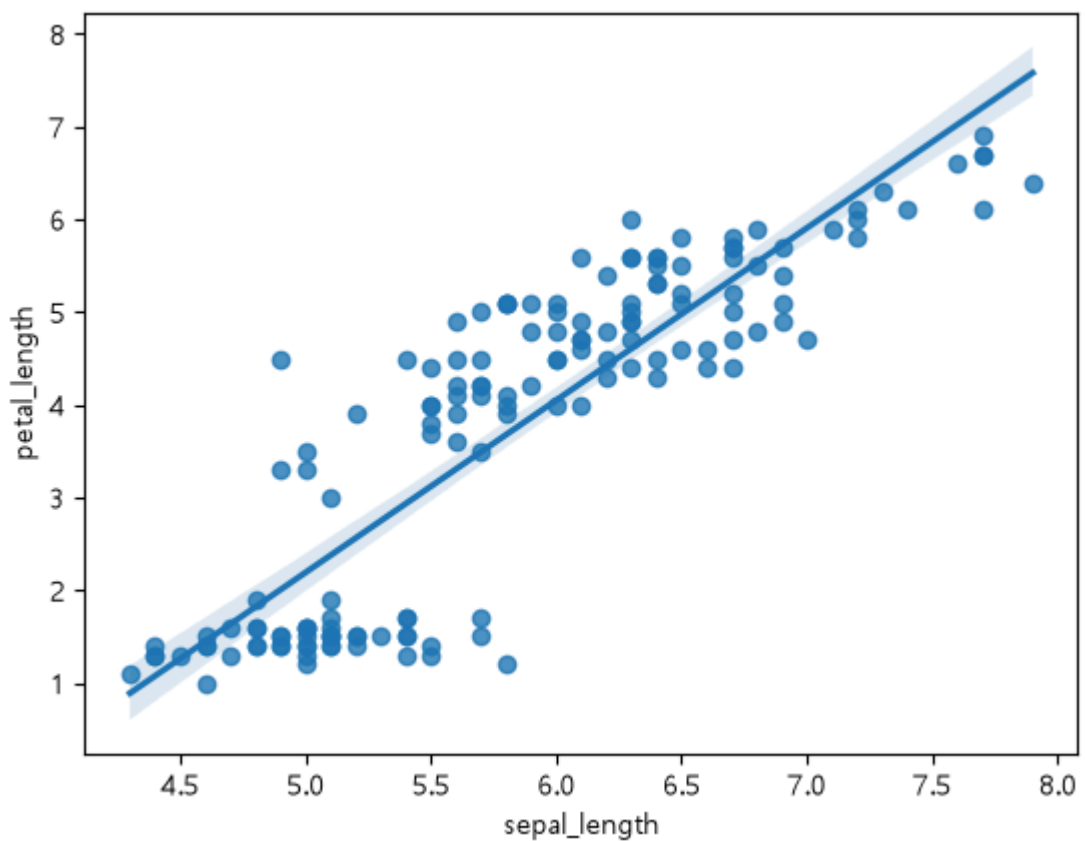
- 선형회귀 적합 선을 포함시키지 않을 때

```
In [31]: sns.regplot(data=iris, x='sepal_length', y='petal_length', fit_reg=False)
plt.show()
```



`sns.regplot(fit_reg=True)`

```
In [32]: sns.regplot(data=iris, x='sepal_length', y='petal_length', fit_reg=True)
plt.show()
```



2) 산점도 행렬(scatter plot matrix)

: 여러 개의 연속형 변수에 대해서 두 변수씩 쌍을 이루는 산점도를 그려 한꺼번에 변수 간 관계를 일목요연하게 볼 수 있음

① seaborn.pairplot()을 이용한 산점도 행렬

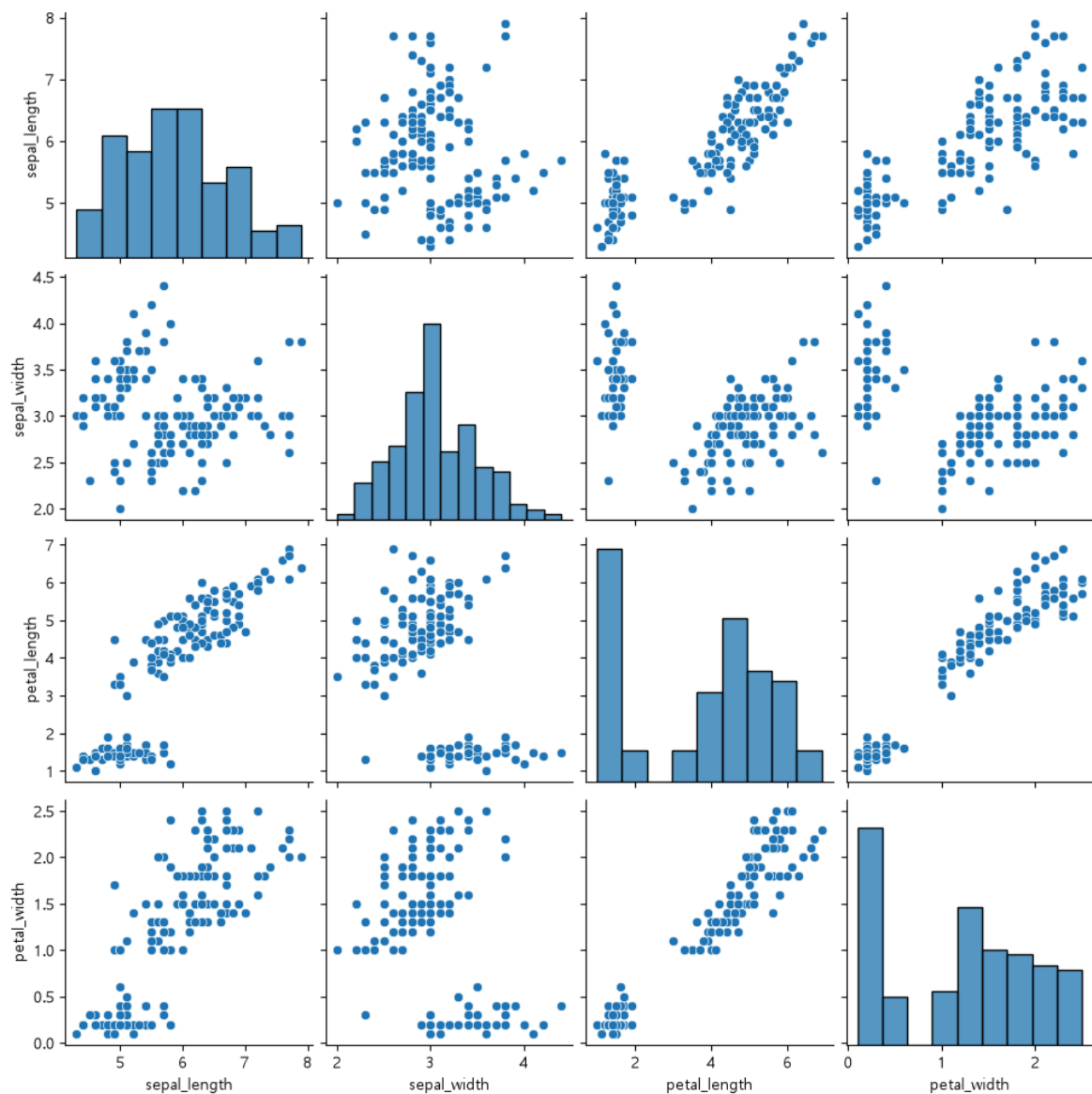
- 형식

```
pairplot(data, hue=None, hue_order=None, palette=None, vars=None,
          x_vars=None,
          y_vars=None, kind='scatter', diag_kind='auto',
          markers=None,
          height=2.5, aspect=1, corner=False, dropna=False,
          plot_kws=None,
          diag_kws=None, grid_kws=None, size=None)
```
- <https://seaborn.pydata.org/generated/seaborn.pairplot.html#seaborn.pairplot>

sns.pairplot()의 인수

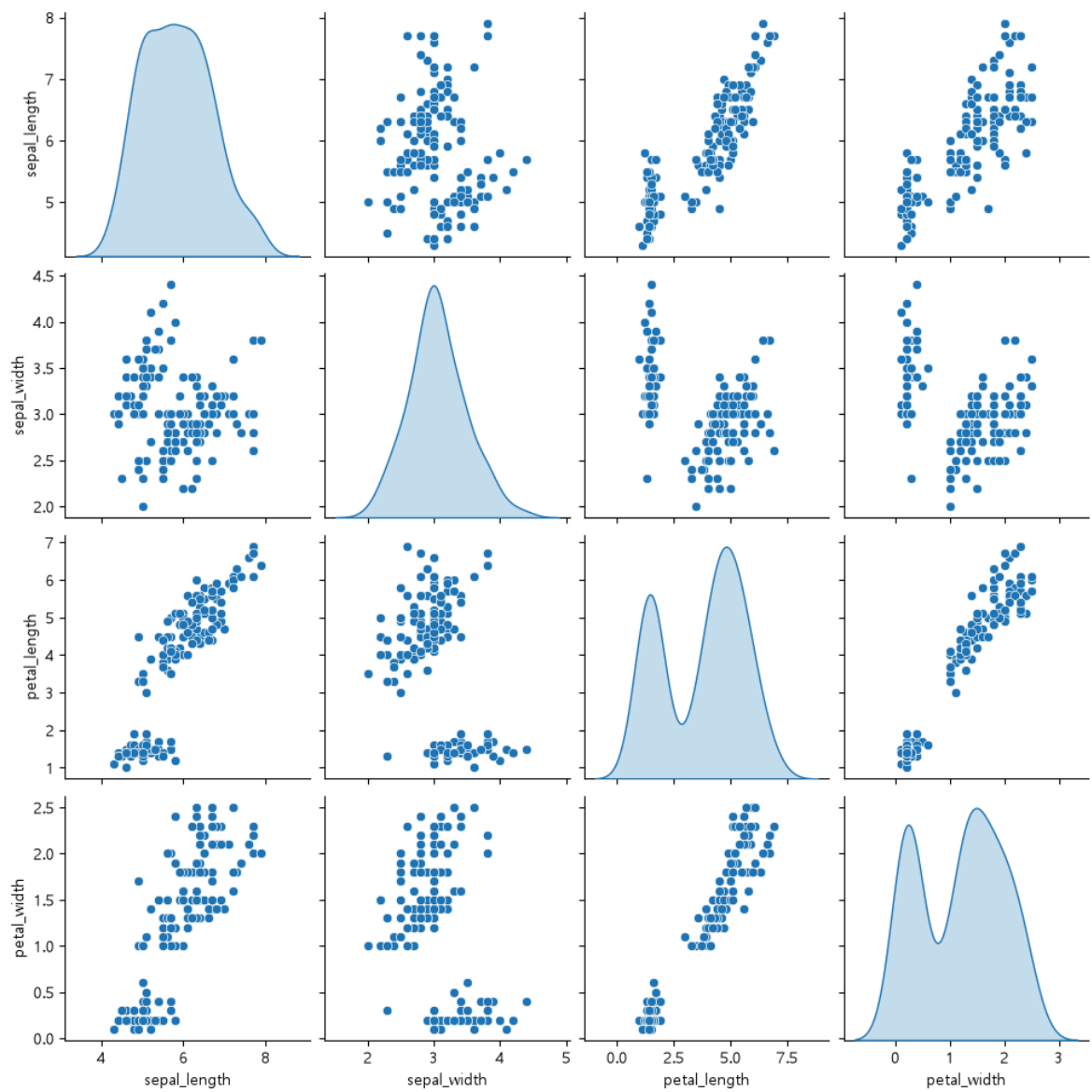
- kind='scatter' : 각 행렬 위치에 scatter|kde|hist 등을 표시
- diag_kind='kde' : 각 변수별 커널밀도추정곡선 그림
- hue='species' : 'species'(setosa, versicolor, virginica) 별로 색깔을 다르게 표시
- 색깔은 palette 에 'bright', 'pastel', 'deep', 'muted', 'colorblind', 'dark' 중에서 가독성이 좋고 선호하는 색상으로 선택
- corner : 상감각의 산점도는 표시하지 않음

```
In [33]: sns.pairplot(data=iris)
plt.show()
```

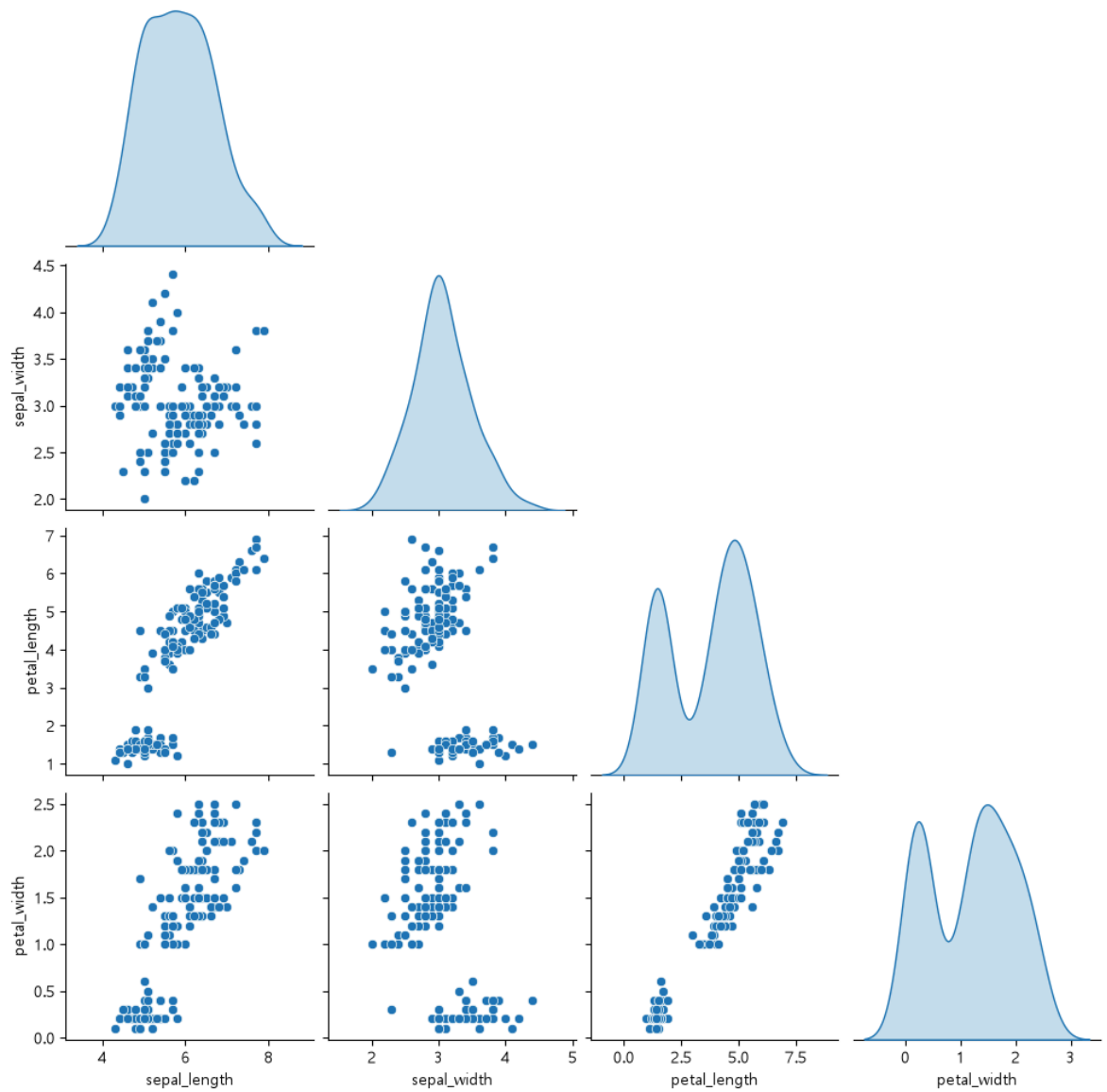


```
In [35]: # sns.pairplot(data=iris, diag_kind='hist')
sns.pairplot(data=iris, diag_kind='kde')

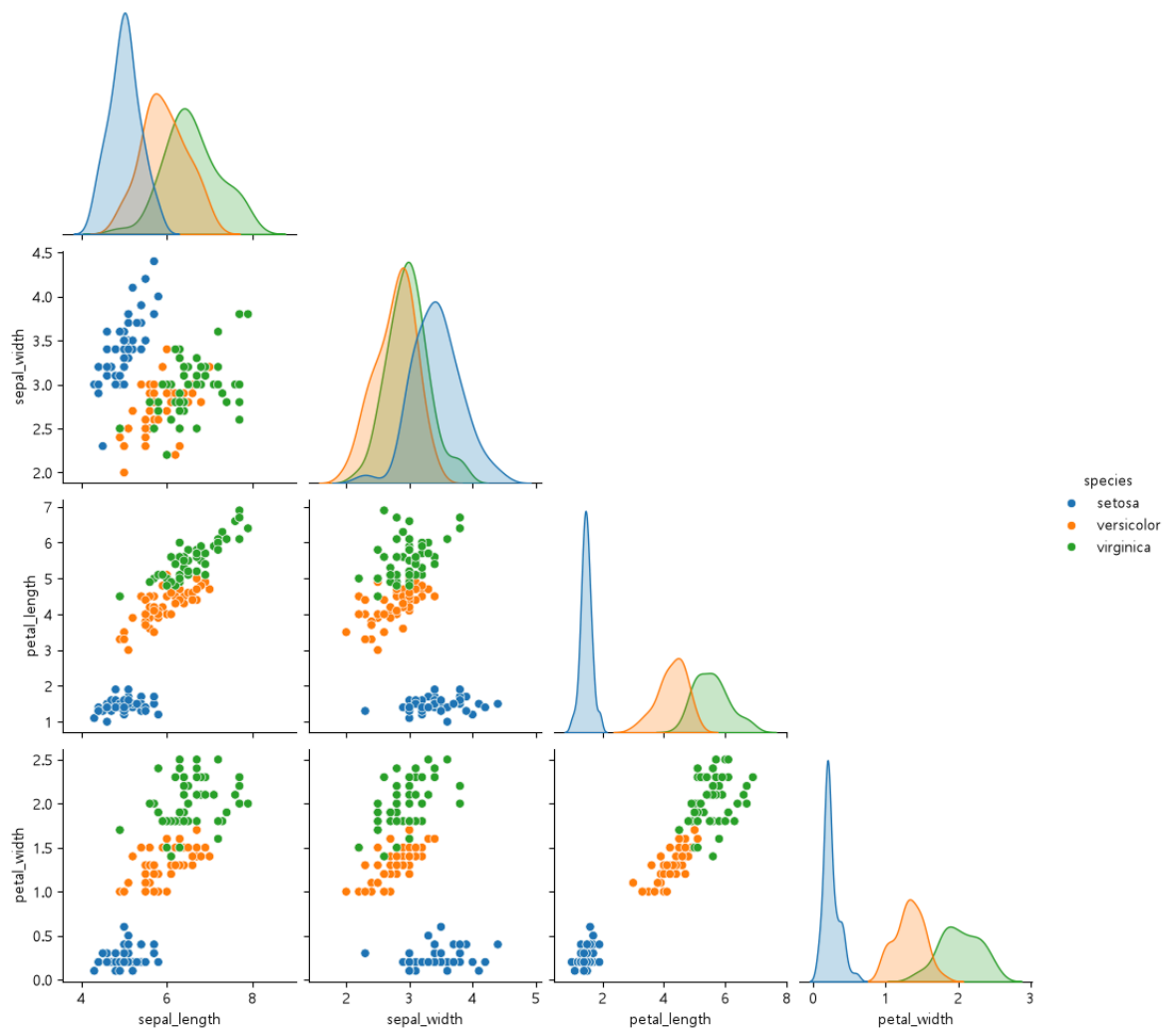
plt.show()
```



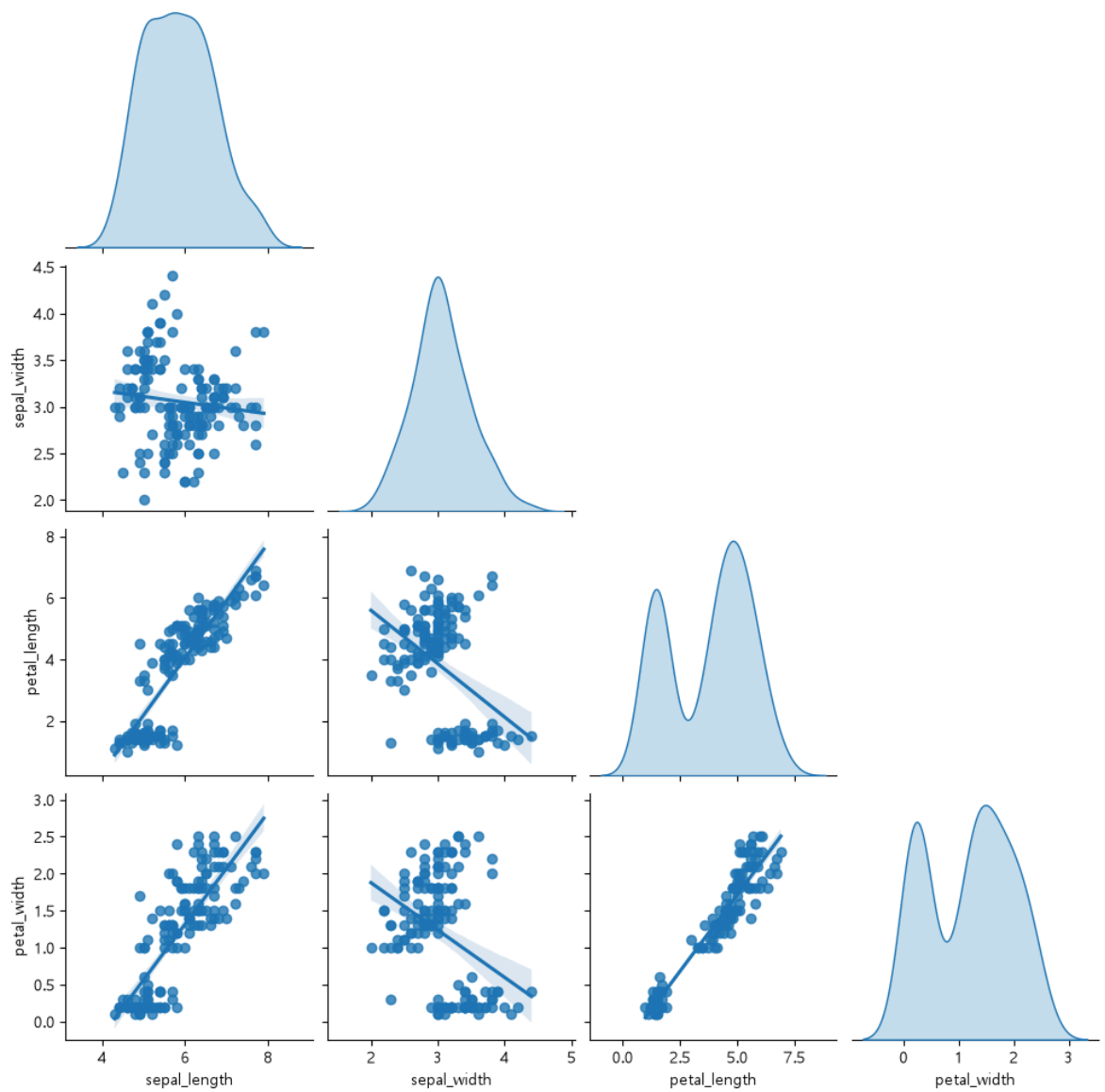
```
In [37]: sns.pairplot(data=iris, diag_kind='kde', corner=True)
plt.show()
```



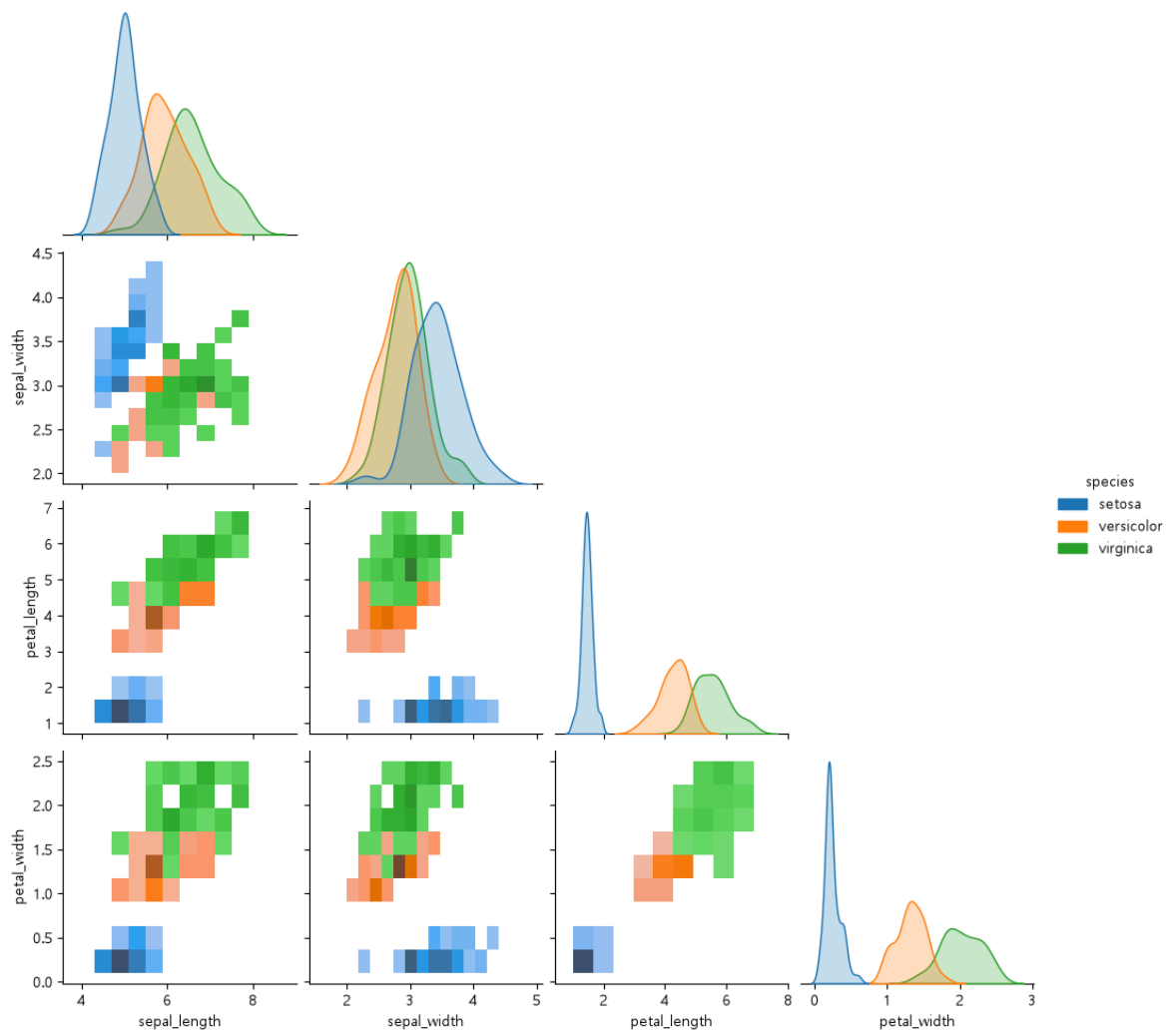
```
In [40]: sns.pairplot(data=iris, diag_kind='kde', corner=True, hue='species')
plt.show()
```



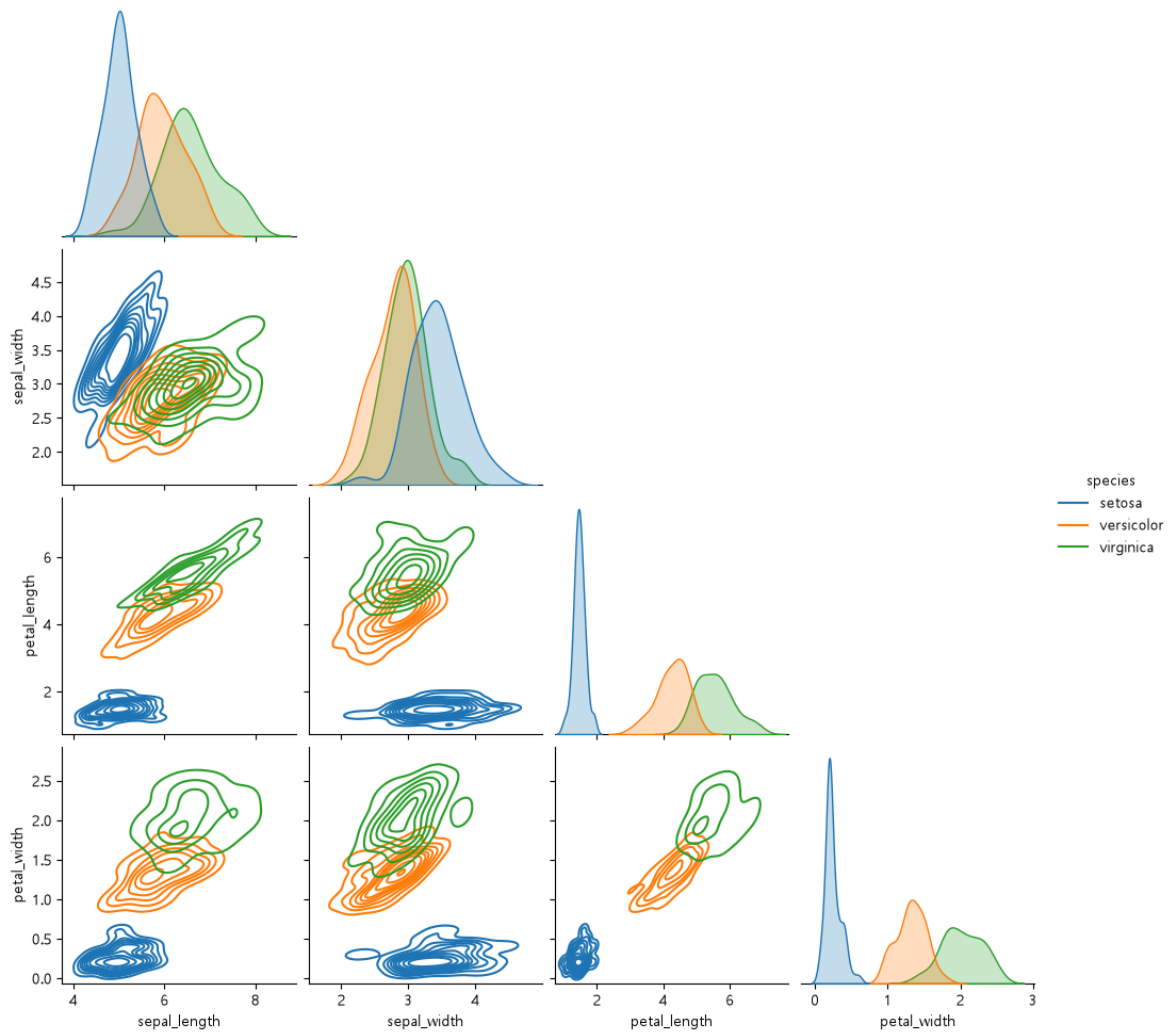
```
In [42]: sns.pairplot(data=iris, kind='reg', diag_kind='kde', corner=True)
plt.show()
```



```
In [43]: sns.pairplot(data=iris, kind='hist', diag_kind='kde', corner=True, hue='species',
plt.show())
```

```
In [44]: sns.pairplot(data=iris, kind='kde', diag_kind='kde', corner=True, hue='species')
plt.show()
```



② pandas.plotting 의 scatter_matrix() 함수를 사용하여 산점도 행렬

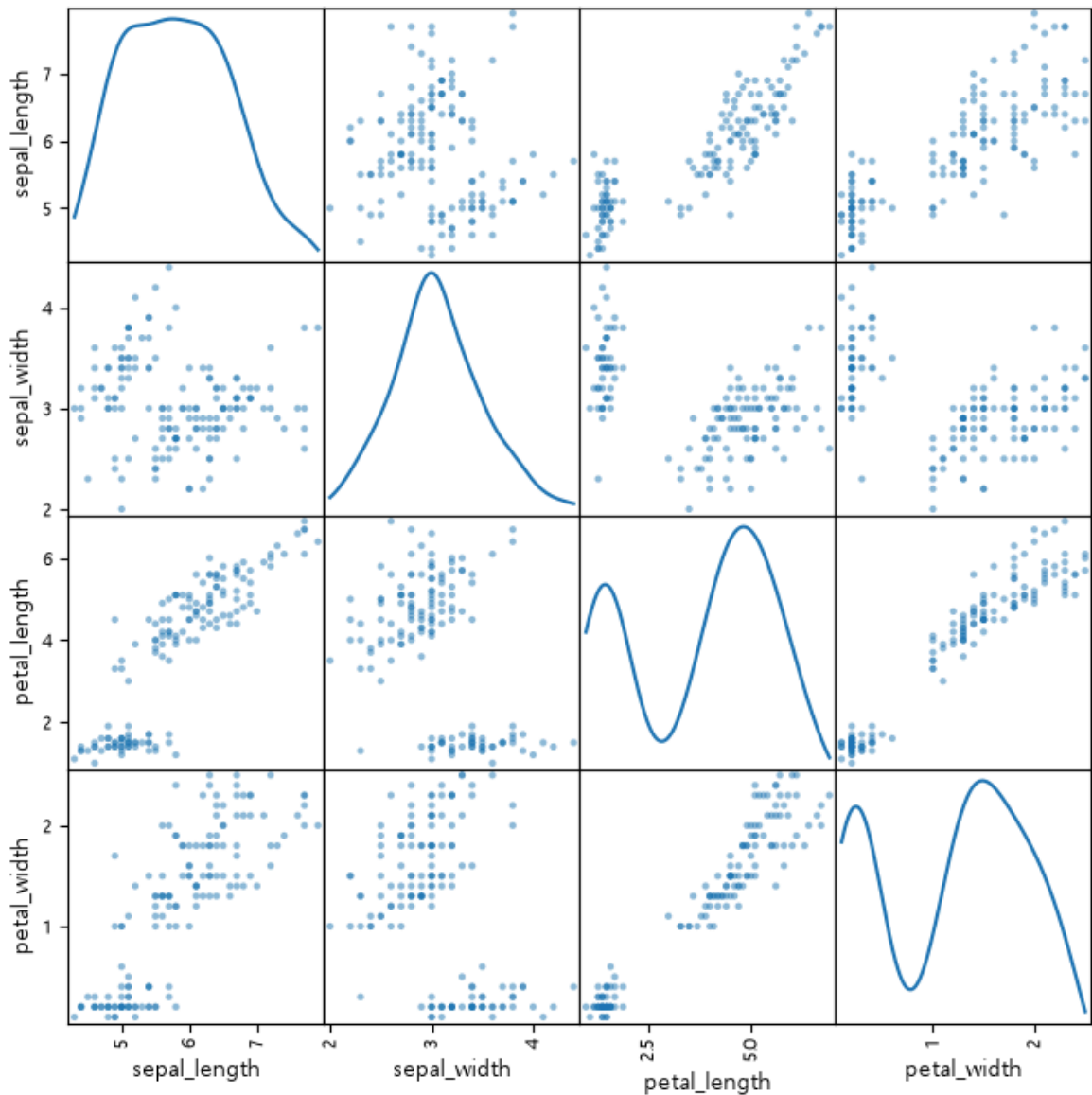
- 형식

```
scatter_matrix(frame, alpha=0.5, figsize=None, ax=None, grid=False,
               diagonal='hist', marker='.', density_kws=None,
               hist_kws=None, range_padding=0.05, **kwargs)
```

- https://pandas.pydata.org/docs/reference/api/pandas.plotting.scatter_matrix.html

```
In [39]: from pandas.plotting import scatter_matrix

scatter_matrix(iris, diagonal='kde', figsize=(8,8))
plt.show()
```



3) 버블 플롯

: 산점도에서 x, y이외의 다른 변수를 포인트의 크기로 매핑하여 변수들간의 관계를 파악하게 함

```
In [49]: iris.iloc[49:51, :]
```

```
Out[49]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
49	5.0	3.3	1.4	0.2	setosa
50	7.0	3.2	4.7	1.4	versicolor

```
In [50]: iris.iloc[99:101, :]
```

```
Out[50]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
99	5.7	2.8	4.1	1.3	versicolor
100	6.3	3.3	6.0	2.5	virginica

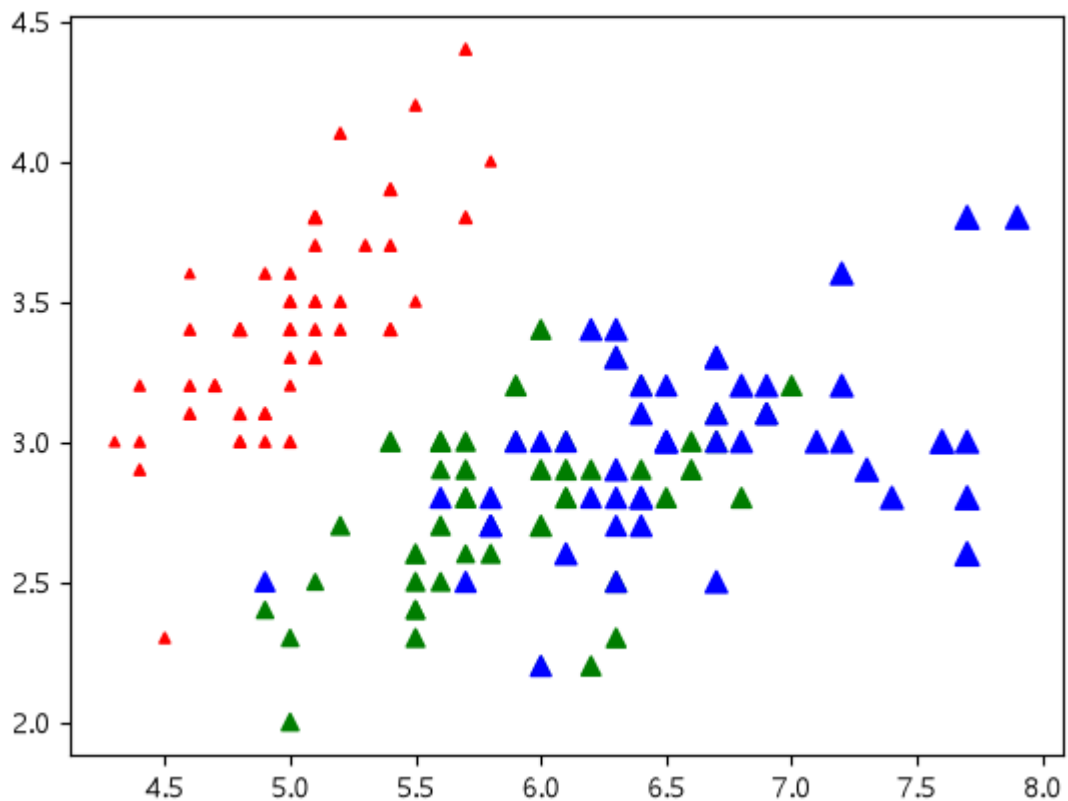
① matplotlib.pyplot의 scatter()를 이용한 버블플롯

- scatter()의 s, c 인수 사용

```
In [53]: sizes = iris.petal_length * 10
          colors = ['r']*50 + ['g']*50 + ['b']*50
          print(colors)
```

[illegible]

```
In [57]: # plt.scatter(data=iris, x='sepal_length', y='sepal_width', s=sizes, c=colors)
plt.scatter(data=iris, x='sepal_length', y='sepal_width', s=sizes, c=colors, mar
plt.show()
```



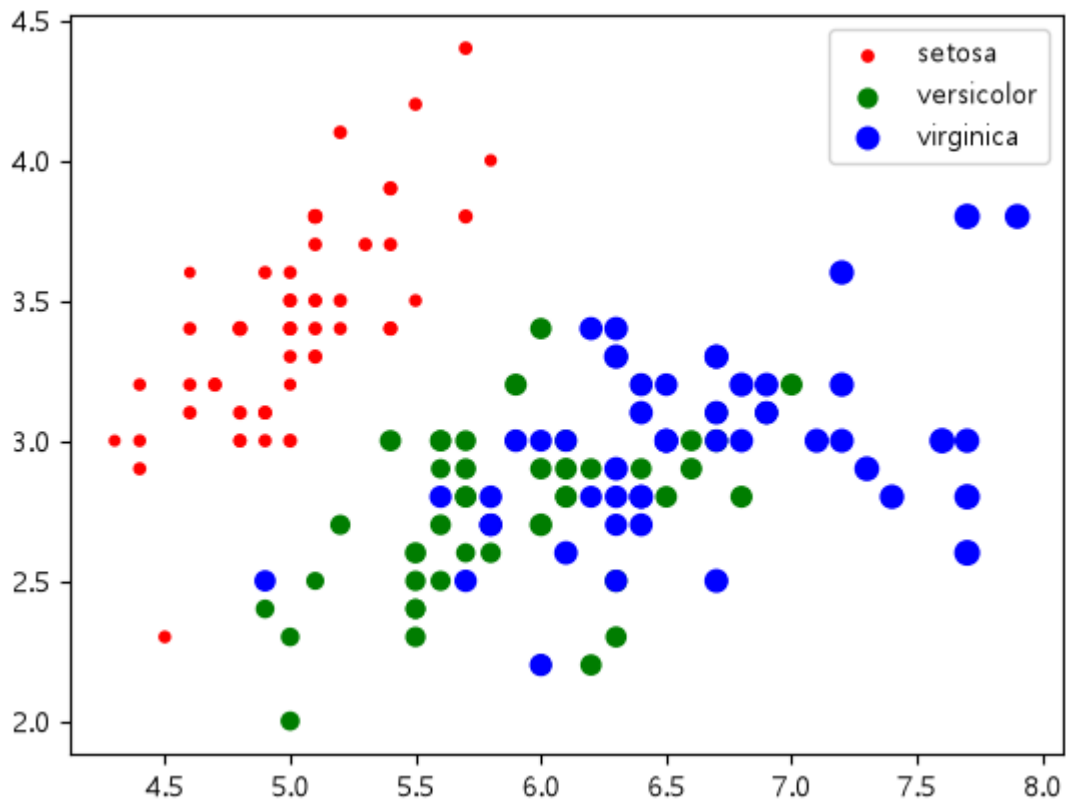
```
In [61]: colors = ['r', 'g', 'b']
markers = ['d', 'o', '^']
values = ['setosa', 'versicolor', 'virginica']

for i, value in enumerate(values):
    df = iris[iris.species == value]
    x = df.sepal_length
    y = df.sepal_width
    s = df.petal_length * 10
    # plt.scatter(x=x, y=y, s=s, c=colors[i], marker=markers[i], label=value)
```

```
plt.scatter(x=x, y=y, s=s, c=colors[i], label=value)
```

```
plt.legend()
```

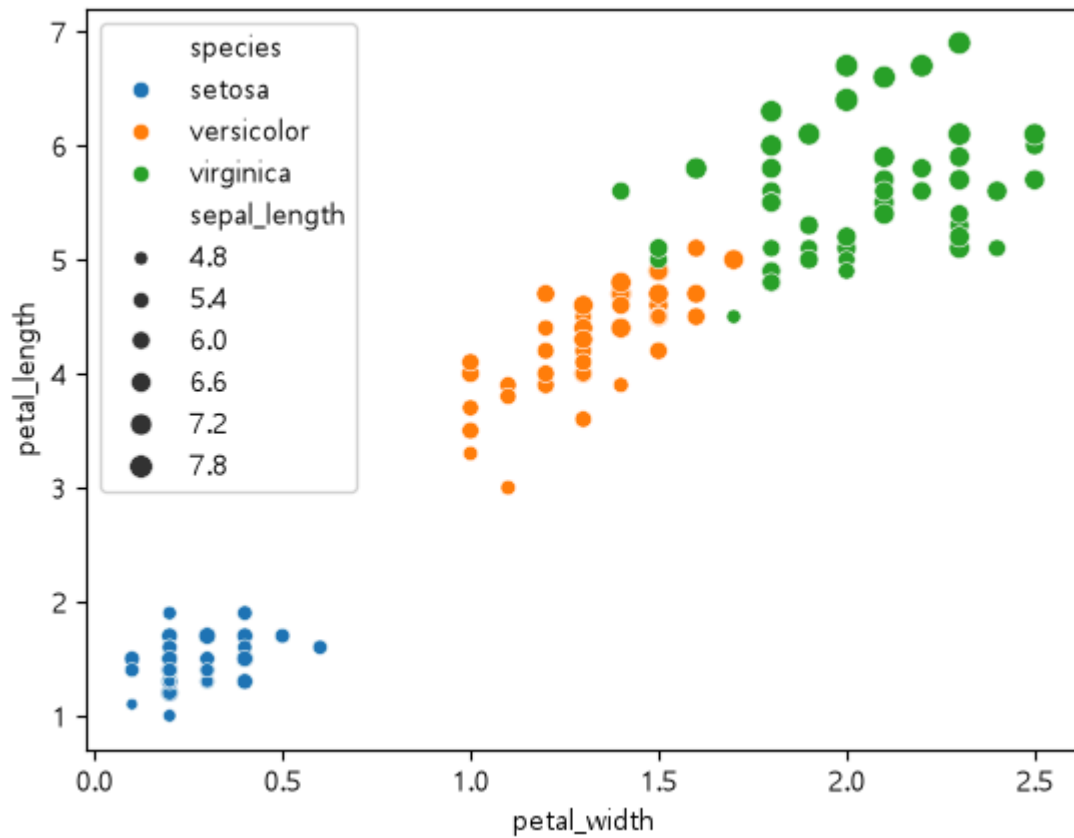
```
plt.show()
```



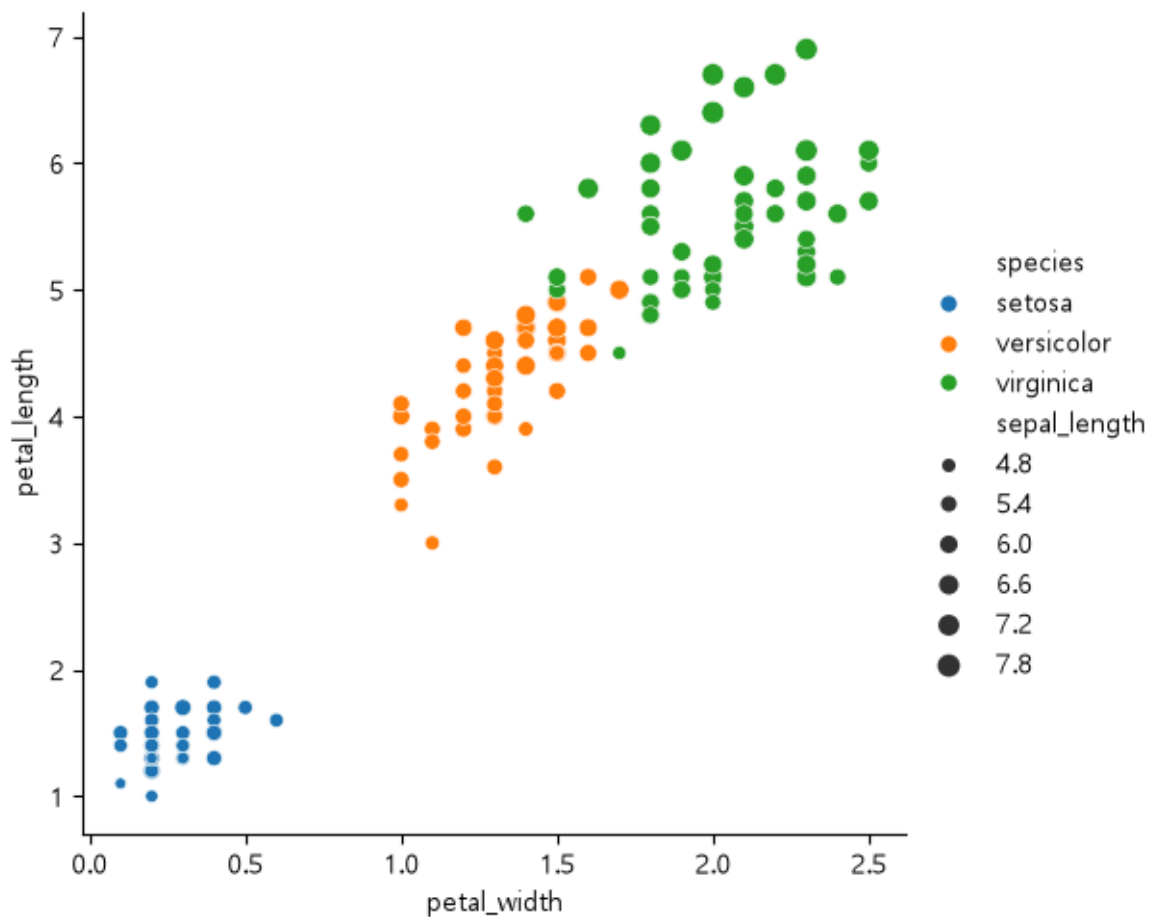
② seaborn의 산점도 함수를 이용한 버블 플롯

- sns.scatterplot() 또는 relplot()의 size인수 사용

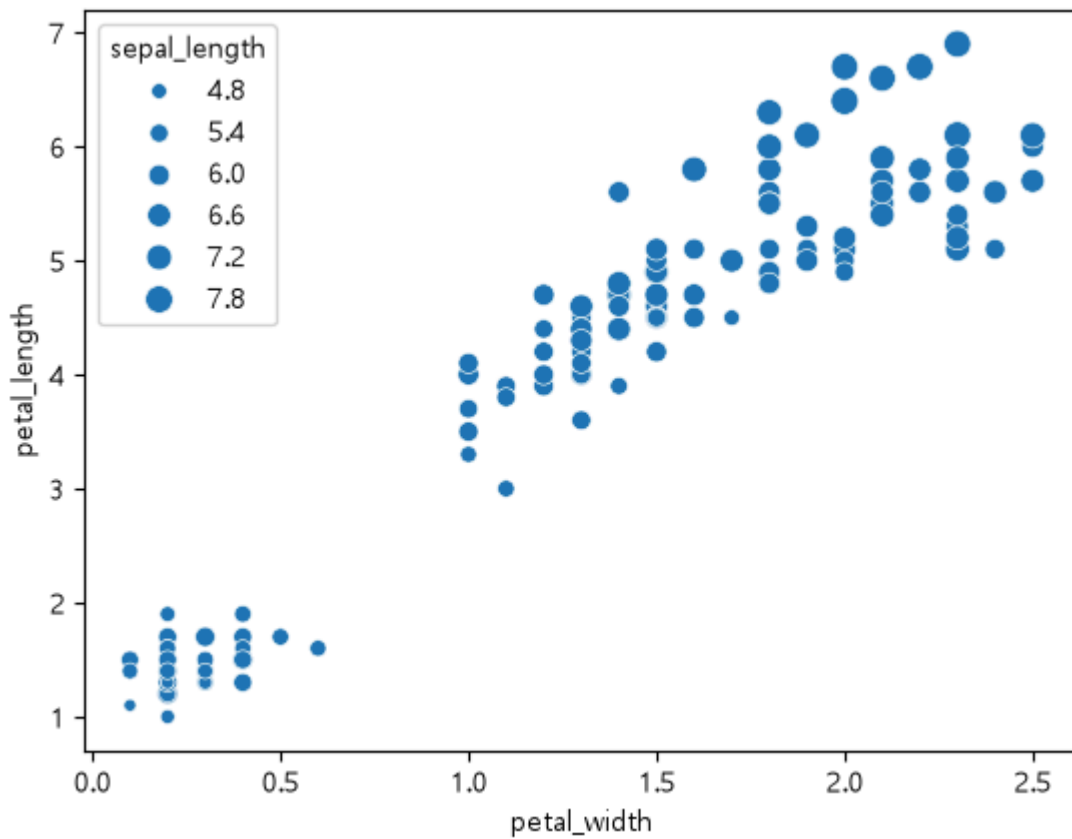
```
In [62]: sns.scatterplot(data=iris, x='petal_width', y='petal_length',  
                        hue='species', size='sepal_length')  
plt.show()
```



```
In [63]: sns.relplot(kind='scatter', data=iris, x='petal_width', y='petal_length',
                    hue='species', size='sepal_length')
plt.show()
```



```
In [65]: sns.scatterplot(data=iris, x='petal_width', y='petal_length',
                        size='sepal_length', sizes=(20, 100))
plt.show()
```



예제데이터. 범죄데이터

```
In [283... crime = pd.read_csv('data/crimeRatesByState2005.csv')
crime.head()
```

```
Out[283...]
   state  murder  forcible_rape  robbery  aggravated_assault  burglary  larceny_theft
0  Alabama      8.2          34.3    141.4              247.8     953.8      2650.
1   Alaska      4.8          81.1     80.9              465.1     622.5      2599.
2  Arizona      7.5          33.8    144.4              327.4     948.4      2965.
3  Arkansas      6.7          42.9     91.1              386.8    1084.6      2711.
4  California      6.9          26.0    176.1              317.3     693.3      1916.
```

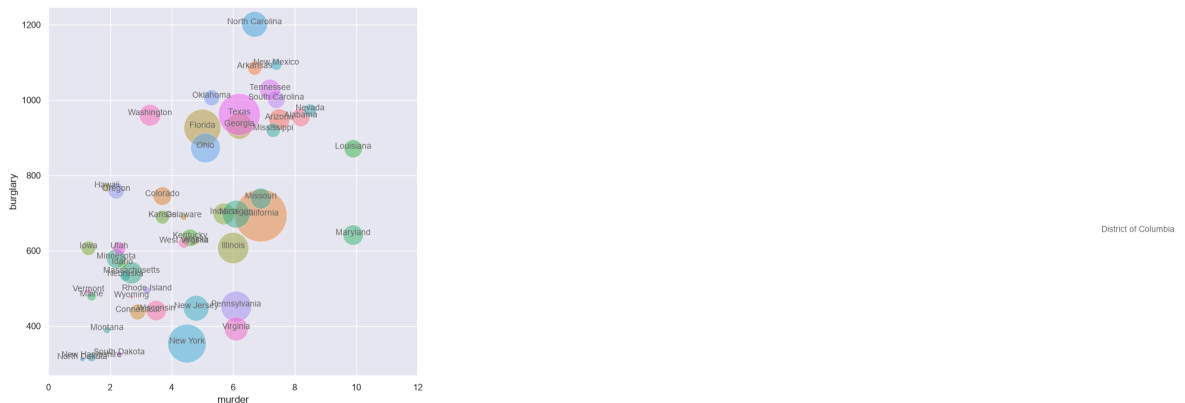


```
In [284... crime.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 51 entries, 0 to 50
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   state                  51 non-null    str
1   murder                 51 non-null    float64
2   forcible_rape          51 non-null    float64
3   robbery                51 non-null    float64
4   aggravated_assault     51 non-null    float64
5   burglary               51 non-null    float64
6   larceny_theft          51 non-null    float64
7   motor_vehicle_theft    51 non-null    float64
8   population             51 non-null    int64
dtypes: float64(7), int64(1), str(1)
memory usage: 3.7 KB
```

In [293...]

```
# 살인건수(murder)와 절도건수(burglary), 주별 인구수에 대한 버블차트
sns.set_theme(rc={'figure.figsize':(8, 8)})
sns.scatterplot(data=crime, x='murder', y='burglary',
                size='population', sizes=(20, 4000),
                hue='state', alpha=0.5,
                legend=False)
plt.xlim(0, 12)
# 주 이름을 버블 위에 표시
for i in range(0, crime.shape[0]):
    plt.text(x=crime.murder[i], y=crime.burglary[i], s=crime.state[i],
            horizontalalignment='center',
            size='small', color='dimgray')
plt.show()
```



3. 집단 간 비교

: 그룹별 데이터 분포 비교

- FacetGrid를 이용한 범주형 플롯 시각화
- 막대그래프(barplot)
 - matplotlib.pyplot.bar()
 - matplotlib.pyplot.barh()
 - DataFrame.plot(kind='bar'|'barh')
 - DataFrame.plot.bar()
 - DataFrame.plot.barh()
 - seaborn.barplot()

- seaborn.catplot(kind='bar')
- **바이올린플롯(violin plot)**
 - matplotlib.pyplot.violinplot()
 - seaborn.violinplot()
 - seaborn.catplot(kind='violin')
- **strip plot**
 - seaborn.stripplot()
 - seaborn.catplot(kind='strip')
- **swarm plot**
 - seaborn.swarmplot()
 - seaborn.catplot(kind='swarm')

```
In [66]: titanic = sns.load_dataset('titanic')
titanic.tail()
```

```
Out[66]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who
886	0	2	male	27.0	0	0	13.00	S	Second	man
887	1	1	female	19.0	0	0	30.00	S	First	woman
888	0	3	female	NaN	1	2	23.45	S	Third	woman
889	1	1	male	26.0	0	0	30.00	C	First	man
890	0	3	male	32.0	0	0	7.75	Q	Third	man

```
In [67]: titanic.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  -
0   survived        891 non-null    int64
1   pclass          891 non-null    int64
2   sex             891 non-null    str
3   age            714 non-null    float64
4   sibsp          891 non-null    int64
5   parch          891 non-null    int64
6   fare           891 non-null    float64
7   embarked       889 non-null    str
8   class          891 non-null    category
9   who            891 non-null    str
10  adult_male     891 non-null    bool
11  deck          203 non-null    category
12  embark_town    889 non-null    str
13  alive          891 non-null    str
14  alone         891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), str(5)
memory usage: 80.7 KB
```

1) 막대 그래프(barplot)

: 범주형 데이터의 범주별 빈도 비교

① matplotlib.pyplot의 막대그래프 함수

- 형식
`bar(x, height, width=0.8, bottom=None, align='center', data=None, ...)`
`barh(y, width, height=0.8, left=None, *, align='center', **kwargs)`

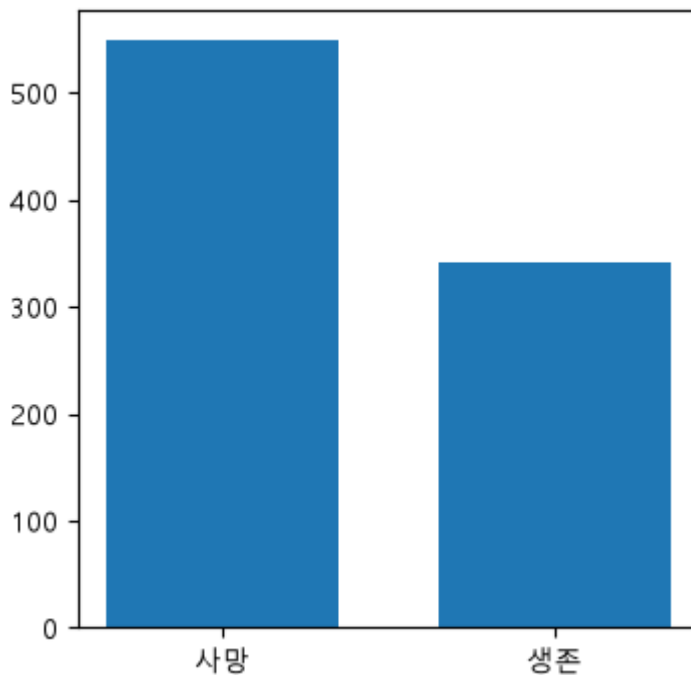
```
In [68]: surv_cnt = titanic.survived.value_counts()  
surv_cnt
```

```
Out[68]: survived  
0      549  
1      342  
Name: count, dtype: int64
```

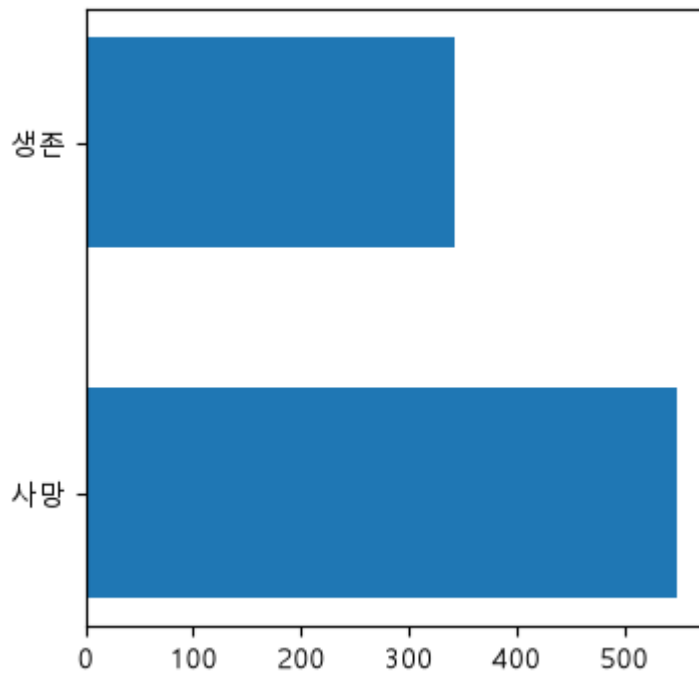
```
In [69]: surv_cnt.index = ['사망', '생존']  
surv_cnt
```

```
Out[69]: 사망      549  
생존      342  
Name: count, dtype: int64
```

```
In [74]: plt.figure(figsize=(4,4))  
plt.bar(x=surv_cnt.index, height=surv_cnt.values, width=0.7)  
plt.show()
```



```
In [75]: plt.figure(figsize=(4,4))  
plt.barh(y=surv_cnt.index, width=surv_cnt.values, height=0.6)  
plt.show()
```



② seaborn의 막대그래프 함수

- 형식

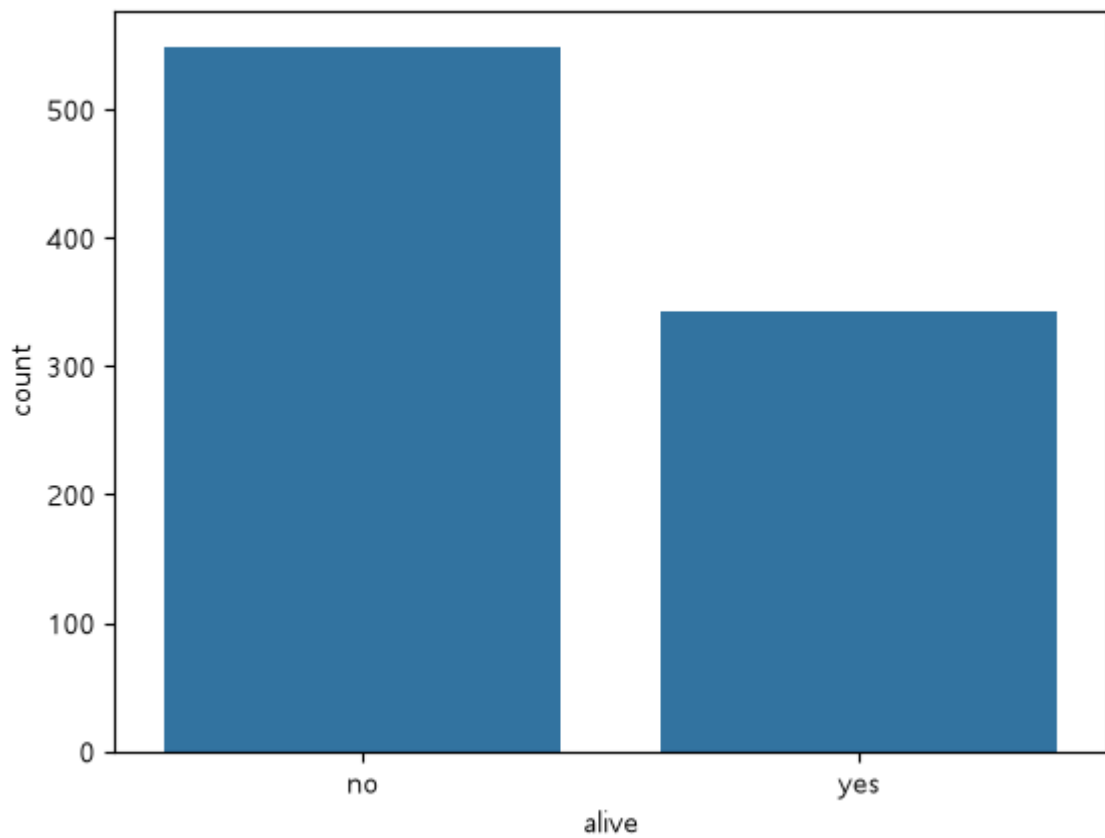
```
barplot(x=None, y=None, hue=None, data=None, order=None,  
orient='v' | 'h', dodge=True, ...)  
countplot(data=None, *, x=None, y=None, hue=None, order=None,  
hue_order=None, ...)  
catplot(kind='bar | count', x=None, y=None, hue=None, data=None,  
row=None, col=None, ...)
```

- <https://seaborn.pydata.org/generated/seaborn.barplot.html#seaborn.barplot>
- <https://seaborn.pydata.org/generated/seaborn.countplot.html>
- <https://seaborn.pydata.org/generated/seaborn.catplot.html#seaborn.catplot>

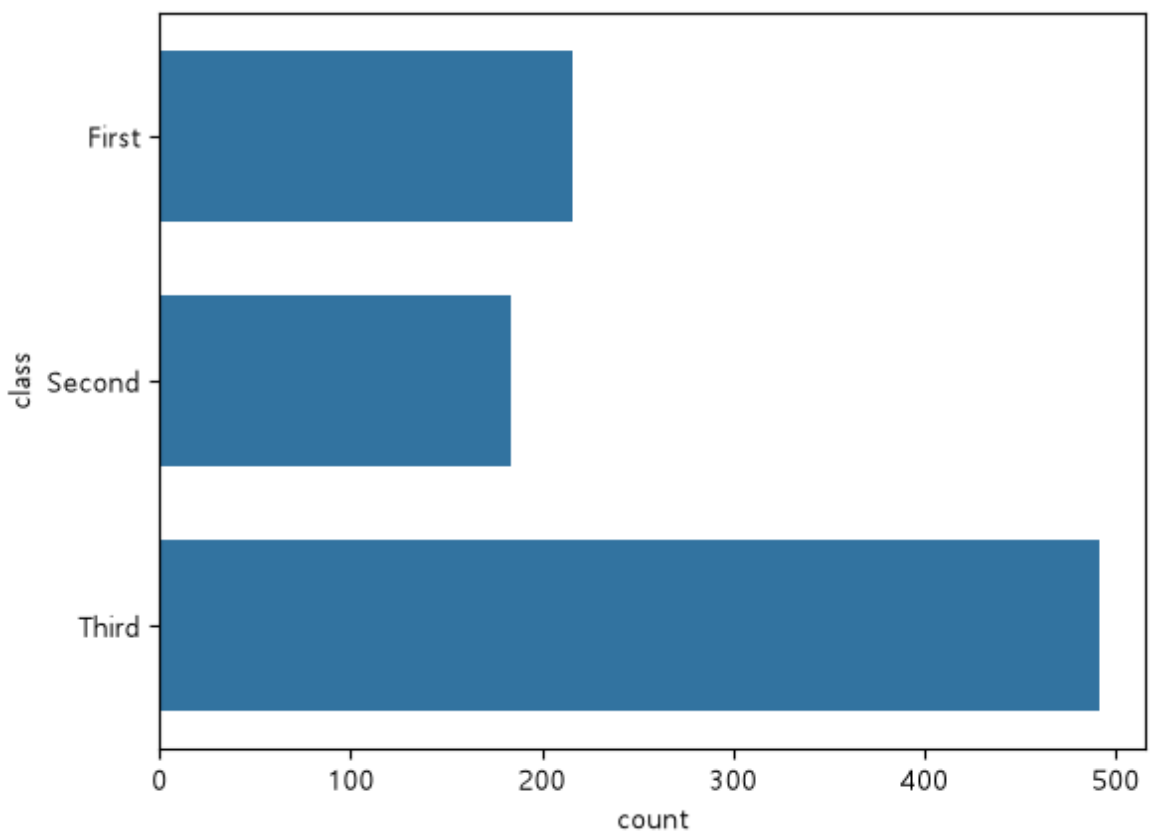
sns.countplot()

: 빈도 플롯

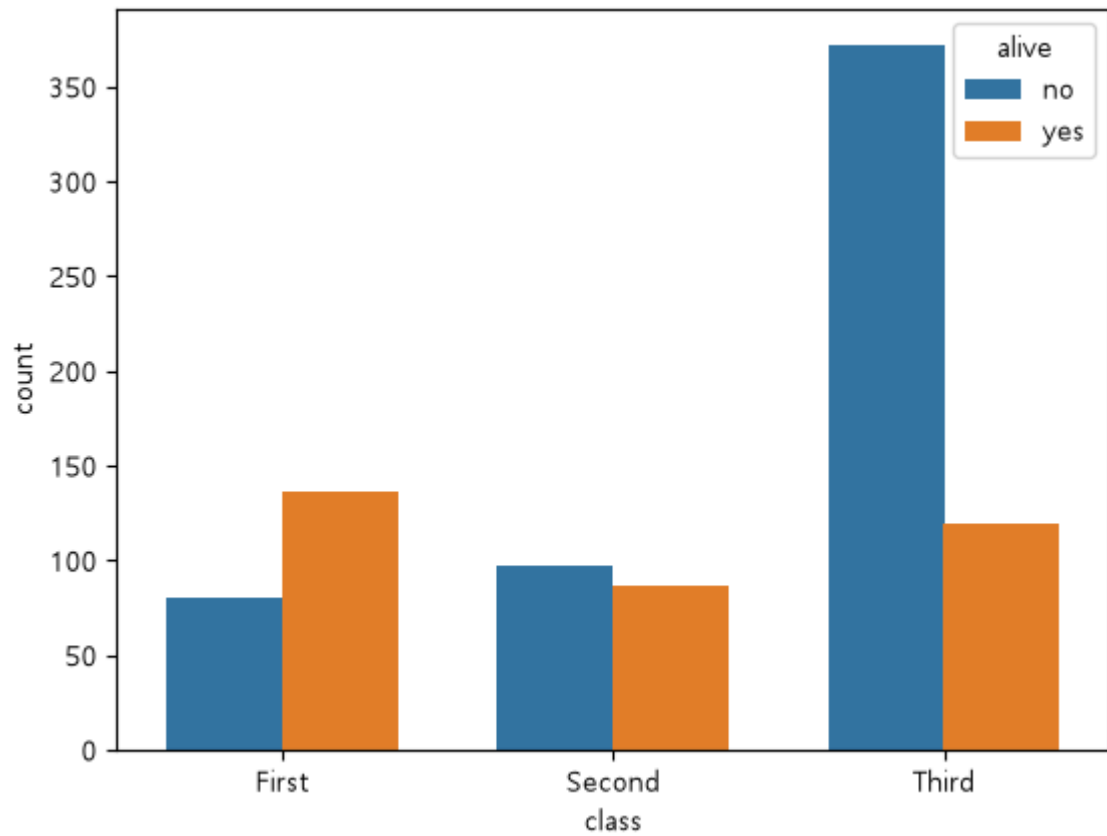
```
In [77]: sns.countplot(data=titanic, x='alive')  
plt.show()
```



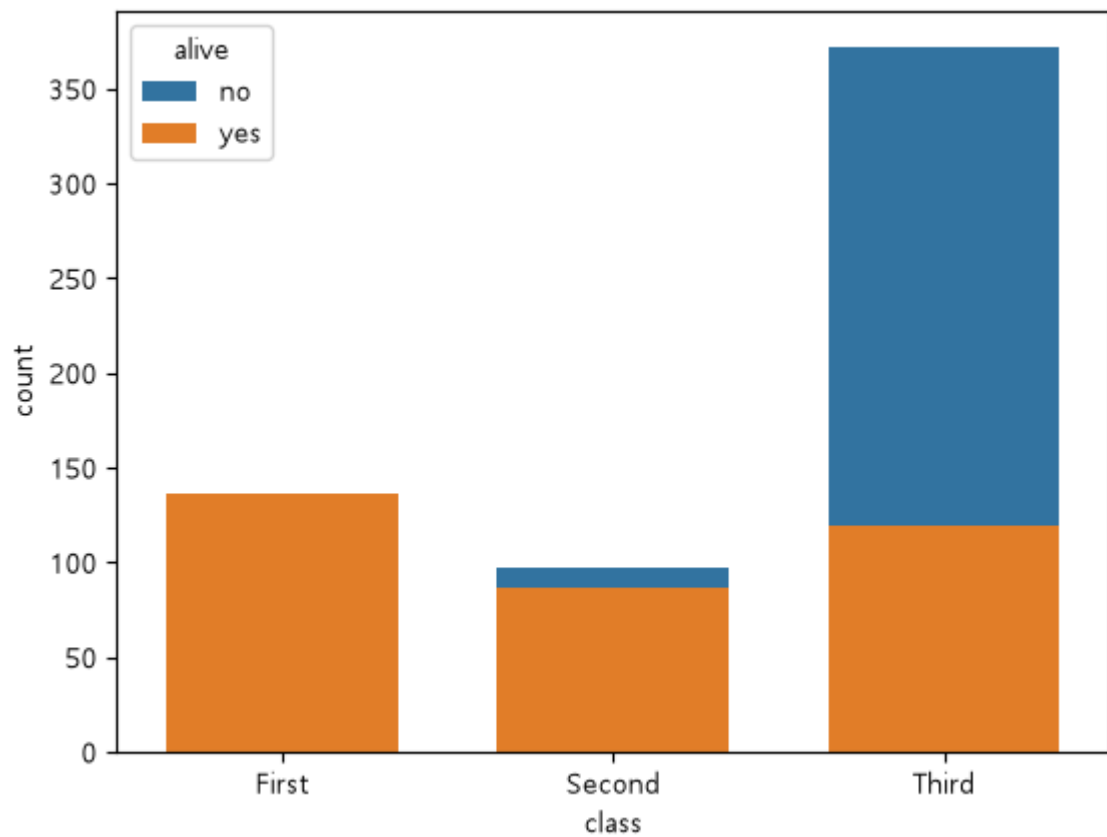
```
In [81]: sns.countplot(data=titanic, y='class', width=0.7)  
plt.show()
```



```
In [84]: sns.countplot(data=titanic, x='class', width=0.7, hue='alive', dodge=True)  
plt.show()
```



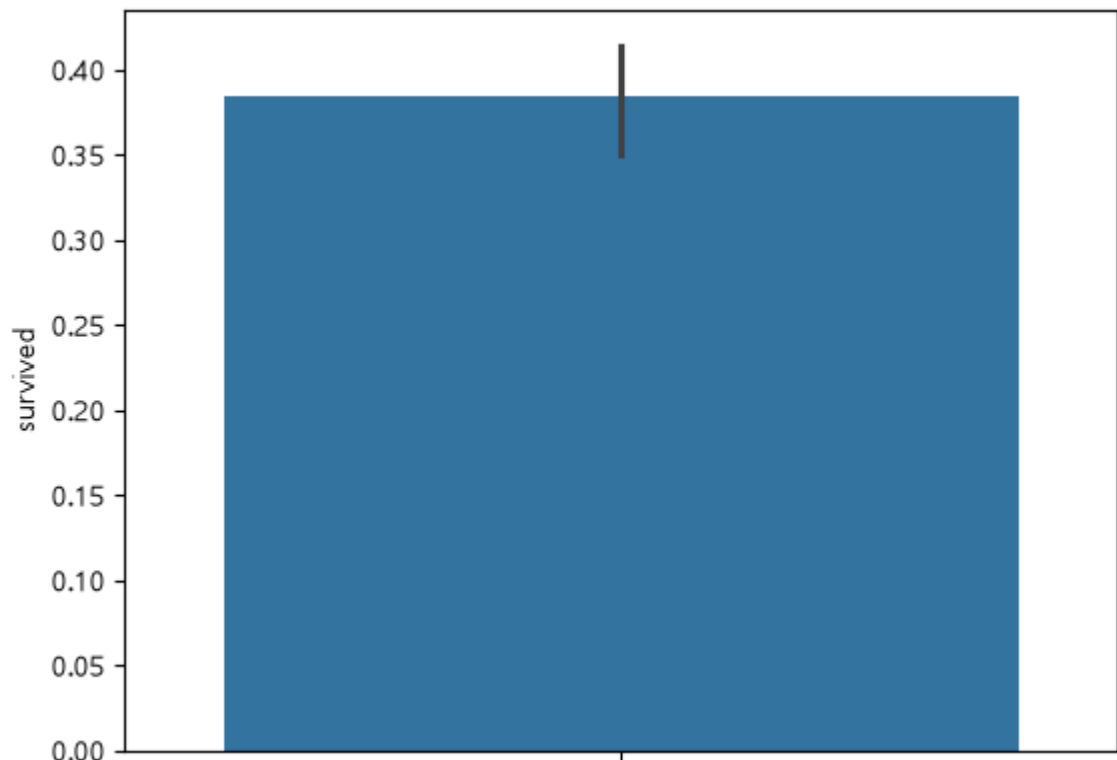
In [86]: `# dodge=False : 겹쳐져 그려짐`
`sns.countplot(data=titanic, x='class', width=0.7, hue='alive', dodge=False)`
`plt.show()`



sns.barplot()

- 데이터의 평균 플롯

```
In [92]: sns.barplot(data=titanic, y='survived')
plt.show()
```



```
In [94]: surv_cnt/891
```

```
Out[94]: 사망    0.616162
생존    0.383838
Name: count, dtype: float64
```

```
In [122... df = pd.crosstab(index=titanic.pclass, columns=titanic.survived,
                    margins=True)
print(df)
df[1]/df['All']
```

survived	0	1	All
pclass			
1	80	136	216
2	97	87	184
3	372	119	491
All	549	342	891

```
Out[122... pclass
1    0.629630
2    0.472826
3    0.242363
All   0.383838
dtype: float64
```

```
In [100... pd.crosstab(index=titanic.pclass, columns=titanic.survived,
                normalize=True, margins=True)
```

Out[100...

survived	0	1	All
----------	---	---	-----

pclass

1	0.089787	0.152637	0.242424
---	----------	----------	----------

2	0.108866	0.097643	0.206510
---	----------	----------	----------

3	0.417508	0.133558	0.551066
---	----------	----------	----------

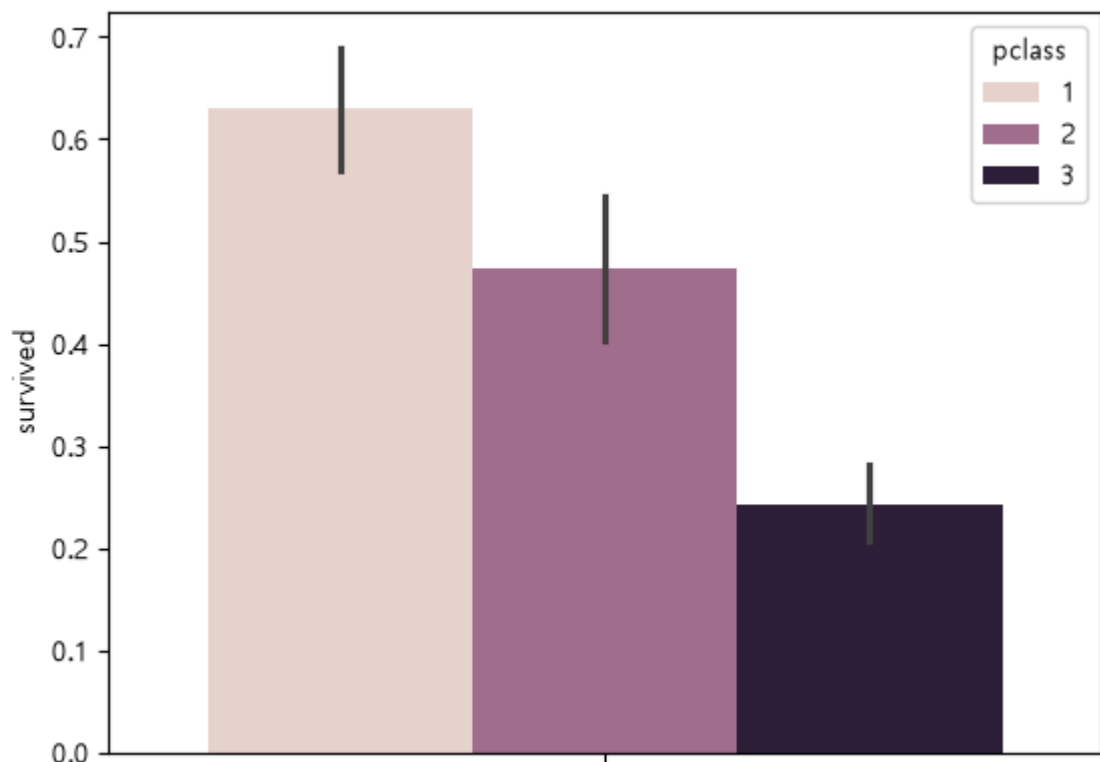
All	0.616162	0.383838	1.000000
-----	----------	----------	----------

```
In [123... df = pd.crosstab(index=titanic.sex, columns=titanic.survived,
                    margins=True)
print(df)
df[1]/df['All']
```

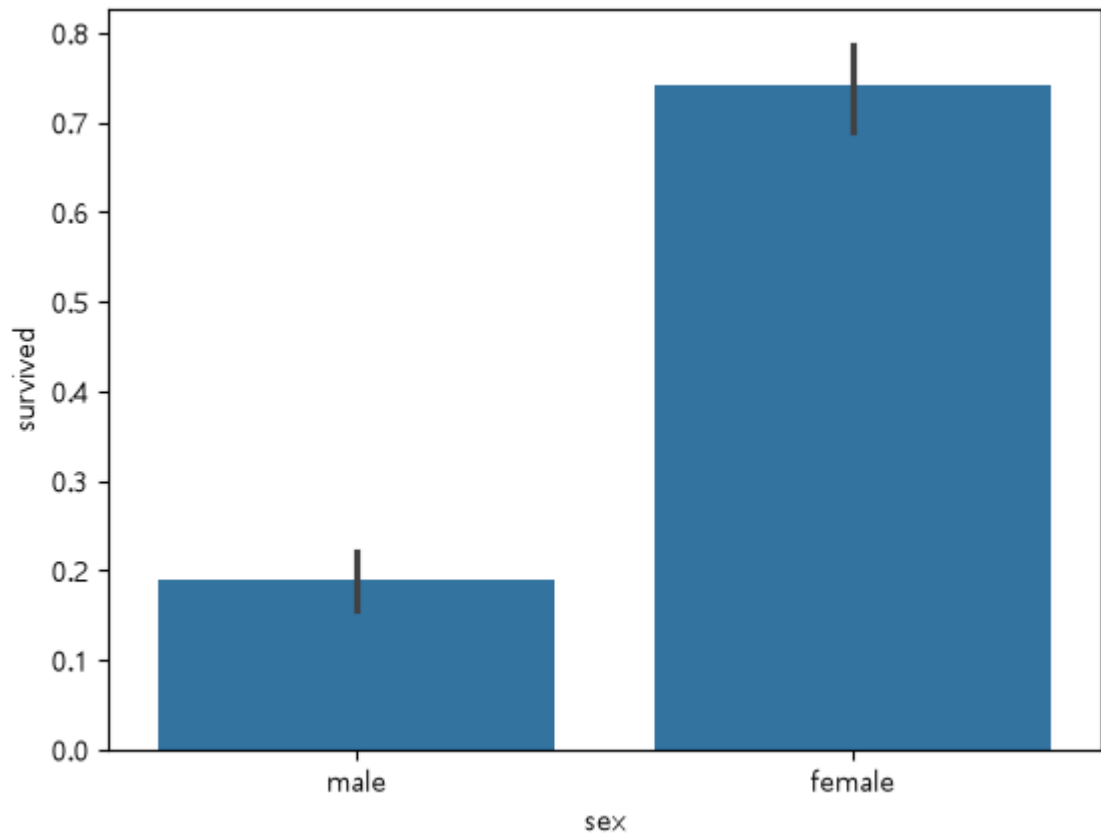
survived	0	1	All
sex			
female	81	233	314
male	468	109	577
All	549	342	891

Out[123... sex
female 0.742038
male 0.188908
All 0.383838
dtype: float64

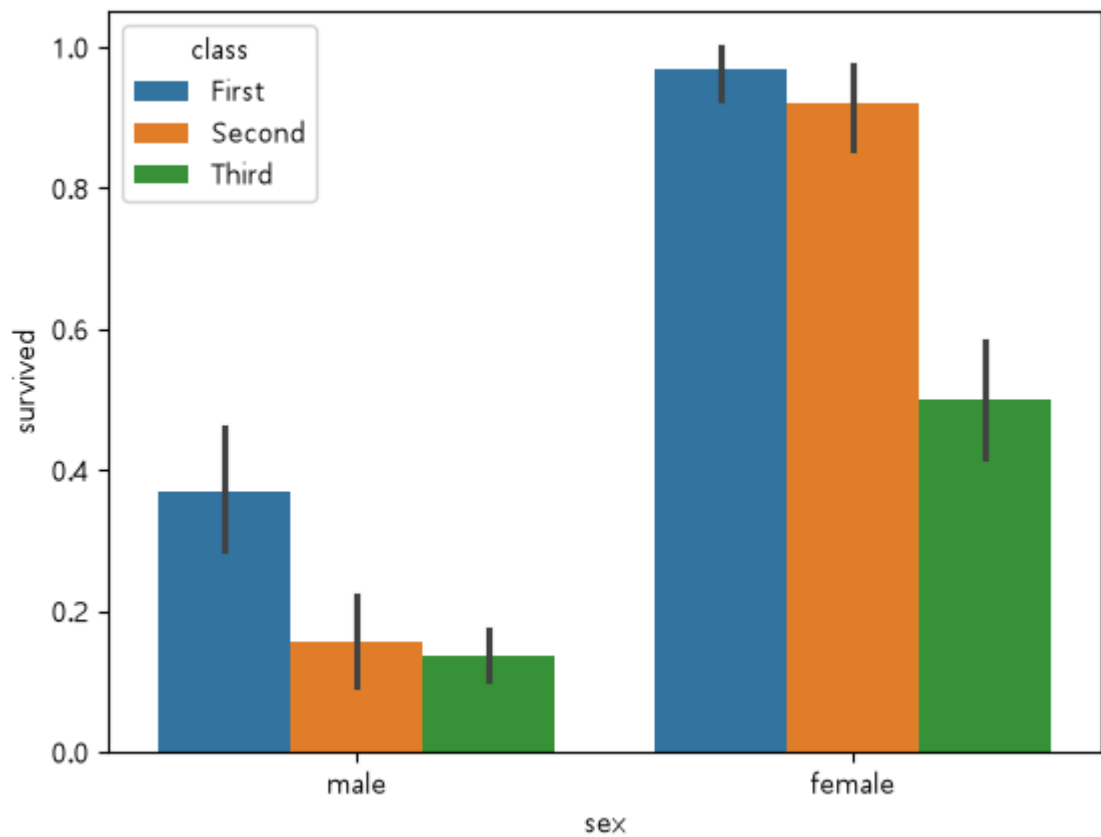
```
In [102... sns.barplot(data=titanic, y='survived', hue='pclass')
plt.show()
```



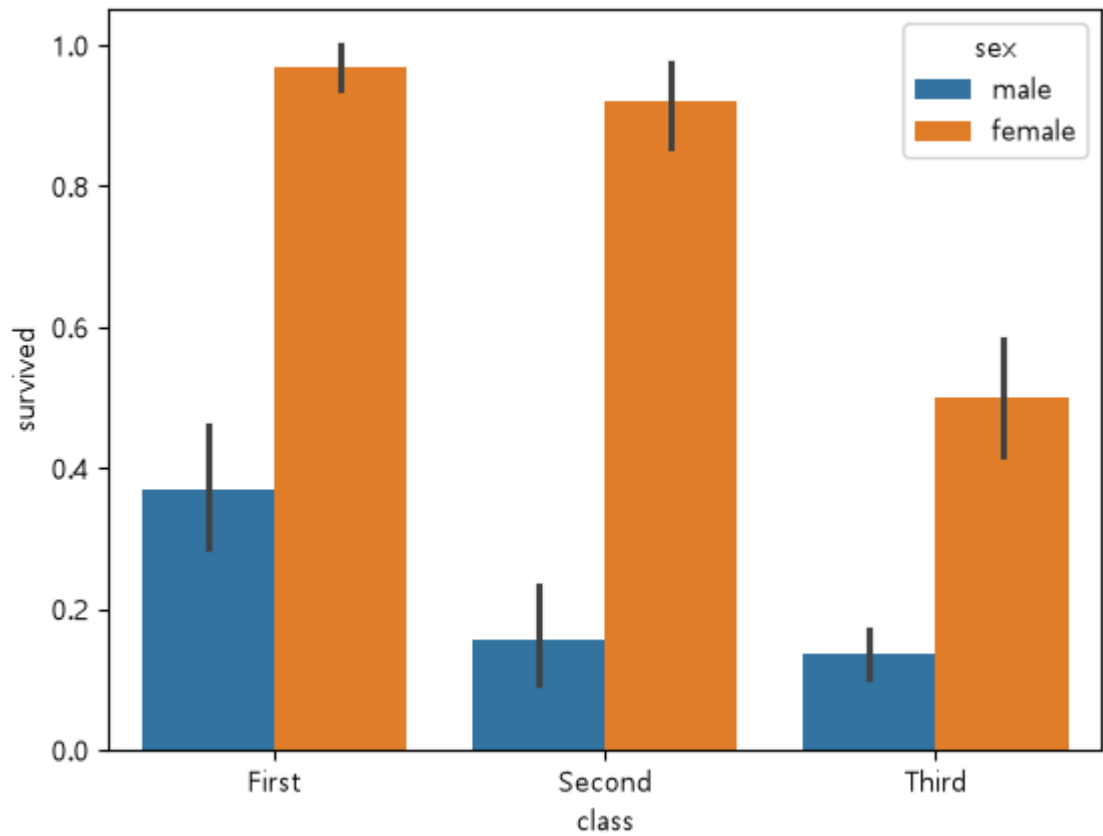
```
In [97]: # 성별에 따른 생존율
sns.barplot(data=titanic, x='sex', y='survived')
plt.show()
```



In [98]: # 성별과 선실등급에 따른 생존율
 sns.barplot(data=titanic, x='sex', y='survived', hue='class')
 plt.show()



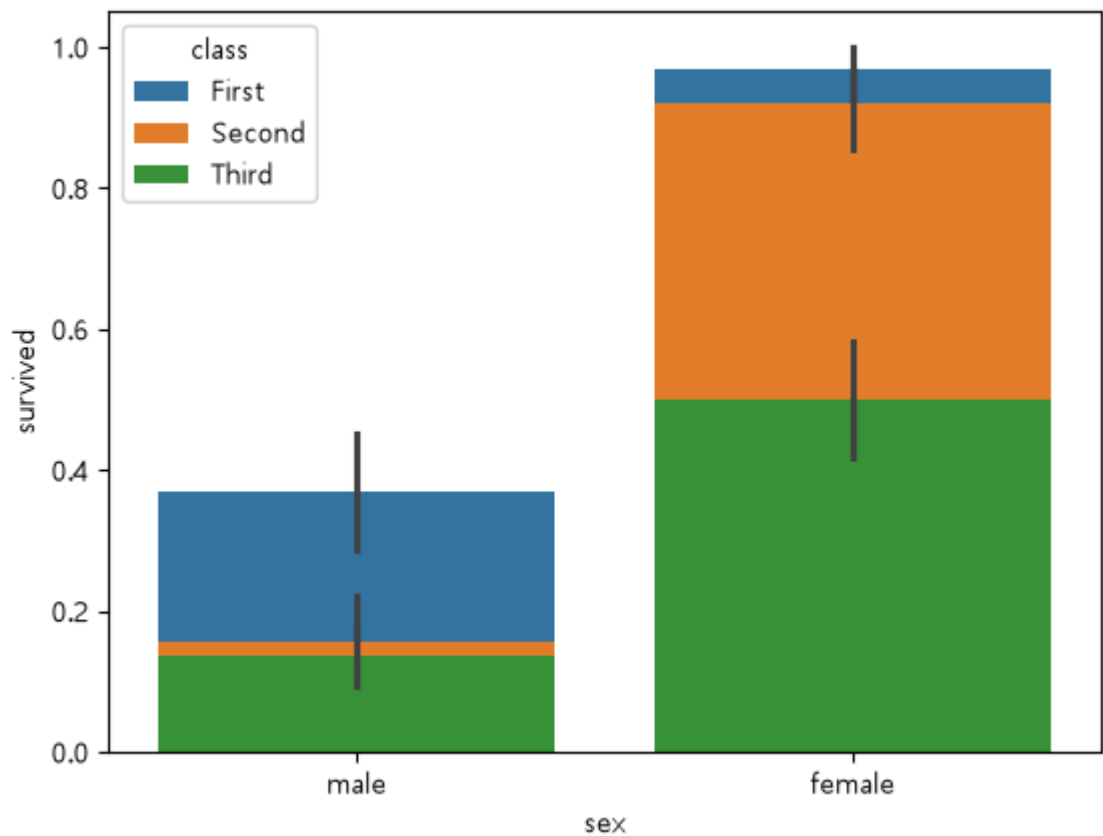
In [104... sns.barplot(data=titanic, x='class', y='survived', hue='sex')
 plt.show()



`sns.barplot(dodge=False)`

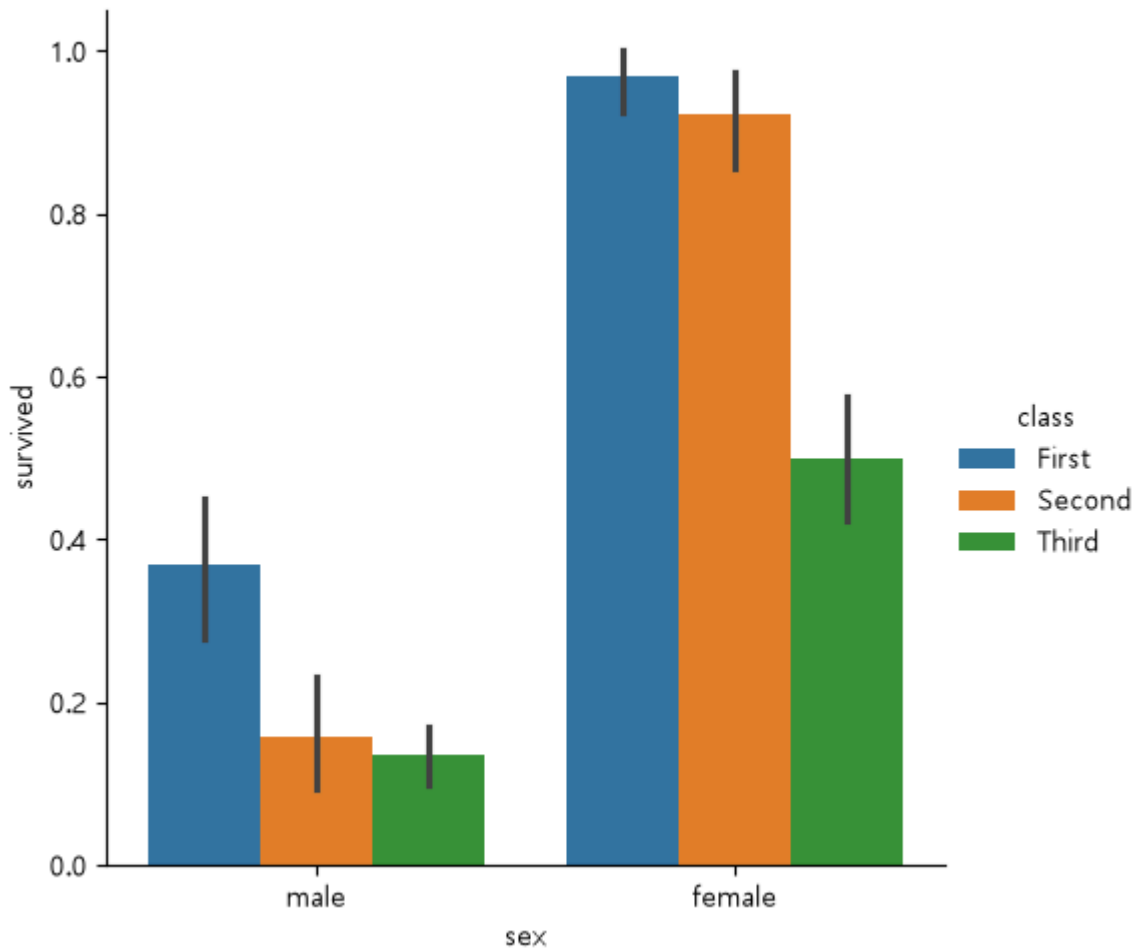
: 겹쳐서 그림

In [105... `sns.barplot(data=titanic, x='sex', y='survived', hue='class', dodge=False)`
`plt.show()`



sns.catplot(kind='bar')

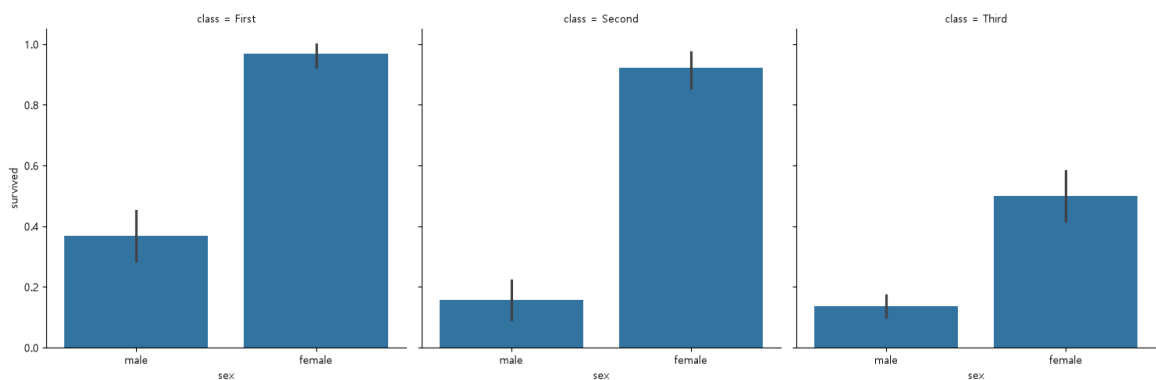
```
In [106... sns.catplot(kind='bar', data=titanic, x='sex', y='survived', hue='class')
plt.show()
```



sns.catplot(col=)

: 윈도우에 범주별 막대그래프를 따로 표현

```
In [107... sns.catplot(kind='bar', data=titanic, x='sex', y='survived', col='class')
plt.show()
```



참고. countplot()과 barplot() 비교

```
In [108... data = pd.DataFrame({'cate':['A','B','A','A','B','A','B','A'],  
                      'value':[10, 13, 12, 8, 9, 5, 10, 20]})  
data
```

```
Out[108...  


|   | cate | value |
|---|------|-------|
| 0 | A    | 10    |
| 1 | B    | 13    |
| 2 | A    | 12    |
| 3 | A    | 8     |
| 4 | B    | 9     |
| 5 | A    | 5     |
| 6 | B    | 10    |
| 7 | A    | 20    |


```

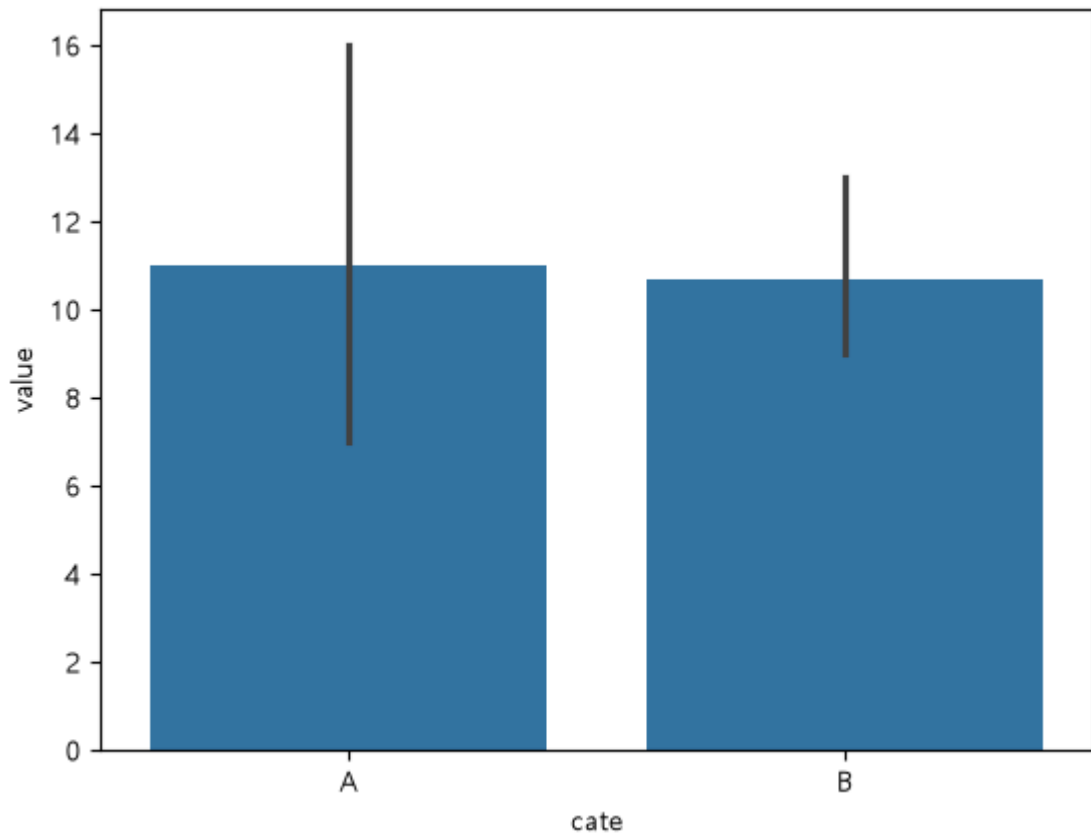
```
In [113... data.groupby('cate').mean()
```

```
Out[113...  

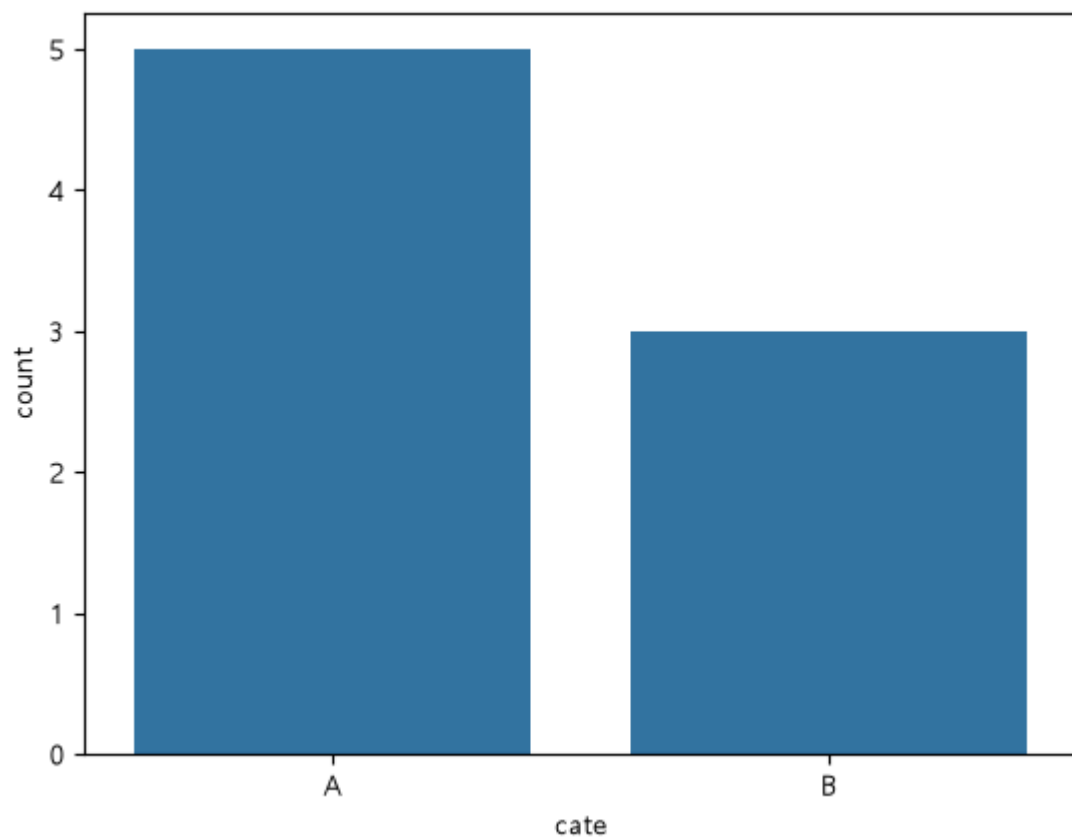

|      | value     |
|------|-----------|
| cate |           |
| A    | 11.000000 |
| B    | 10.666667 |


```

```
In [112... sns.barplot(data=data, x='cate', y='value')  
# sns.barplot(data=data, x='cate') # 수치값이 아니므로 평균계산 불가  
  
plt.show()
```



```
In [110... sns.countplot(data=data, x='cate')  
plt.show()
```

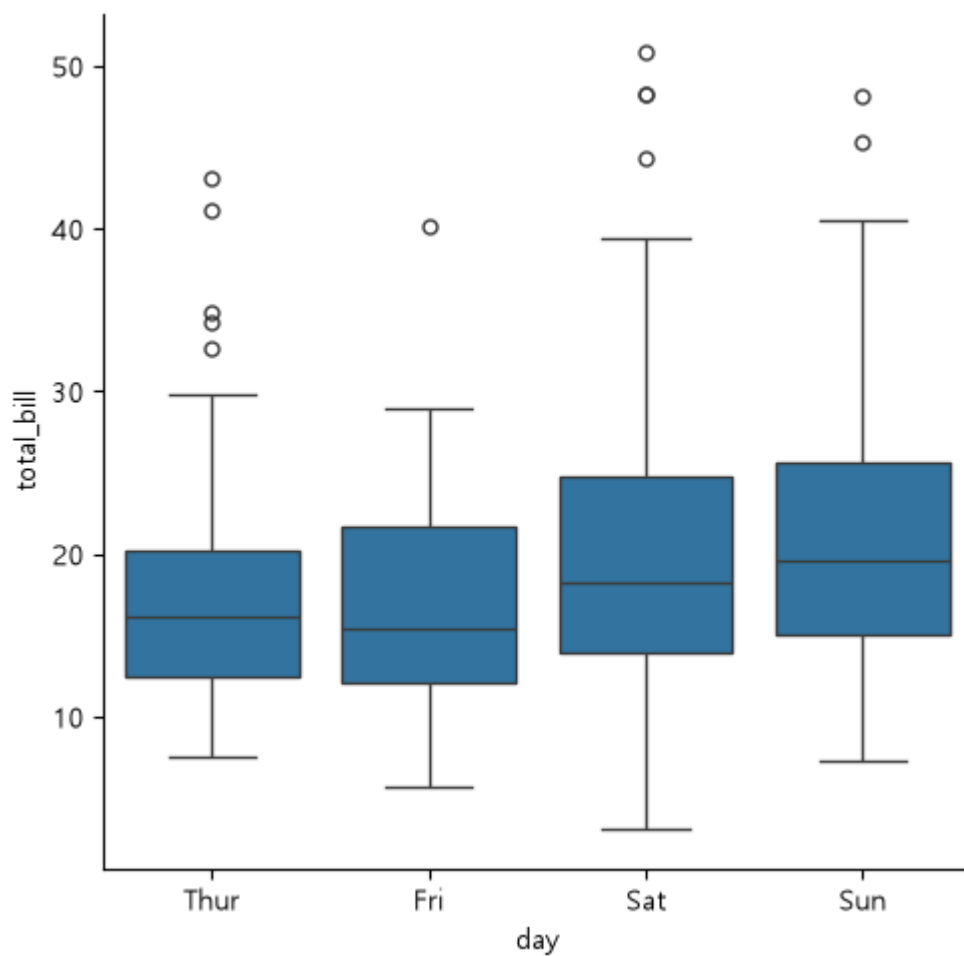


연습1. Tips 데이터에서 요일별 평균결제금액 비교하기

```
In [124... tips = sns.load_dataset('tips')
tips.head()
```

```
Out[124...
   total_bill  tip  sex  smoker  day  time  size
0      16.99  1.01 Female     No  Sun  Dinner    2
1      10.34  1.66  Male     No  Sun  Dinner    3
2      21.01  3.50  Male     No  Sun  Dinner    3
3      23.68  3.31  Male     No  Sun  Dinner    2
4      24.59  3.61 Female     No  Sun  Dinner    4
```

```
In [129... sns.catplot(kind='box', data=tips, y='total_bill', x='day')
plt.show()
```



```
In [125... pd.pivot_table(data=tips, index='day', values='total_bill', aggfunc='mean')
```

Out[125...

total_bill	
day	
Thur	17.682742
Fri	17.151579
Sat	20.441379
Sun	21.410000

In []:

연습2. Tips 데이터에서 성별 인원수 시각화

In []:

In []:

2) 바이올린 플롯(violin plot)

- 박스플롯의 기능과 함께 데이터의 커널밀도곡선을 제시
- 데이터의 분포와 범위를 한눈에 보기 쉽게 나타내는 그래프
- 더 실제에 가까운 분포를 알 수 있음

예제 데이터. iris dataset

In [130...

```
iris.tail()
```

Out[130...

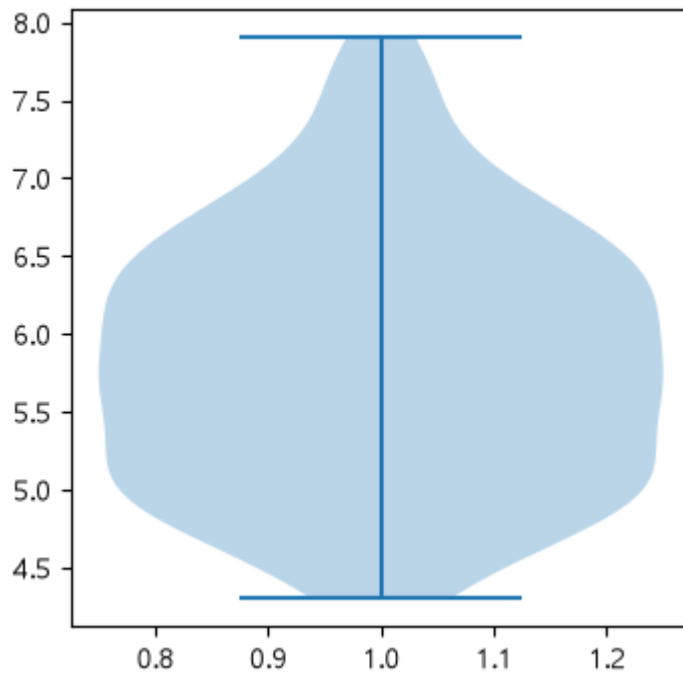
	sepal_length	sepal_width	petal_length	petal_width	species
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

① matplotlib.pyplot의 바이올린 플롯 함수

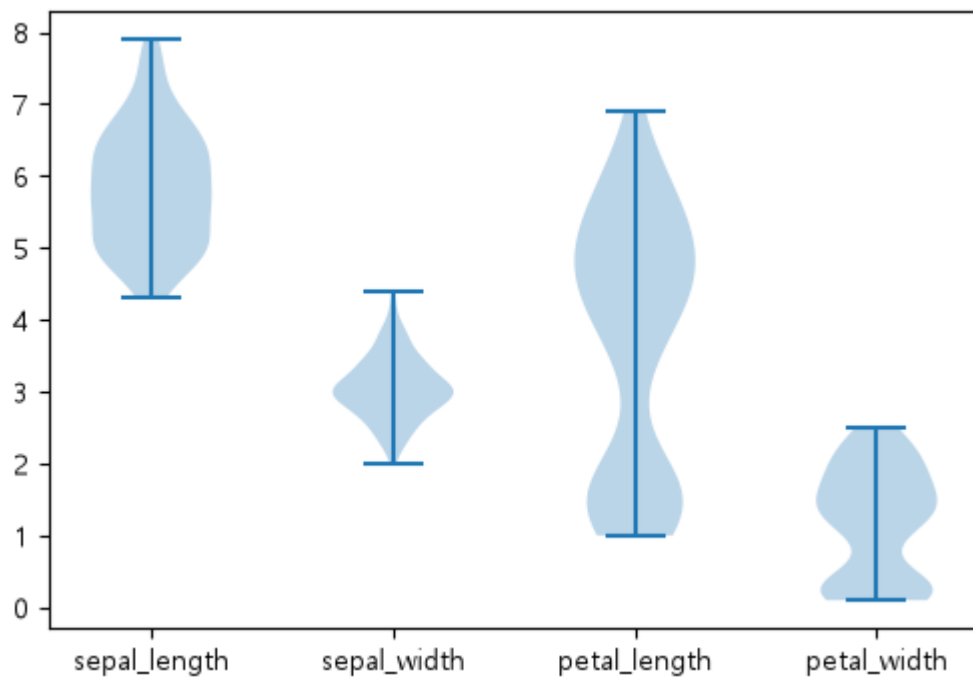
- 형식
`violinplot(dataset, positions=None, vert=True, widths=0.5, *, data=None)`
- https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.violinplot.html

In [131...

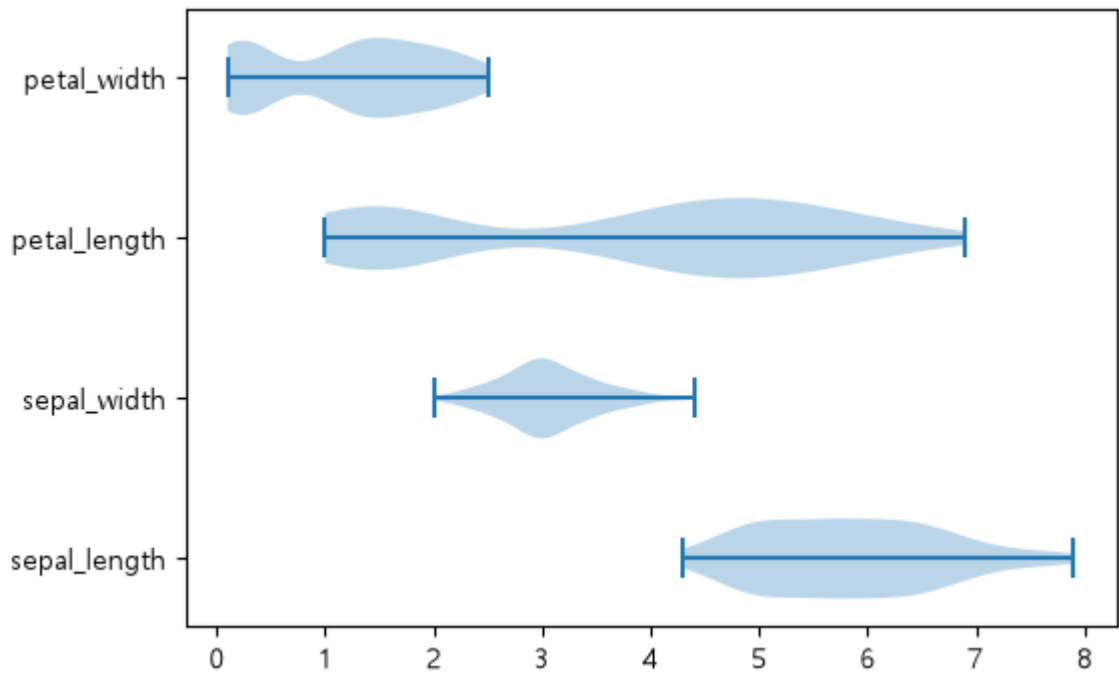
```
plt.figure(figsize=(4,4))
plt.violinplot(iris.sepal_length)
plt.show()
```



```
In [133... plt.figure(figsize=(6,4))
plt.violinplot(iris.iloc[:, :-1])
plt.xticks([1,2,3,4], labels=iris.columns[: -1])
plt.show()
```



```
In [137... plt.figure(figsize=(6,4))
plt.violinplot(iris.iloc[:, :-1], orientation='horizontal')
plt.yticks([1,2,3,4], labels=iris.columns[: -1])
plt.show()
```

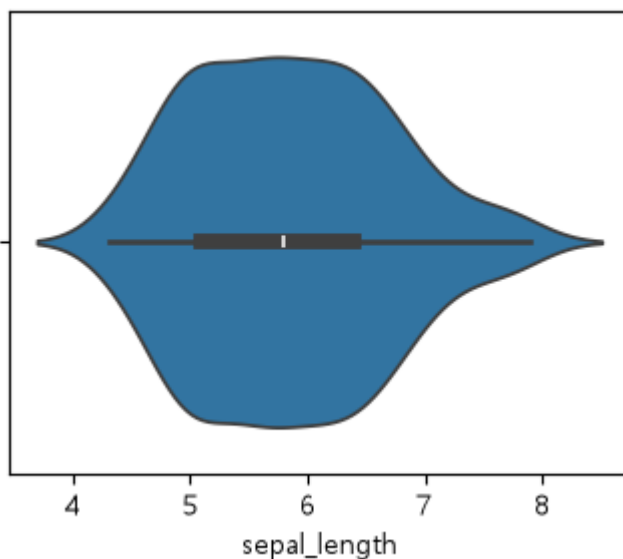


② seaborn의 바이올린 플롯 함수

- 형식
`violinplot(x=None, y=None, hue=None, data=None, order=None, ...)`
`catplot(kind='violin', x=None, y=None, hue=None, data=None, row=None, col=None, ...)`
- <https://seaborn.pydata.org/generated/seaborn.violinplot.html#seaborn.violinplot>
- <https://seaborn.pydata.org/generated/seaborn.catplot.html#seaborn.catplot>

sns.violinplot()

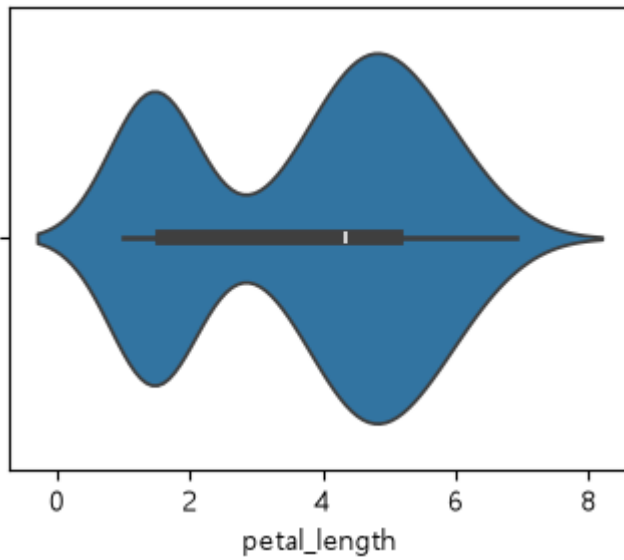
```
In [139... plt.figure(figsize=(4,3))
sns.violinplot(data=iris, x='sepal_length')
plt.show()
```



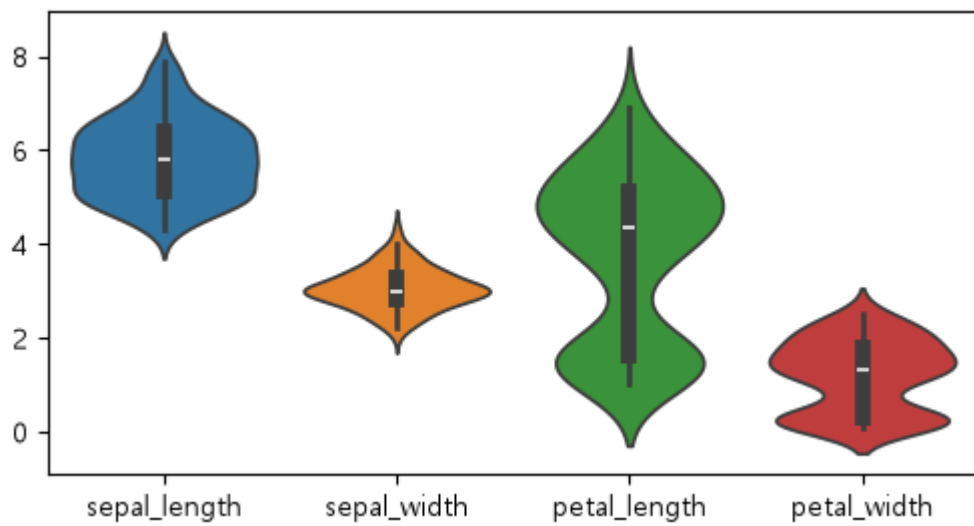
```
In [140... plt.figure(figsize=(4,3))
sns.violinplot(data=iris, x='petal_length')
```



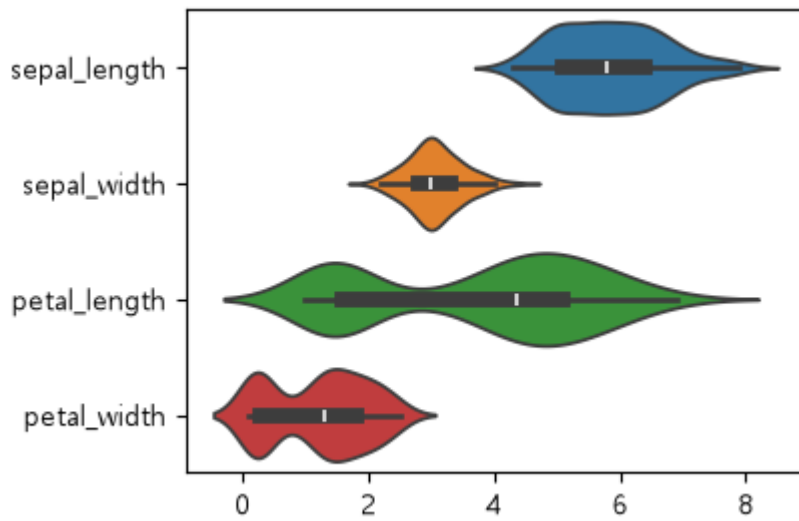
```
plt.show()
```



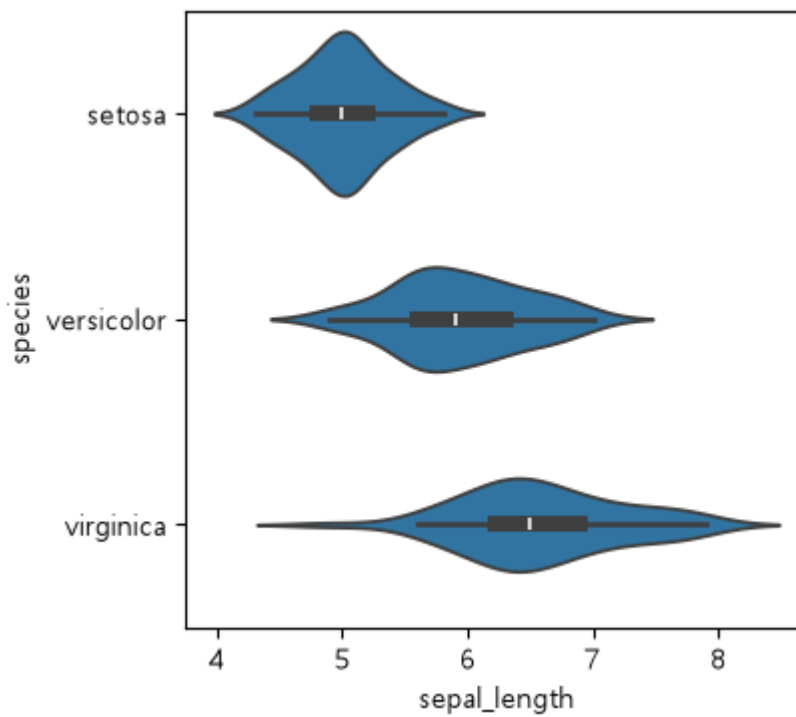
```
In [141... plt.figure(figsize=(6,3))
sns.violinplot(data=iris)
plt.show()
```



```
In [142... plt.figure(figsize=(4,3))
sns.violinplot(data=iris, orient='h')
plt.show()
```

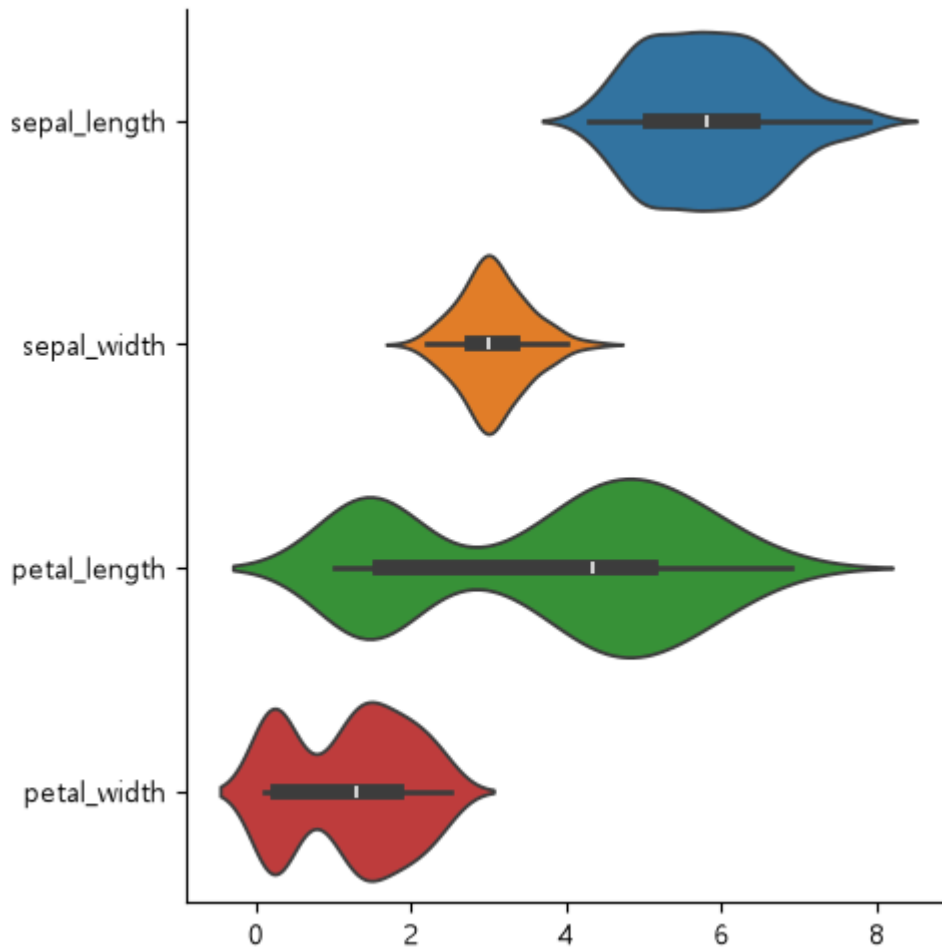


```
In [144... plt.figure(figsize=(4,4))
sns.violinplot(data=iris, x='sepal_length', y='species')
plt.show()
```

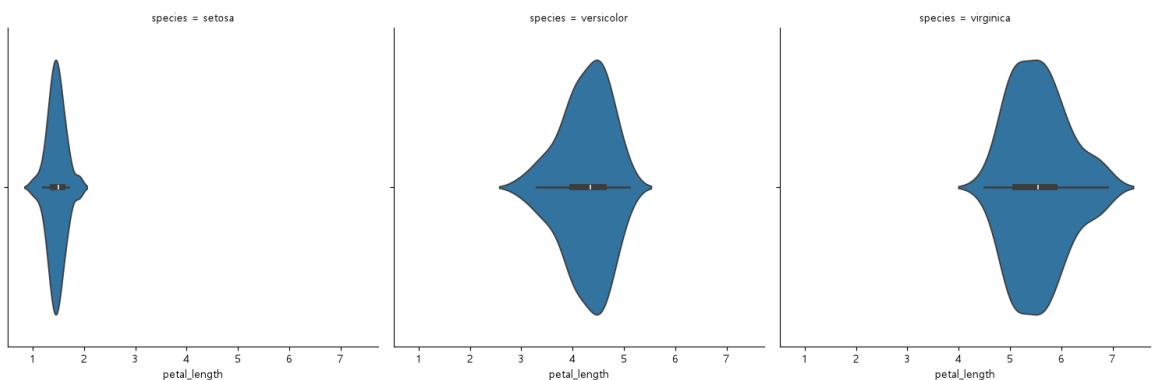


sns.catplot(kind='violin')

```
In [145... sns.catplot(kind='violin', data=iris, orient='h')
plt.show()
```



In [146... `sns.catplot(kind='violin', data=iris, x='petal_length', col='species', orient='h', plt.show())`



3) Strip plot

: 겹치는 데이터를 파악할 수 있도록 같은 값에 jitter를 통해 겹치지 않게 그려주는 그래프

- 형식
`stripplot(x=None, y=None, hue=None, data=None, order=None, jitter=True, ...)`
`catplot(kind='strip', x=None, y=None, hue=None, data=None, row=None, col=None, ...)`
- <https://seaborn.pydata.org/generated/seaborn.stripplot.html#seaborn.stripplot>

- <https://seaborn.pydata.org/generated/seaborn.catplot.html#seaborn.catplot>

sns.stripplot()

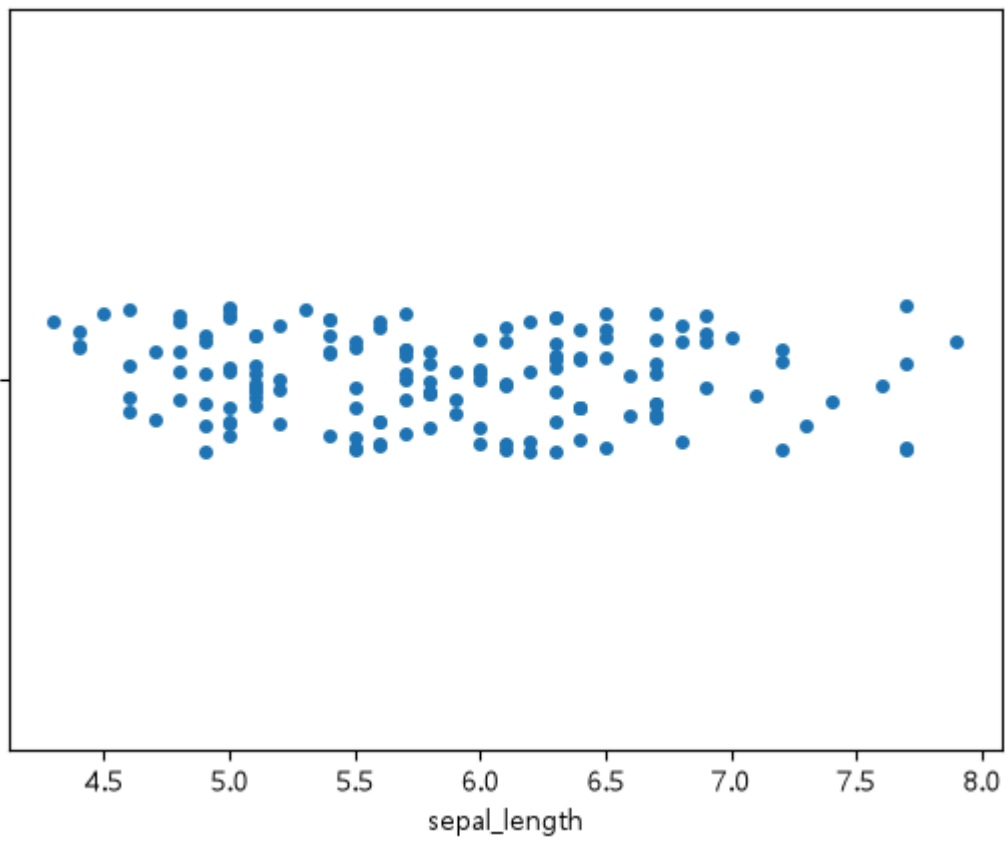
```
In [147... iris.sepal_length.unique()
```

```
Out[147... array([5.1, 4.9, 4.7, 4.6, 5. , 5.4, 4.4, 4.8, 4.3, 5.8, 5.7, 5.2, 5.5,  
        4.5, 5.3, 7. , 6.4, 6.9, 6.5, 6.3, 6.6, 5.9, 6. , 6.1, 5.6, 6.7,  
        6.2, 6.8, 7.1, 7.6, 7.3, 7.2, 7.7, 7.4, 7.9])
```

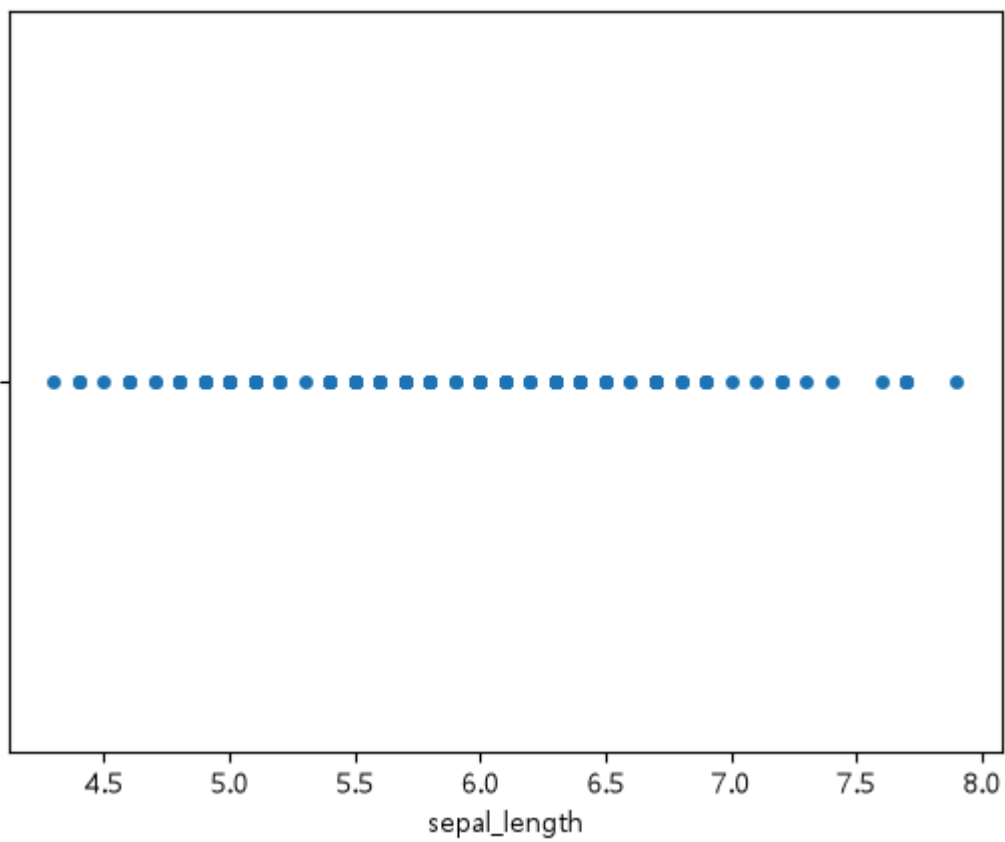
```
In [148... iris.sepal_length.value_counts().sort_index()
```

```
Out[148... sepal_length  
4.3      1  
4.4      3  
4.5      1  
4.6      4  
4.7      2  
4.8      5  
4.9      6  
5.0     10  
5.1      9  
5.2      4  
5.3      1  
5.4      6  
5.5      7  
5.6      6  
5.7      8  
5.8      7  
5.9      3  
6.0      6  
6.1      6  
6.2      4  
6.3      9  
6.4      7  
6.5      5  
6.6      2  
6.7      8  
6.8      3  
6.9      4  
7.0      1  
7.1      1  
7.2      3  
7.3      1  
7.4      1  
7.6      1  
7.7      4  
7.9      1  
Name: count, dtype: int64
```

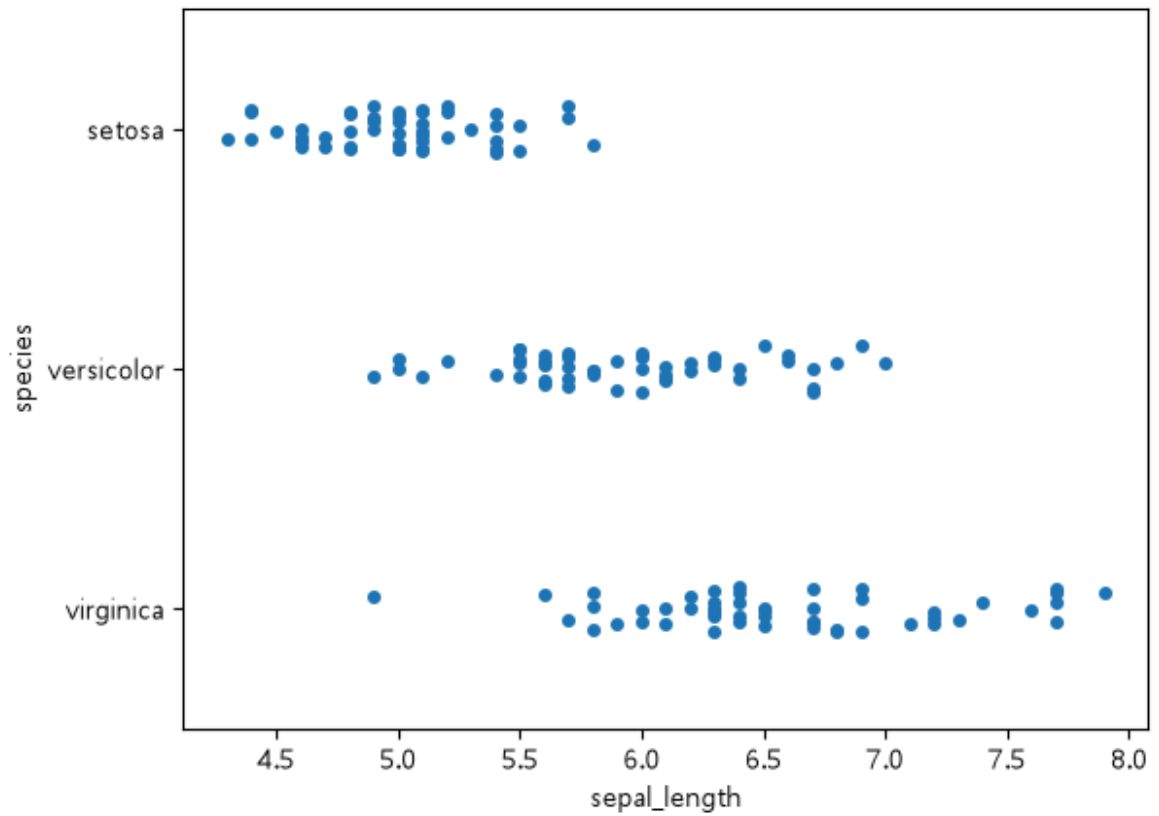
```
In [150... sns.stripplot(data=iris, x='sepal_length')  
plt.show()
```



```
In [151... sns.stripplot(data=iris, x='sepal_length', jitter=False)  
plt.show()
```



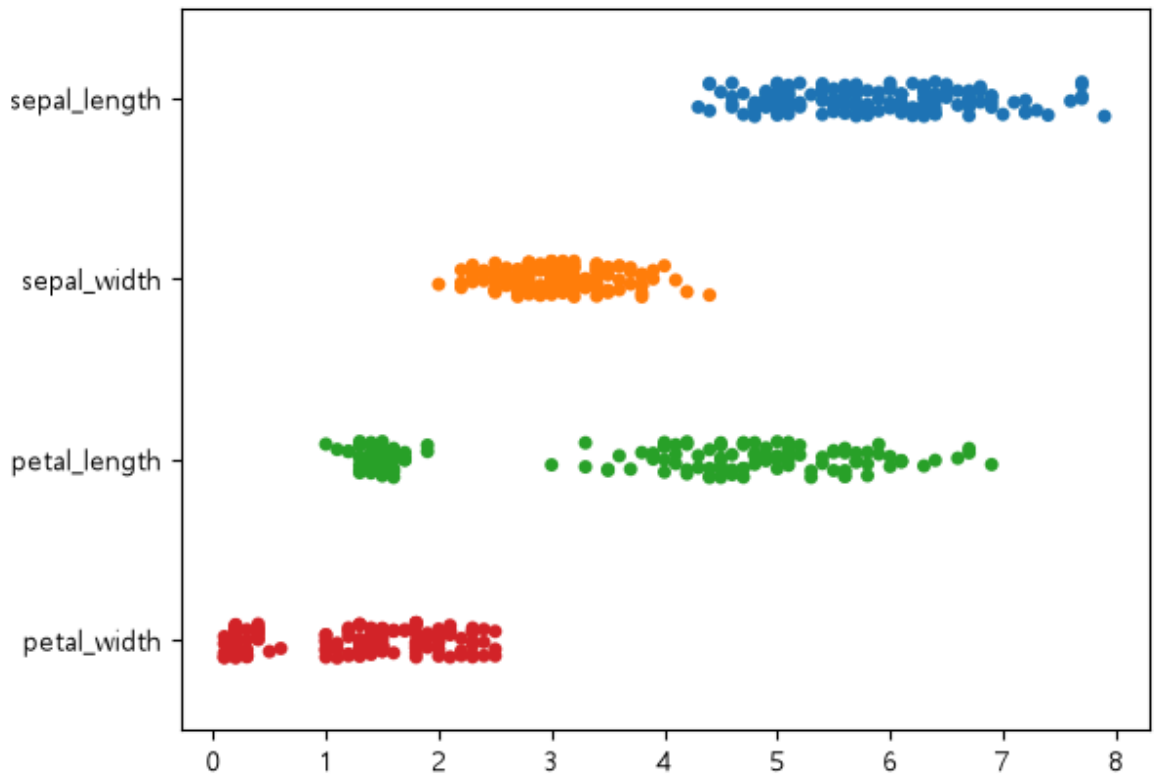
```
In [152... sns.stripplot(data=iris, x='sepal_length', y='species')  
plt.show()
```



In [155...

```
# sns.stripplot(data=iris)
sns.stripplot(data=iris, orient='h')

plt.show()
```



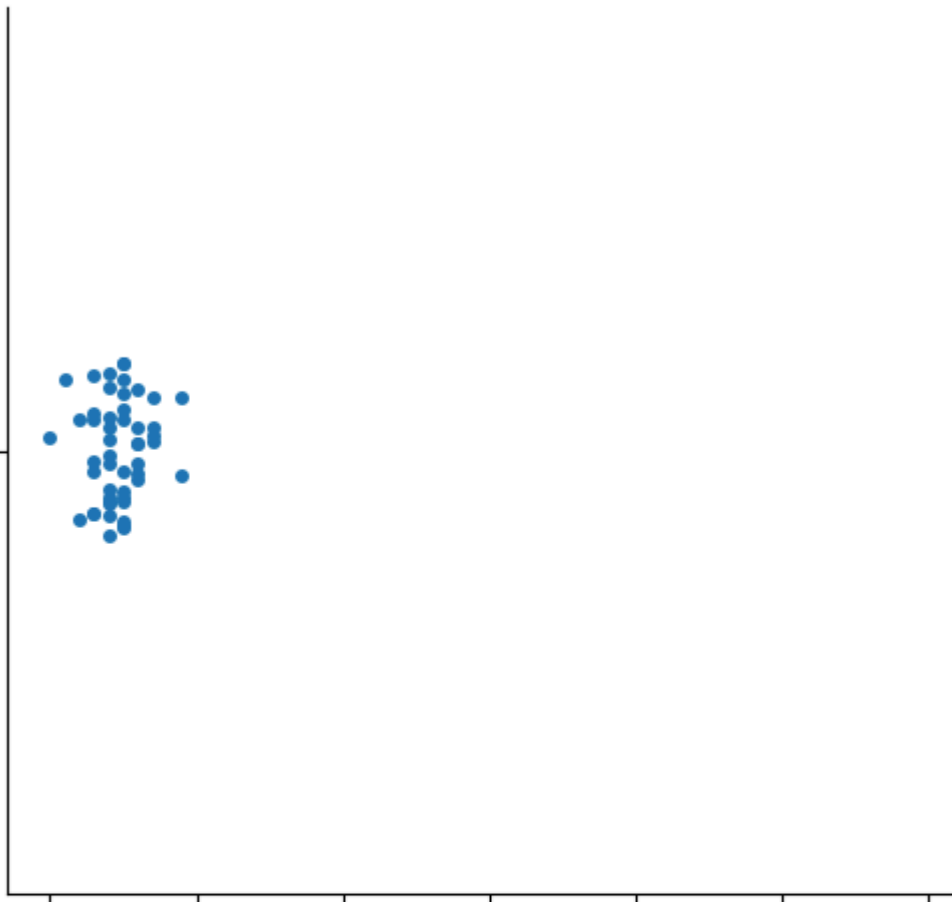
sns.catplot(kind='strip')

In [157...

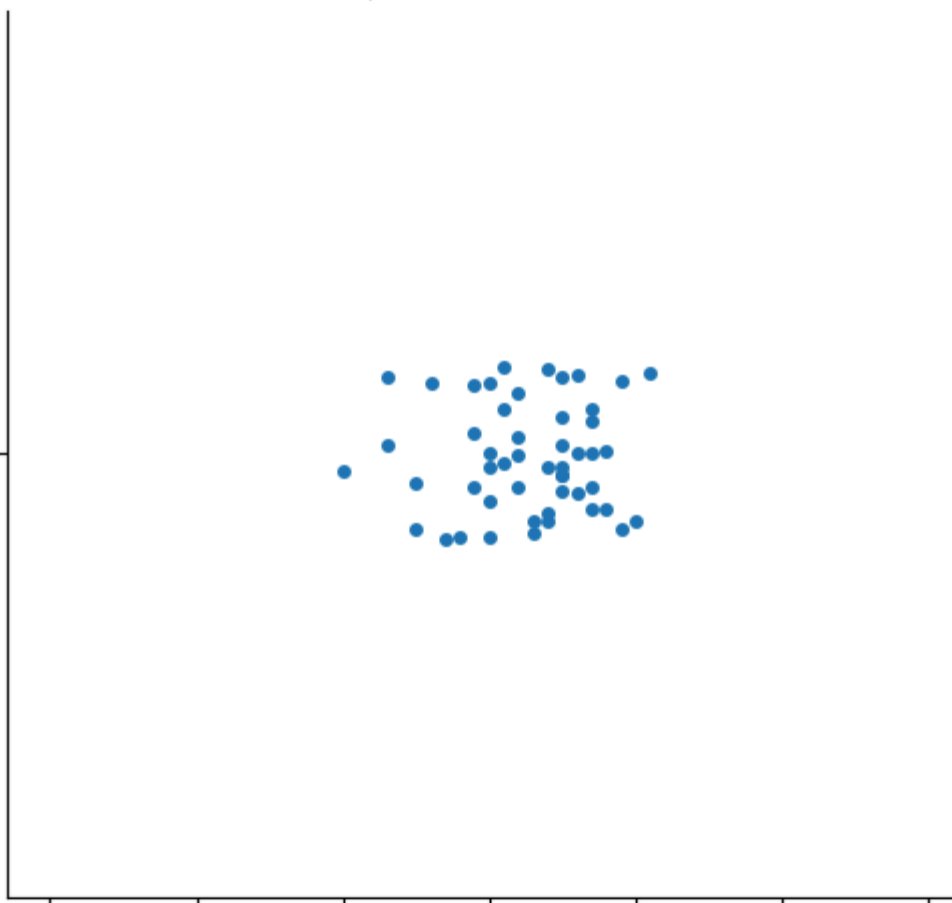
```
# sns.catplot(kind='strip', data=iris, x='petal_length', y='species')
sns.catplot(kind='strip', data=iris, x='petal_length', row='species')
```

```
plt.show()
```

species = setosa

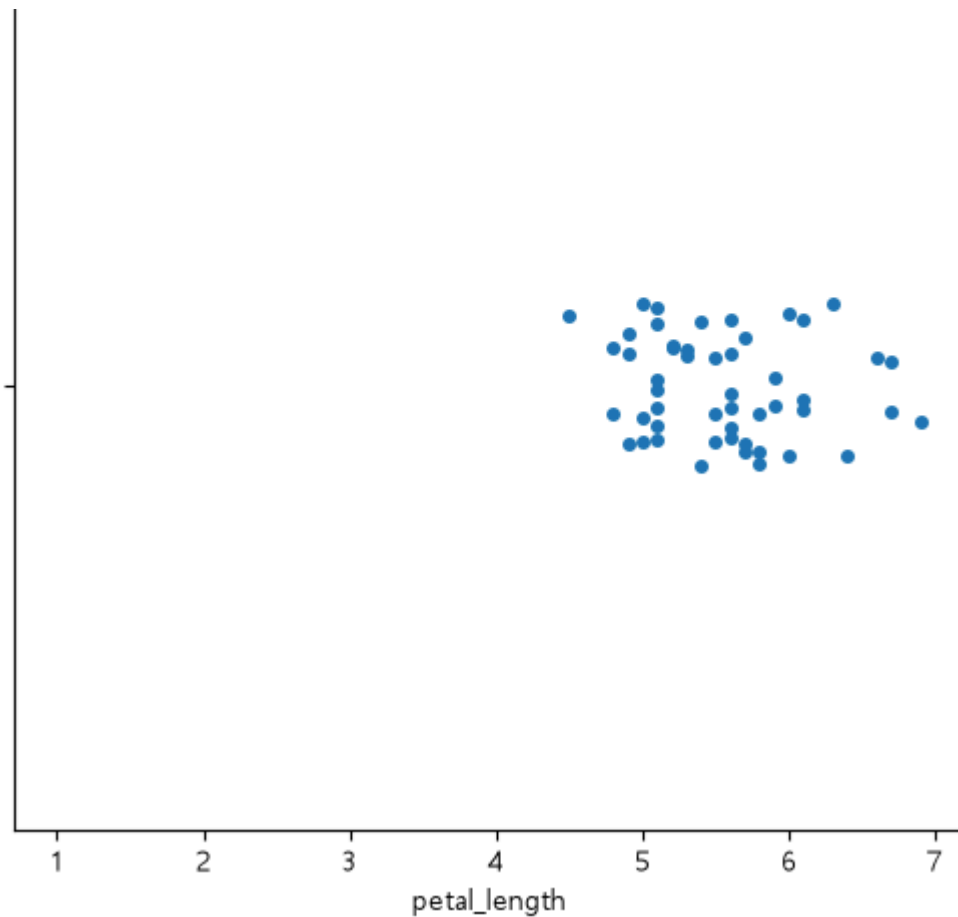


species = versicolor



species = virginica



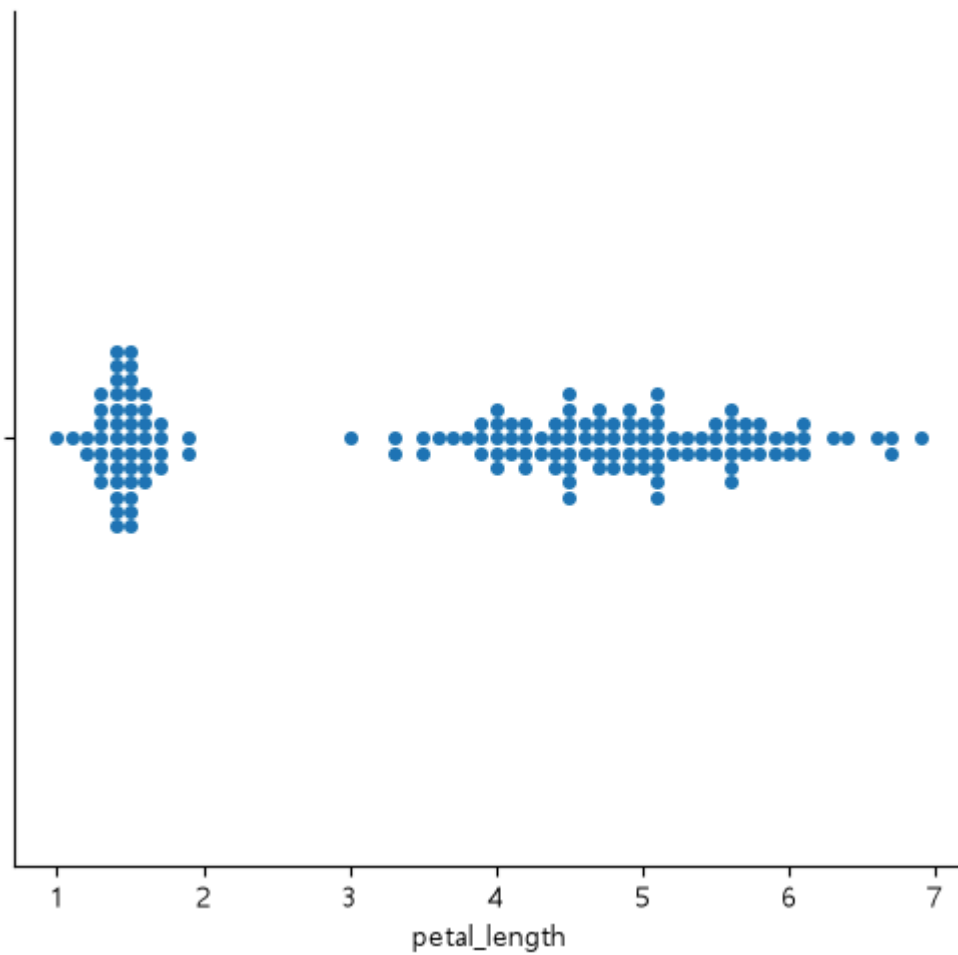


4) Swarm plot

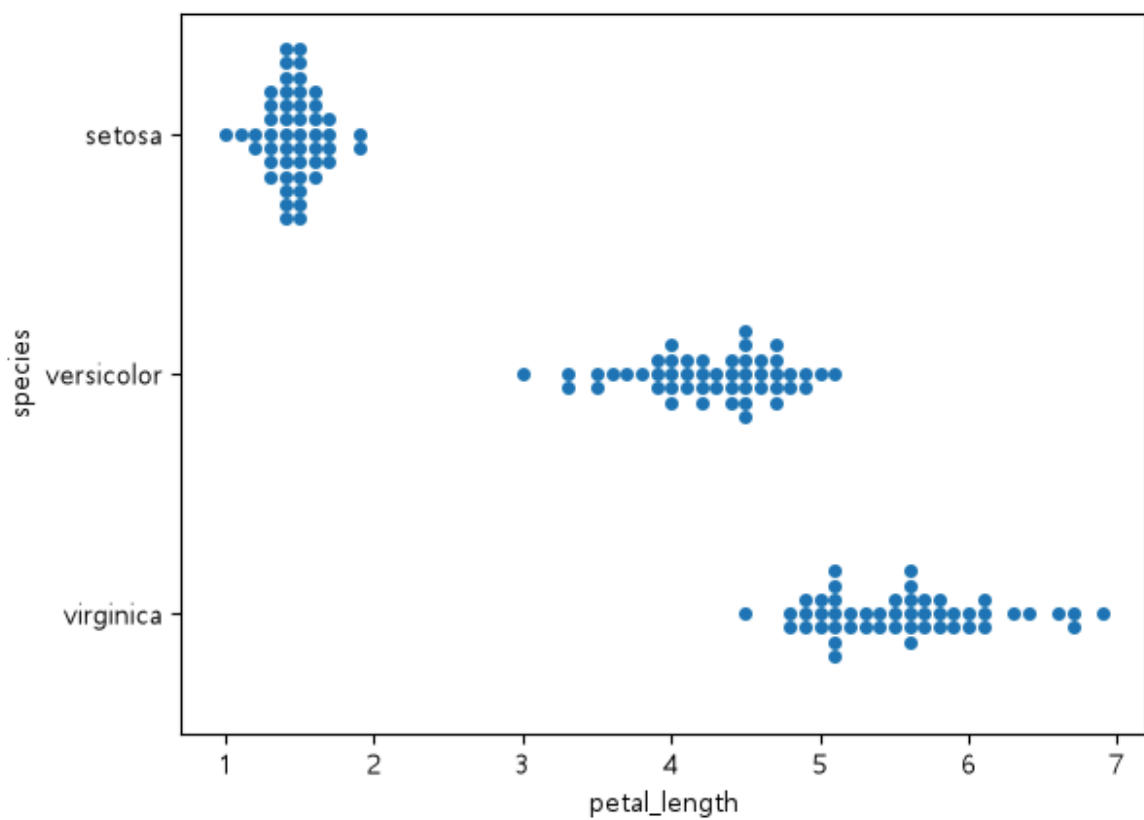
- 데이터의 분산까지 고려하여 겹치는 데이터를 겹치지 않게 그려주는 그래프
- 데이터가 퍼져 있는 정도를 입체적으로 볼 수 있음
- 형식

```
swarmplot(x=None, y=None, hue=None, data=None, order=None, ...)  
catplot(kind='swarm', x=None, y=None, hue=None, data=None,  
row=None, col=None, ...)
```
- <https://seaborn.pydata.org/generated/seaborn.swarmplot.html#seaborn.swarmplot>
- <https://seaborn.pydata.org/generated/seaborn.catplot.html#seaborn.catplot>

```
In [163... sns.catplot(kind='swarm', data=iris, x='petal_length')  
plt.show()
```

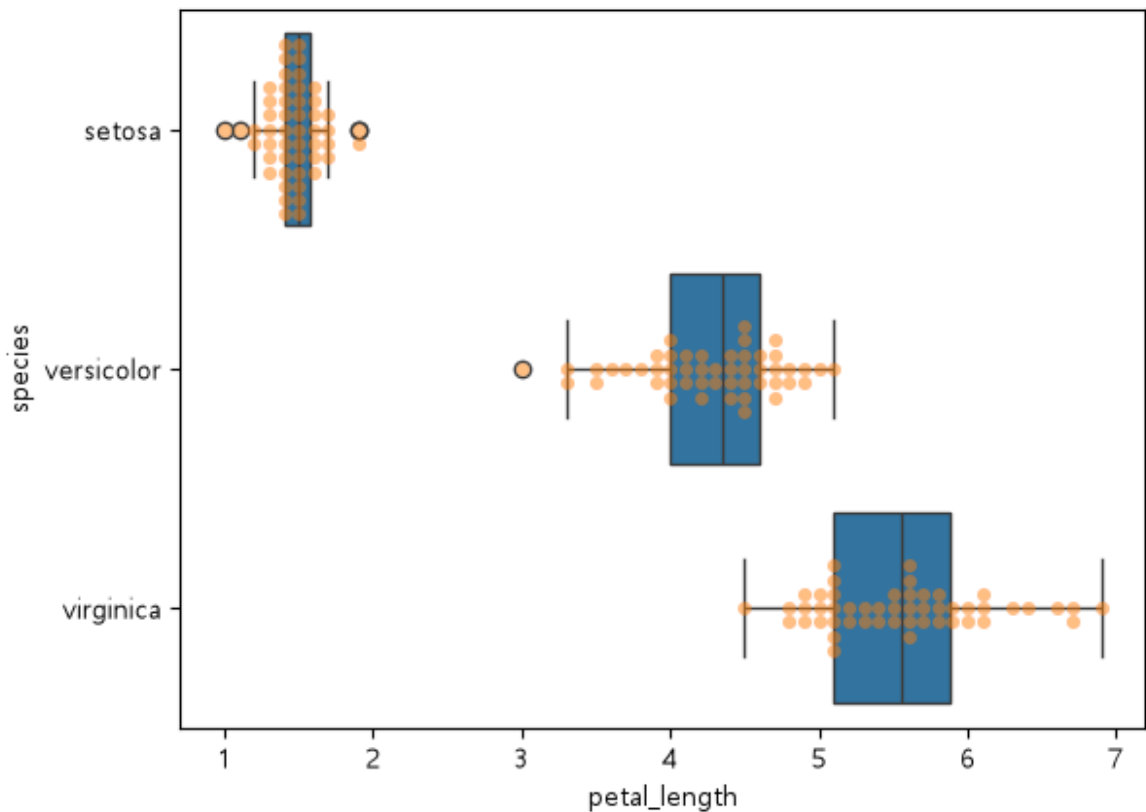


```
In [165... sns.swarmplot(data=iris, x='petal_length', y='species')  
plt.show()
```



box plot과 swarmplot을 같이 표시

```
In [166... sns.boxplot(data=iris, x='petal_length', y='species')
sns.swarmplot(data=iris, x='petal_length', y='species', alpha=0.5)
plt.show()
```



4. 시간에 따라 변화하는 데이터 표현

- 시계열 데이터 시각화
- 시간에 따른 추세 파악
- 선그래프(line plot)
 - matplotlib.pyplot.plot()
 - seaborn.lineplot()
 - seaborn.relplot(kind='line')
 - DataFrame.plot(kind='line')
 - DataFrame.plot.line()

예제 데이터. 미세먼지

```
In [168... # xlsx파일 입출력을 위한 openpyxl 설치
# xls 파일 입출력을 위한 xlrd 설치
# !pip install openpyxl
```

```
In [170... dust = pd.read_excel('data/fine_dust.xlsx', index_col='area')
dust.tail()
```

Out[170...

	2001	2002	2003	2004	2005	2006	2007	2008	2009	2010	2011
area											
Gangwon	49.0	58.0	57.0	59.0	62.0	58	55	56.0	53.0	51.0	50
Chungcheong	58.5	54.0	52.0	55.5	52.0	54	58	55.0	54.0	54.0	50
Jeolla	54.5	52.5	44.5	48.5	46.5	49	52	47.5	49.5	47.5	46
Gyeongsang	51.0	55.0	52.0	58.0	54.5	54	52	50.5	47.5	46.5	47
Jeju	46.0	45.0	37.0	43.0	45.0	49	44	43.0	42.0	48.0	42

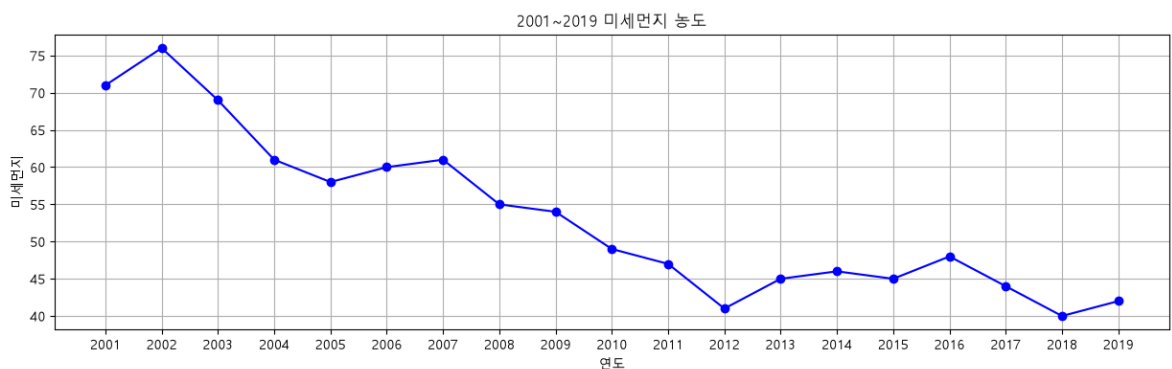
① matplotlib의 plot()을 이용하여 라인플롯 그리기

- 형식

```
plot(x, y, fmt, ..., data=None, scalex=True, scaley=True, **kwargs)
- x, y : array-like or scalar
- fmt : str, optional
```

In [177...

```
# 서울의 연도별 미세먼지 선그래프 그리기
plt.figure(figsize=(15,4))
plt.plot(dust.loc['Seoul'], color='b', marker='o')
# plt.plot(dust.columns, dust.loc['Seoul'], color='b', marker='o')
plt.title('2001~2019 미세먼지 농도')
plt.xlabel('연도')
plt.ylabel('미세먼지')
plt.xticks(range(2001,2020))
plt.grid()
plt.show()
```



plot(data=)

- indexable object : If given, provide the label names to plot in x and y.

In [181...

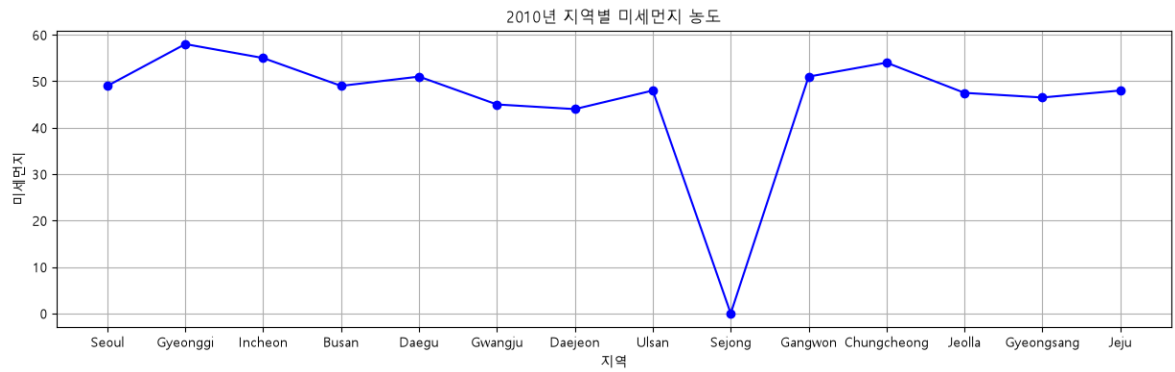
```
dust.columns = dust.columns.astype(str)
dust.columns
```

Out[181...

```
Index(['2001', '2002', '2003', '2004', '2005', '2006', '2007', '2008', '2009',
      '2010', '2011', '2012', '2013', '2014', '2015', '2016', '2017', '2018',
      '2019'],
      dtype='str')
```

In [185...

```
# 2010년 지역별 미세먼지 농도(비교)
plt.figure(figsize=(15,4))
plt.plot(dust.index, '2010', color='b', marker='o', data=dust)
# plt.plot(dust.columns, dust.loc['Seoul'], color='b', marker='o')
plt.title('2010년 지역별 미세먼지 농도')
plt.xlabel('지역')
plt.ylabel('미세먼지')
plt.grid()
plt.show()
```



In [186...

```
dust.T
```

Out[186...

	area	Seoul	Gyeonggi	Incheon	Busan	Daegu	Gwangju	Daejeon	Ulsan	Sejong	G
2001		71.0	71.0	52.0	60.0	67.0	57.0	48.0	0.0	0.0	
2002		76.0	74.0	57.0	69.0	71.0	52.0	53.0	54.0	0.0	
2003		69.0	68.0	61.0	55.0	59.0	36.0	43.0	40.0	0.0	
2004		61.0	67.0	62.0	60.0	0.0	46.0	49.0	50.0	0.0	
2005		58.0	65.0	61.0	58.0	55.0	49.0	48.0	50.0	0.0	
2006		60.0	68.0	68.0	59.0	54.0	55.0	49.0	52.0	0.0	
2007		61.0	66.0	64.0	57.0	53.0	52.0	49.0	53.0	0.0	
2008		55.0	60.0	57.0	51.0	57.0	50.0	45.0	54.0	0.0	
2009		54.0	60.0	60.0	49.0	48.0	46.0	43.0	49.0	0.0	
2010		49.0	58.0	55.0	49.0	51.0	45.0	44.0	48.0	0.0	
2011		47.0	56.0	55.0	47.0	47.0	43.0	44.0	49.0	0.0	
2012		41.0	49.0	47.0	43.0	42.0	38.0	39.0	46.0	0.0	
2013		45.0	54.0	49.0	49.0	45.0	42.0	42.0	47.0	0.0	
2014		46.0	54.0	49.0	48.0	45.0	41.0	41.0	46.0	0.0	
2015		45.0	53.0	53.0	46.0	46.0	43.0	46.0	46.0	0.0	
2016		48.0	53.0	49.0	44.0	43.0	40.0	44.0	43.0	46.0	
2017		44.0	51.0	46.0	44.0	42.0	40.0	45.0	43.0	47.0	
2018		40.0	44.0	40.0	41.0	39.0	41.0	44.0	40.0	40.0	
2019		42.0	46.0	43.0	36.0	39.0	42.0	42.0	37.0	44.0	



In [189...

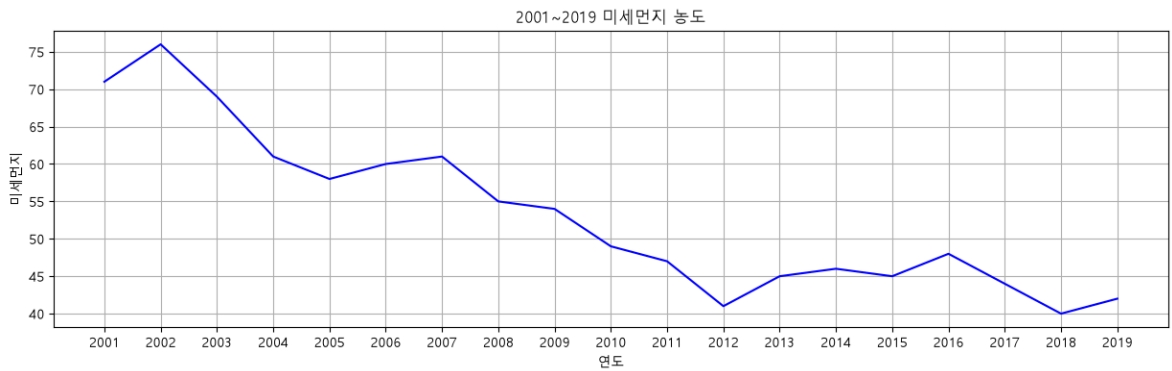
```
dust.columns = dust.columns.astype(int)
dust.columns
```

Out[189...

```
Index([2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012,
       2013, 2014, 2015, 2016, 2017, 2018, 2019],
      dtype='int64')
```

In [191...

```
plt.figure(figsize=(15,4))
plt.plot(dust.T.index, 'Seoul', color='b', data=dust.T)
plt.title('2001~2019 미세먼지 농도')
plt.xlabel('연도')
plt.ylabel('미세먼지')
plt.xticks(range(2001,2020))
plt.grid()
plt.show()
```



② seaborn의 선그래프 함수 이용

- 형식
`lineplot(*, x=None, y=None, hue=None, size=None, style=None, data=None, palette=None, ...)`
`relplot(kind='line', x=None, y=None, hue=None, size=None, style=None, data=None, row=None, col=None, ...)`
- <https://seaborn.pydata.org/generated/seaborn.lineplot.html#seaborn.lineplot>
- <https://seaborn.pydata.org/generated/seaborn.relplot.html#seaborn.relplot>

In [192... `dust.index`

Out[192... `Index(['Seoul', 'Gyeonggi', 'Incheon', 'Busan', 'Daegu', 'Gwangju', 'Daejeon', 'Ulsan', 'Sejong', 'Gangwon', 'Chungcheong', 'Jeolla', 'Gyeongsang', 'Jeju'], dtype='str', name='area')`

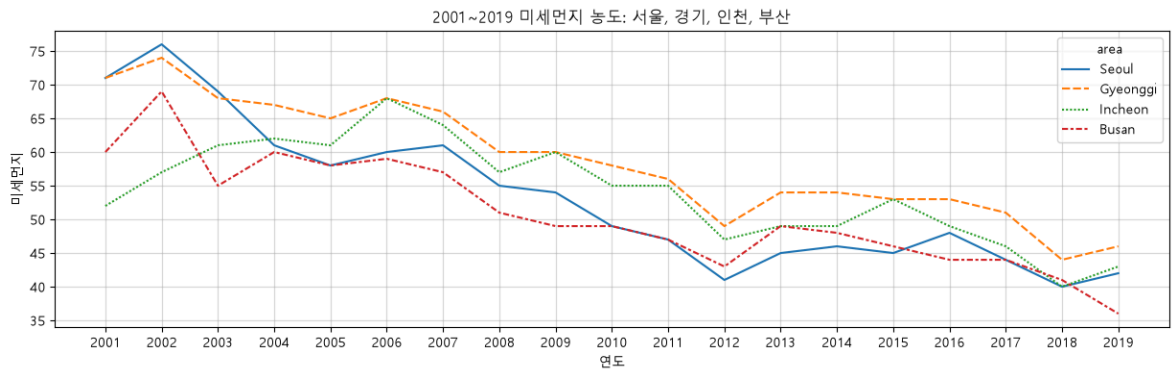
In [195... `data = dust.loc[['Seoul', 'Gyeonggi', 'Incheon', 'Busan']].T`
`data.tail()`

Out[195...

	area	Seoul	Gyeonggi	Incheon	Busan
2015		45.0	53.0	53.0	46.0
2016		48.0	53.0	49.0	44.0
2017		44.0	51.0	46.0	44.0
2018		40.0	44.0	40.0	41.0
2019		42.0	46.0	43.0	36.0

`sns.lineplot()`

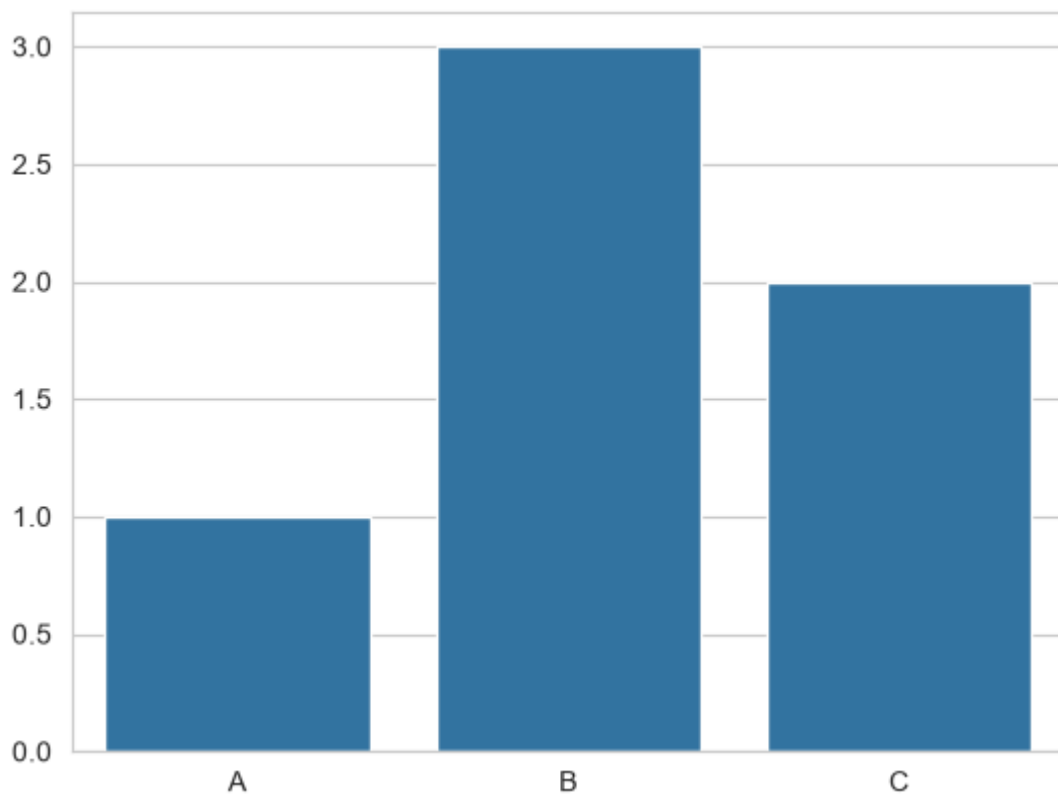
In [197... `plt.figure(figsize=(15,4))`
`sns.lineplot(data=data)`
`plt.title('2001~2019 미세먼지 농도: 서울, 경기, 인천, 부산')`
`plt.xlabel('연도')`
`plt.ylabel('미세먼지')`
`plt.xticks(range(2001,2020))`
`plt.grid(alpha=0.5)`
`plt.show()`



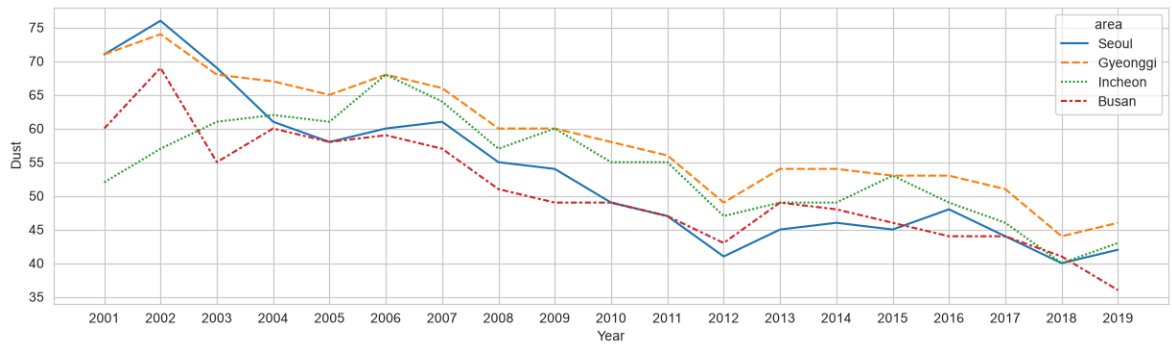
seaborn의 `set_style()` 이용하여 라인플롯 그리기

- `set_style()`
 - https://seaborn.pydata.org/generated/seaborn.set_style.html
- `axes_style()`
 - https://seaborn.pydata.org/generated/seaborn.axes_style.html

```
In [200... sns.set_style("whitegrid")
sns.barplot(x=["A", "B", "C"], y=[1, 3, 2])
plt.show()
```

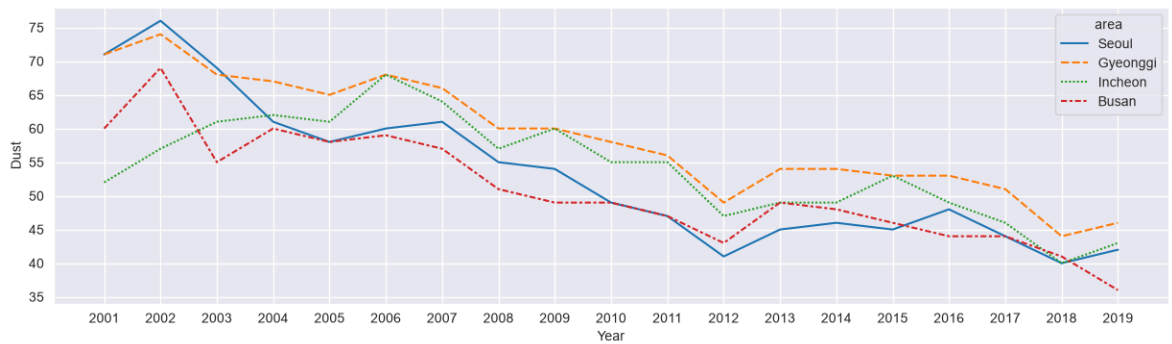


```
In [203... # set_style을 지정한 경우 폰트와 색상 등의 스타일 값들이 변경되므로 한글코드가 깨
plt.figure(figsize=(15,4))
sns.lineplot(data=data)
plt.xlabel('Year')
plt.ylabel('Dust')
plt.xticks(range(2001,2020))
plt.show()
```

In [204...

```
sns.set_style('darkgrid')
plt.figure(figsize=(15,4))
sns.lineplot(data=data)
plt.xlabel('Year')
plt.ylabel('Dust')
plt.xticks(range(2001,2020))
plt.show()
```

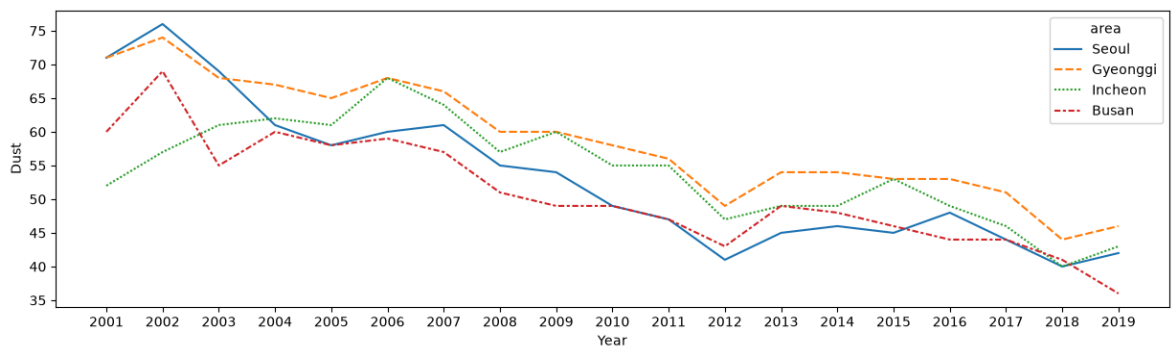


sns.despine() 함수를 이용한 그림 테두리 제거

sns.despine(right=True,left=True,top=False,bottom=False)

In [205...

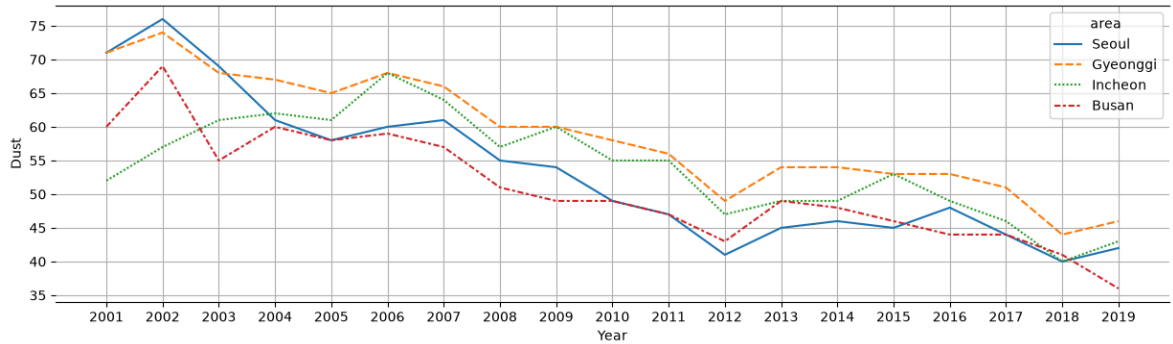
```
plt.rcParams() # 설정 초기화
plt.figure(figsize=(15,4))
sns.lineplot(data=data)
plt.xlabel('Year')
plt.ylabel('Dust')
plt.xticks(range(2001,2020))
plt.show()
```



In [207...

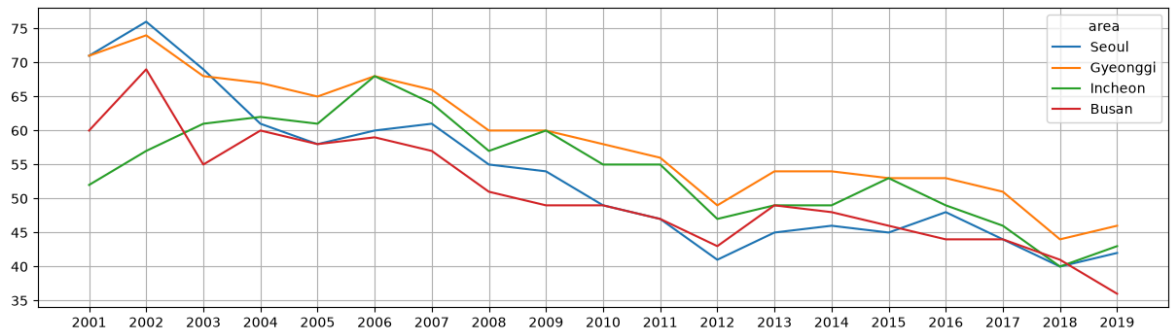
```
plt.figure(figsize=(15,4))
sns.lineplot(data=data)
plt.xlabel('Year')
plt.ylabel('Dust')
plt.xticks(range(2001,2020))
```

```
sns.despine(right=True, left=True, top=False, bottom=False)
plt.grid()
plt.show()
```



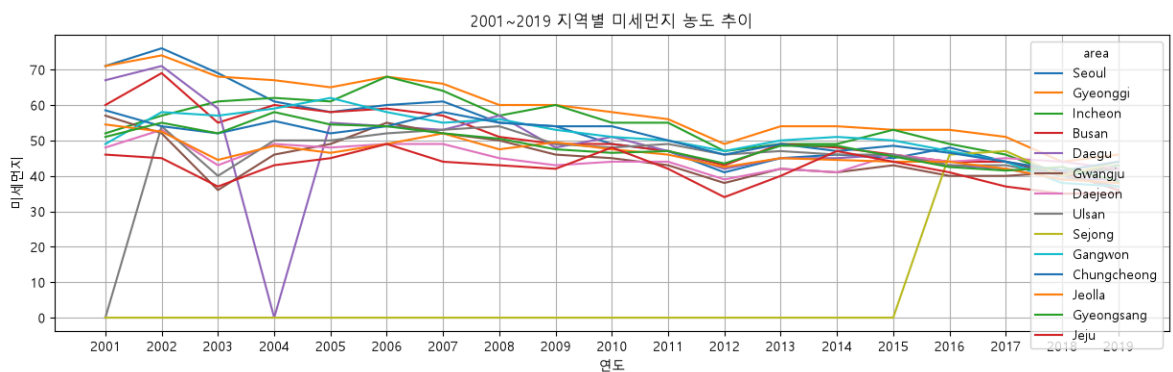
③ pandas.DataFrame.plot(kind='line') 이용

```
In [209... data.plot(figsize=(15,4))
plt.xticks(range(2001,2020))
plt.grid()
plt.show()
```



```
In [210... plt.rcParams['axes.unicode_minus']=False
plt.rcParams['font.family']='Malgun Gothic'
```

```
In [212... dust.T.plot(figsize=(15,4), xlabel='연도', ylabel='미세먼지', title='2001~2019 지
plt.xticks(range(2001,2020))
plt.grid()
plt.show()
```



5. 히트맵(heatmap)

(1) sns.heatmap()

- 범주형-범주형-수치형 자료의 시각화
 - X축과 Y축에 2개의 범주형 자료의 계급(class)별로 연속형 자료를 집계한 자료의 시각화
- 집계한 값에 비례하여 색깔을 다르게 해서 2차원으로 자료를 시각화
- 형식

```
heatmap(data, vmin=None, vmax=None, cmap=None, center=None, robust=False, annot=None,
        fmt='.2g', annot_kws=None, linewidths=0, linecolor='white',
        cbar=True,
        cbar_kws=None, cbar_ax=None, square=False,
        xticklabels='auto',
        yticklabels='auto', mask=None, ax=None, **kwargs)
```
- <https://seaborn.pydata.org/generated/seaborn.heatmap.html#seaborn.heatmap>

예제 데이터

- 캐글 자전거수요예측 데이터
- <https://www.kaggle.com/competitions/bike-sharing-demand>

In [213...

```
bike = pd.read_csv('data/bike.csv')
bike.head()
```

Out[213...

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspe
0	2011-01-01 0:00	1	0	0	1	9.84	14.395	81	C
1	2011-01-01 1:00	1	0	0	1	9.02	13.635	80	C
2	2011-01-01 2:00	1	0	0	1	9.02	13.635	80	C
3	2011-01-01 3:00	1	0	0	1	9.84	14.395	75	C
4	2011-01-01 4:00	1	0	0	1	9.84	14.395	75	C



In [214...

```
bike.tail()
```

Out[214...

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	wind
10881	2012-12-19 19:00	4	0	1	1	15.58	19.695	50	2
10882	2012-12-19 20:00	4	0	1	1	14.76	17.425	57	1
10883	2012-12-19 21:00	4	0	1	1	13.94	15.910	61	1
10884	2012-12-19 22:00	4	0	1	1	13.94	17.425	61	
10885	2012-12-19 23:00	4	0	1	1	13.12	16.665	66	



- datetime : hourly date + timestamp (날짜와 시간)
- season : 1 = spring, 2 = summer, 3 = fall, 4 = winter (계절)
- holiday : whether the day is considered a holiday (휴일)
- workingday ; whether the day is neither a weekend nor holiday (평일)
- weather
 - 1: Clear, Few clouds, Partly cloudy, Partly cloudy (맑은 날씨)
 - 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist (안개)
 - 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds (가벼운 눈, 비)
 - 4: Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog (폭설, 폭우)
- temp : temperature in Celsius (온도)
- atemp : "feels like" temperature in Celsius (체감 온도)
- humidity : relative humidity (상대 습도)
- windspeed : wind speed (풍속)
- casual : number of non-registered user rentals initiated (비회원 대여량)
- registered : number of registered user rentals initiated (회원 대여량)
- count : number of total rentals (총 대여량)

In [215...

```
bike.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   datetime    10886 non-null  str
1   season      10886 non-null  int64
2   holiday     10886 non-null  int64
3   workingday  10886 non-null  int64
4   weather     10886 non-null  int64
5   temp        10886 non-null  float64
6   atemp       10886 non-null  float64
7   humidity    10886 non-null  int64
8   windspeed   10886 non-null  float64
9   casual      10886 non-null  int64
10  registered  10886 non-null  int64
11  count       10886 non-null  int64
dtypes: float64(3), int64(8), str(1)
memory usage: 1020.7 KB

```

datetime 컬럼을 연,월,일,시간으로 분리

```

In [229... bike['datetime'] = pd.to_datetime(bike.datetime)
bike.info()

```

```

<class 'pandas.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0   datetime    10886 non-null  datetime64[us]
1   season      10886 non-null  int64
2   holiday     10886 non-null  int64
3   workingday  10886 non-null  int64
4   weather     10886 non-null  int64
5   temp        10886 non-null  float64
6   atemp       10886 non-null  float64
7   humidity    10886 non-null  int64
8   windspeed   10886 non-null  float64
9   casual      10886 non-null  int64
10  registered  10886 non-null  int64
11  count       10886 non-null  int64
12  date        10886 non-null  str
13  time        10886 non-null  str
dtypes: datetime64[us](1), float64(3), int64(8), str(2)
memory usage: 1.2 MB

```

```

In [230... bike['year'] = bike['datetime'].dt.year
bike['month'] = bike['datetime'].dt.month
bike['day'] = bike['datetime'].dt.day
bike['hour'] = bike['datetime'].dt.hour
bike.head(3)

```

Out[230...

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspe
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	C
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	C
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	C



In [232...

```
# bike = pd.read_csv('data/bike.csv')
# bike[['date', 'time']] = bike['datetime'].str.split(expand=True)
# bike.head()
# bike['time'].str.split(':')
```

In [235...

```
bike['day'].value_counts().sort_index()
```

Out[235...

```
day
1    575
2    573
3    573
4    574
5    575
6    572
7    574
8    574
9    575
10   572
11   568
12   573
13   574
14   574
15   574
16   574
17   575
18   563
19   574
Name: count, dtype: int64
```

hour를 시간대로 범주화

- 변수명은 'hours'
- 시간대는

```
labels=['0~2', '3~5', '6~8', '9~11', '12~14', '15~17', '18~20', '21~23']
```

In [236...

```
labels=['0~2', '3~5', '6~8', '9~11', '12~14', '15~17', '18~20', '21~23']
bins = [-0.1, 2, 5, 8, 11, 14, 17, 20, 23]

bike['hours'] = pd.cut(bike['hour'], bins=bins, labels=labels)
bike.hours.value_counts().sort_index()
```

```
Out[236...] hours
0~2      1357
3~5      1327
6~8      1365
9~11     1365
12~14    1368
15~17    1368
18~20    1368
21~23    1368
Name: count, dtype: int64
```

```
In [238...] bike.season.value_counts().sort_index()
```

```
Out[238...] season
1      2686
2      2733
3      2733
4      2734
Name: count, dtype: int64
```

sns.heatmap() 사용하여 계절(season)과 시간대별(hours) 자전거 총대여량 시각화

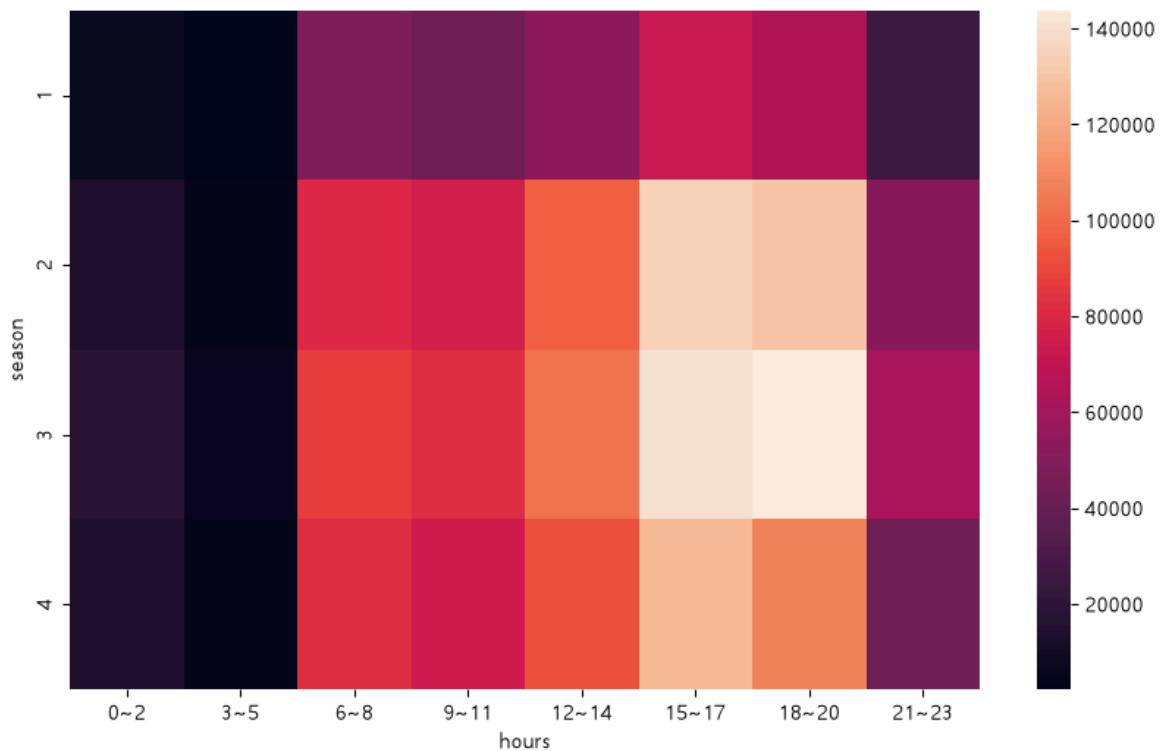
데이터 피봇팅

- pivot() 함수
`pd.pivot(data, index=None, columns=None, values=None)`
 데이터프레임 `.pivot(index=None, columns=None, values=None)`
- pivot_table() 함수
`pd.pivot_table(data, values=None, index=None, columns=None, aggfunc='mean', fill_value=None, margins=False, dropna=True, margins_name='All', observed=False, sort=True)`
 데이터프레임 `.pivot_table( )`
- <https://pandas.pydata.org/docs/reference/api/pandas.pivot.html>
- https://pandas.pydata.org/docs/reference/api/pandas.pivot_table.html

```
In [240...] bike_season_hour = pd.pivot_table(data=bike, index='season',
                                             columns='hours', values='count', aggfunc='sum')
bike_season_hour
```

```
Out[240...] hours    0~2    3~5    6~8    9~11   12~14   15~17   18~20   21~23
season
1      6730   2215   47517   42851   53175   71955   63711   24344
2     13406   4384   80420   76620   96669  135077  129817   51889
3     17276   5638   87133   82778  102348  140410  143687   61392
4     13307   4621   81656   74185   93337  126541  107228   43159
```

```
In [242... plt.figure(figsize=(10,6))
sns.heatmap(bike_season_hour)
plt.show()
```

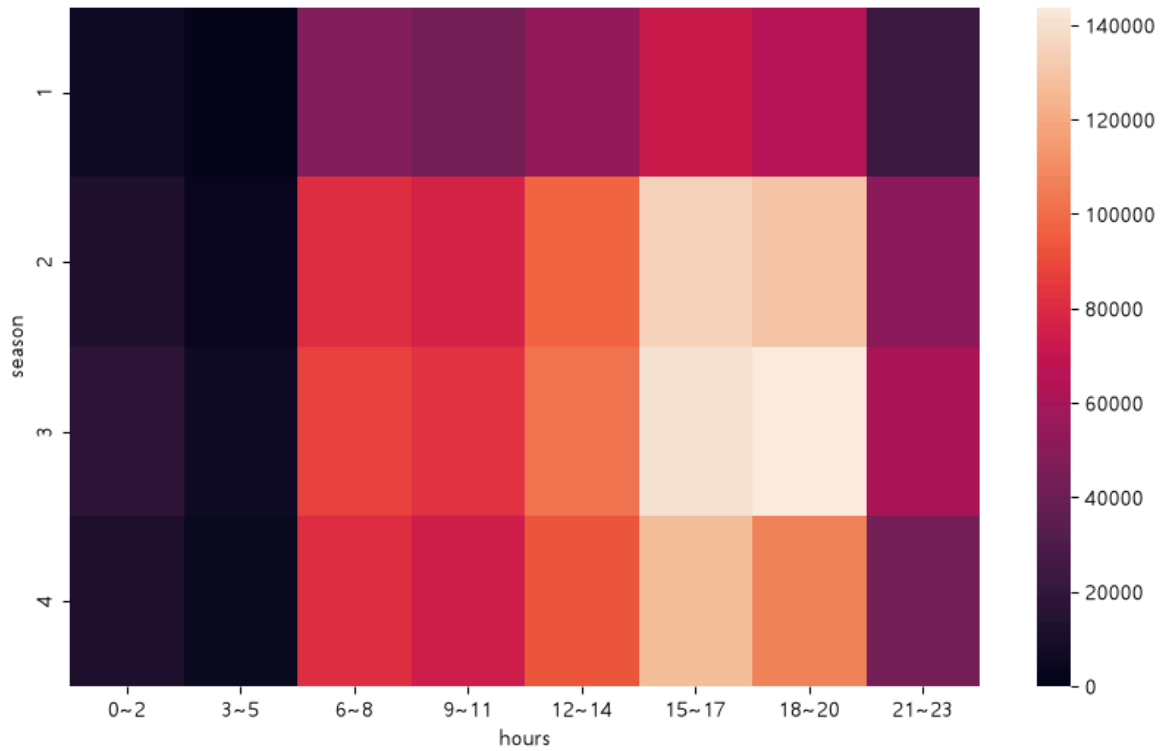


heatmap()의 색상 스케일 최소값(vmin), 최대값(vmax) 인수 사용

```
In [247... bike_season_hour.max().max(), bike_season_hour.min().min())
```

```
Out[247... (np.int64(143687), np.int64(2215))
```

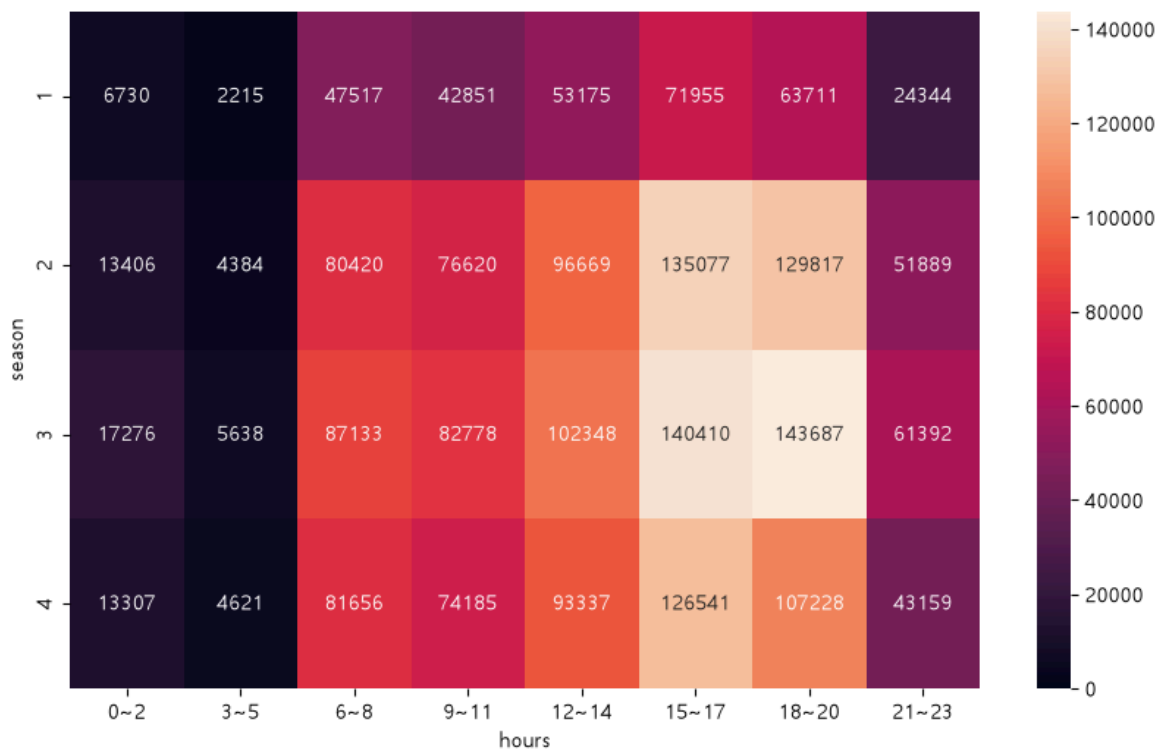
```
In [246... plt.figure(figsize=(10,6))
sns.heatmap(bike_season_hour, vmin=0, vmax=bike_season_hour.max().max())
plt.show()
```

heatmap()의 annot, fmt 인수

- annot=True : 히트맵의 각 셀에 값 표시
- fmt='d' or 'f' : 히트맵에 표시하는 값의 서식(format) 지정 (정수, 실수)

```
In [251...] plt.figure(figsize=(10,6))
sns.heatmap(bike_season_hour, vmin=0, vmax=bike_season_hour.max().max(),
            annot=True, fmt='d') # fmt='.1f'
plt.show()
```



```
In [252...] bike_season_hour_mean = pd.pivot_table(data=bike, index='season',
                                                    columns='hours', values='count', aggfunc='mean')
```

```
bike_season_hour_mean
```

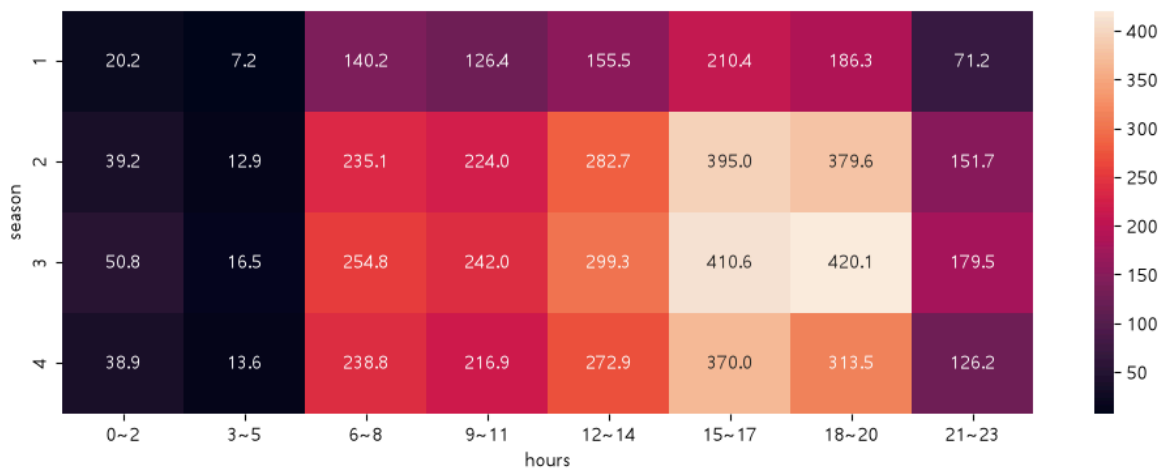
Out[252...

	hours	0~2	3~5	6~8	9~11	12~14	15~17	18~20	21~23
season									
1	20.2	7.2	140.2	126.4	155.5	210.4	186.3	71.2	
2	39.2	12.9	235.1	224.0	282.7	395.0	379.6	151.7	
3	50.8	16.5	254.8	242.0	299.3	410.6	420.1	179.5	
4	38.9	13.6	238.8	216.9	272.9	370.0	313.5	126.2	



In [255...

```
plt.figure(figsize=(12,4))
sns.heatmap(bike_season_hour_mean, annot=True, fmt='.1f')
plt.show()
```

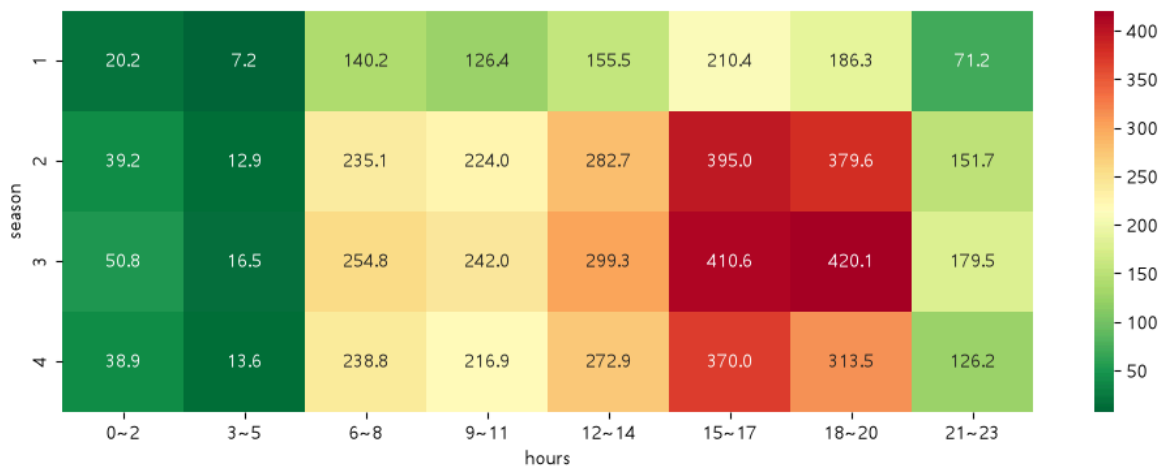


sns.heatmap()함수의 cmap인수 사용하여 다른 color map 지정

- color maps in Matplotlib 링크 참고:
- <https://matplotlib.org/stable/tutorials/colors/colormaps.html>

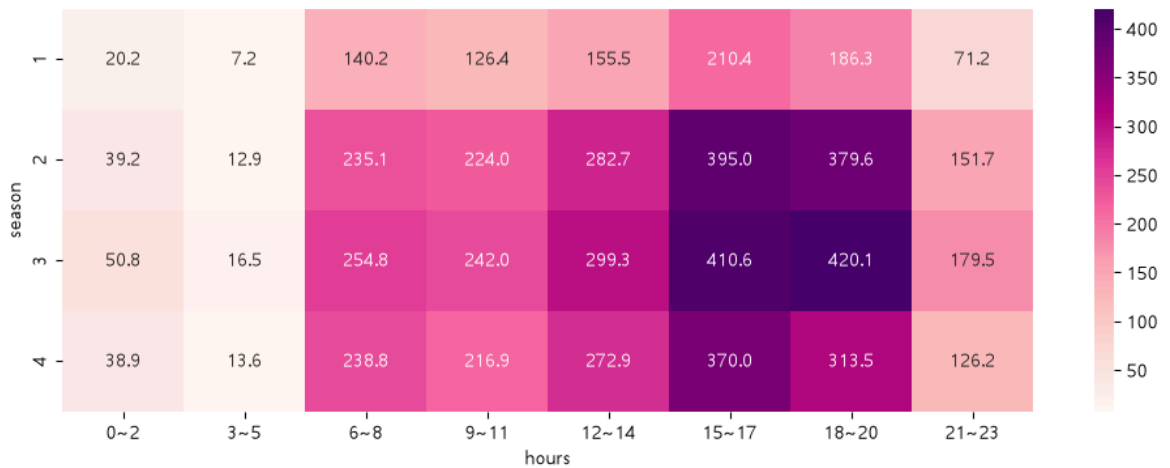
In [256...

```
plt.figure(figsize=(12,4))
sns.heatmap(bike_season_hour_mean, annot=True, fmt='.1f', cmap='RdYlGn_r')
plt.show()
```



In [258...

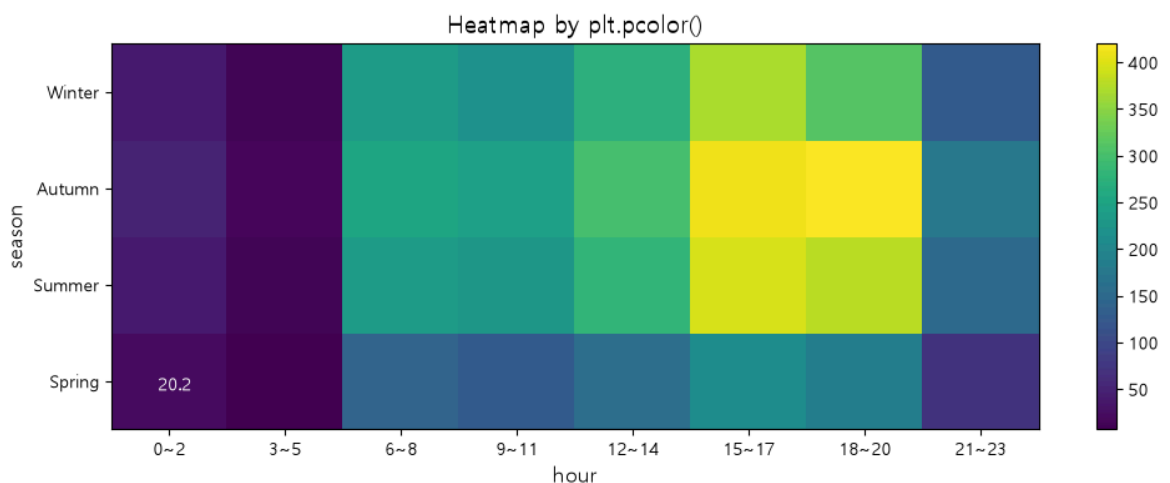
```
plt.figure(figsize=(12,4))
sns.heatmap(bike_season_hour_mean, annot=True, fmt='.1f', cmap='RdPu') # cmap='
plt.show()
```



(2) matplotlib의 pcolor()를 이용한 heatmap 그리기

In [271...

```
col_n = len(bike_season_hour_mean.columns)
index_n = len(bike_season_hour_mean.index)
plt.figure(figsize=(12,4))
plt.pcolor(bike_season_hour_mean)
plt.xticks(np.arange(0.5, col_n, 1), bike_season_hour_mean.columns)
plt.yticks(np.arange(0.5, index_n, 1), ['Spring', 'Summer', 'Autumn', 'Winter'])
plt.title('Heatmap by plt.pcolor()', fontsize=14)
plt.xlabel('hour', fontsize=12)
plt.ylabel('season', fontsize=12)
plt.colorbar()
plt.text(0.5, 0.5, f'{bike_season_hour_mean.iloc[0,0]:.1f}', c='white')
plt.show()
```



예제 데이터

- seaborn 패키지의 flights dataset

```
In [273... flights = sns.load_dataset('flights')
flights.head()
```

```
Out[273...   year month passengers
0  1949   Jan         112
1  1949   Feb         118
2  1949   Mar         132
3  1949   Apr         129
4  1949   May         121
```

```
In [274... flights.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 144 entries, 0 to 143
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   year        144 non-null    int64
1   month       144 non-null    category
2   passengers  144 non-null    int64
dtypes: category(1), int64(2)
memory usage: 2.9 KB
```

heatmap을 그리기 위한 데이터 테이블 생성

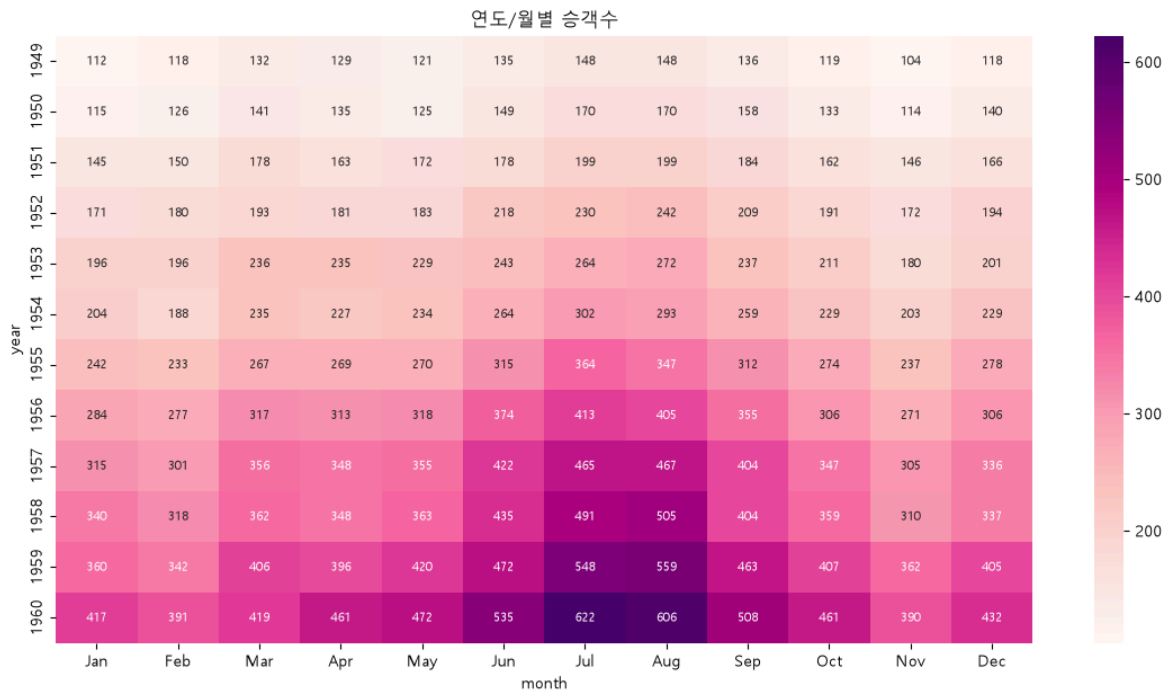
: pivot 테이블을 이용해서 연도별 각 월 승객수로 구조를 변환

```
In [277... flights_p = pd.pivot(data=flights, index='year', columns='month', values='passen
flights_p.head()
```

```
Out[277...   month Jan  Feb  Mar  Apr  May  Jun  Jul  Aug  Sep  Oct  Nov  Dec
year
1949  112  118  132  129  121  135  148  148  136  119  104  118
1950  115  126  141  135  125  149  170  170  158  133  114  140
1951  145  150  178  163  172  178  199  199  184  162  146  166
1952  171  180  193  181  183  218  230  242  209  191  172  194
1953  196  196  236  235  229  243  264  272  237  211  180  201
```

seaborn.heatmap() 그리기

```
In [282... plt.figure(figsize=(14,7))
sns.heatmap(data=flights_p, annot=True, fmt='d', annot_kws={'size':8}, cmap='RdP
plt.title('연도/월별 승객수')
plt.show()
```



6. 모자이크(Mosaic) 플롯

In [294...]

```
titanic.tail()
```

Out[294...]

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	...
886	0	2	male	27.0	0	0	13.00	S	Second	man	
887	1	1	female	19.0	0	0	30.00	S	First	woman	
888	0	3	female	NaN	1	2	23.45	S	Third	woman	
889	1	1	male	26.0	0	0	30.00	C	First	man	
890	0	3	male	32.0	0	0	7.75	Q	Third	man	

In [297...]

```
titanic = titanic.replace({'survived':{0:'사망', 1:'생존'}})
titanic = titanic.replace({'sex':{'male':'남성', 'female':'여성'}})
titanic.tail()
```

Out[297...

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adu
886	사망	2	남성	27.0	0	0	13.00	S	Second	man	
887	생존	1	여성	19.0	0	0	30.00	S	First	woman	
888	사망	3	여성	NaN	1	2	23.45	S	Third	woman	
889	생존	1	남성	26.0	0	0	30.00	C	First	man	
890	사망	3	남성	32.0	0	0	7.75	Q	Third	man	



In [295...

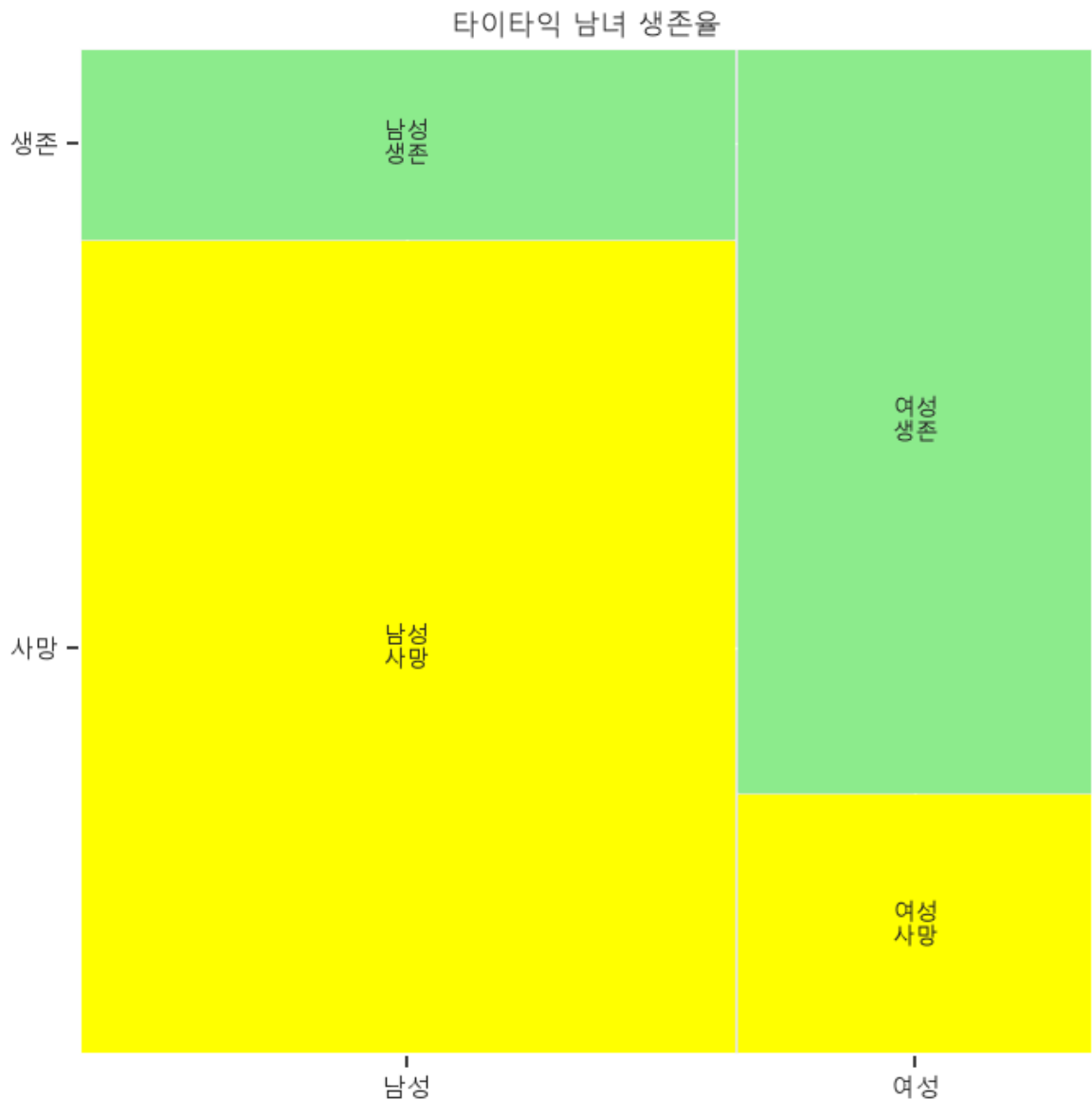
```
# statsmodels 설치 필요
# !pip install statsmodels
```

In [301...

```
from statsmodels.graphics.mosaicplot import mosaic

def props(key):
    return {'color':'lightgreen' if '생존' in key else 'yellow'}

plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False
mosaic(data=titanic.sort_values('sex'),
       index=['sex','survived'],
       properties=props, # tile의 색상 변경
       axes_label = True, title='타이타닉 남녀 생존율')
plt.show()
```



7. 레이더 차트

```
In [ ]: # plotly 설치
# !pip install plotly
# !pip install kaleido
```

```
In [1]: import matplotlib.image as mpimg
import plotly.graph_objects as go
import pandas as pd
import matplotlib.pyplot as plt

def radar(df, fills, min_max, title=''):
    fig = go.Figure()
    categories = df.columns.to_list()
    categories.append(categories[0])
    i = 0
    while (i < len(df)):
        scores = df.iloc[i, :].to_list()
        scores.append(scores[0])
        fig.add_trace(go.Scatterpolar(
            r=scores, # 축의 값
            theta=categories, # 축의 레이블
```

```

        fill=fills[i], # 다각형 채우기 색
        name=df.index[i] # 다각형 레이블
    ))
    i += 1

fig.update_layout(
    polar_radialaxis_visible=True,
    polar_radialaxis_range=min_max, # 축의 값 범위
    showlegend=True,
    margin_t=50, # 상단 여백
    margin_l=100, # 좌측 여백
    margin_r=100, # 우측 여백
    margin_b=25, # 하단 여백
    width=700, # 그래프의 폭(pixel)
    height=700, # 그래프의 높이(pixel)
    title_text=title, # 그래프 제목
    title_font_size=30, # 제목 폰트 사이즈
    font_size=20 # 폰트 사이즈
)

# 그래프 저장 & display
plt.axis('off')
fig.write_image('rader.png')
plt.imshow(mping.imread('rader.png'))
plt.show()

```

```

In [2]: # 예제 데이터
df = pd.DataFrame({
    'Kor': [72, 70, 90, 60, 66],
    'Eng': [84, 85, 95, 70, 85],
    'Math': [71, 40, 88, 80, 75],
    'Sci': [83, 80, 91, 90, 70],
    'Phy': [60, 60, 60, 70, 50]
})
df.index = ['AVG', 'John', 'Tom', 'Smith', 'Grace']
df

```

```

Out[2]:

```

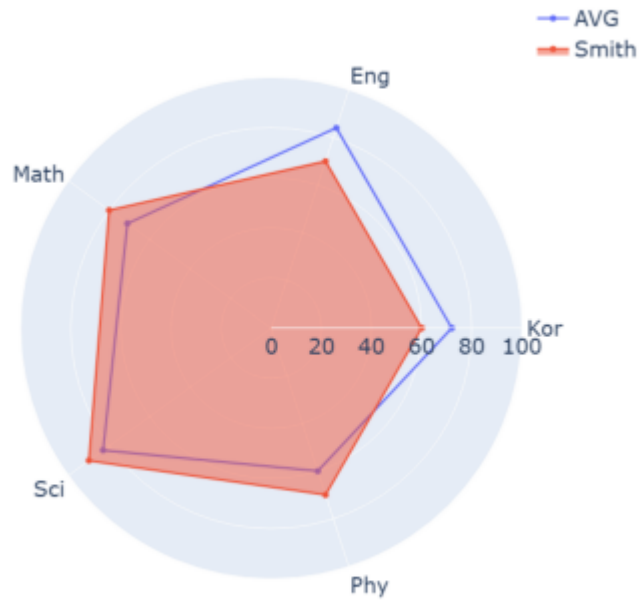
	Kor	Eng	Math	Sci	Phy
AVG	72	84	71	83	60
John	70	85	40	80	60
Tom	90	95	88	91	60
Smith	60	70	80	90	70
Grace	66	85	75	70	50

```

In [3]: fills = [None, 'toself']
radar(df=df.iloc[[0, 3], :],
      fills=fills,
      min_max=[0, 100],
      title='Scores of Smith'
    )

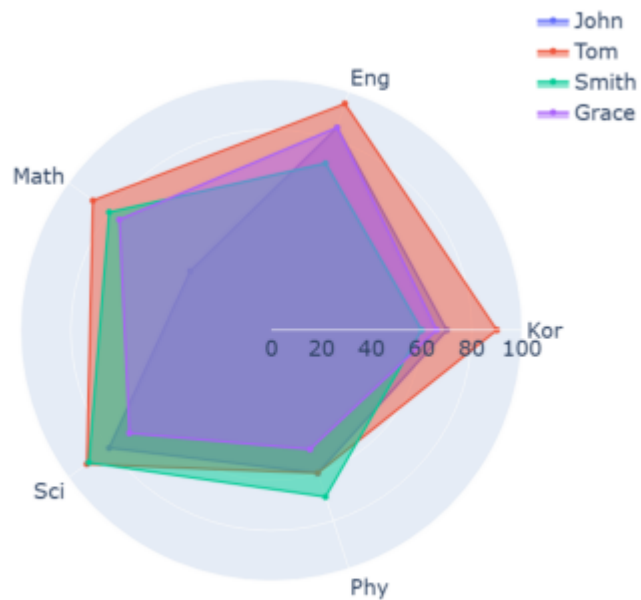
```


Scores of Smith



```
In [4]: fills = ['toself', 'toself', 'toself', 'toself']
        radar(df=df.iloc[1:, :],
              fills=fills,
              min_max=[0, 100],
              title='Scores of Student'
              )
```

Scores of Student



8. 트리 맵(Tree Map)

```
In [5]: import plotly.express as px
import webbrowser
import matplotlib.pyplot as plt
import pandas as pd
```

```
In [6]: df = pd.read_csv('data/GNI2014.csv')
df.head()
```

```
Out[6]:
```

	iso3	country	continent	population	GNI
0	BMU	Bermuda	North America	67837	106140
1	NOR	Norway	Europe	4676305	103630
2	QAT	Qatar	Asia	833285	92200
3	CHE	Switzerland	Europe	7604467	88120
4	MAC	Macao SAR, China	Asia	559846	76270

```
In [7]: fig = px.treemap(data_frame=df,
                        path=['continent', 'iso3'], # 데이터의 계층 구조
                        values='population', # 타일 면적 기준 컬럼
                        color='GNI', # 색 온도 기준 컬럼
                        color_continuous_scale='Bluy1' # 컬러 팔레트
                        )
```

```
In [9]: plt.axis('off') # 축 눈금 제거
fig.update_layout(margin_t=50, margin_l=25, # 여백 설정
                  margin_r=25, margin_b=25,
                  width=800, # 그래프의 폭(pixel)
                  height=600, # 그래프의 높이(pixel)
                  title_text='GNI 2014', # 그래프 제목
                  title_font_size=20 # 제목 폰트 크기
                  )
```

```
In [10]: # 그래프 저장 & 화면에 표시하기  
fig.write_html('treemap.html') # html 파일로 결과 저장  
webbrowser.open('treemap.html') # 웹 브라우저에서 그래프 확인
```

Out[10]: True

In []: